학습 교안

하네스 엔지니어링 입문

AI 에이전트에게 맡기고, 이해하고, 검증해서 저장하는 메모 CRUD 프로젝트

Docker · OpenRouter · OpenCode · Python · FastAPI · PostgreSQL · pytest · Git

3일 × 8시간 · 12교시 · 초보자 대상 실습 과정

강사 김대철

작성일 2026.11.10

실습 환경 Windows + Docker Desktop (모든 도구는 컨테이너 안에서 실행)

실습 저장소 https://github.com/sinaboro/harness-crud-lab

교안의 API 응답 · 테스트 결과 · SQL 결과는 예제 코드를 실제로 실행해 얻은 것입니다.

AI 의 답변과 AI 가 만든 코드는 실행할 때마다 달라지므로, 결과 대신 확인 방법을 실었습니다.

목차

과정 안내 . . . 7

1. 바이브 코딩과 하네스 엔지니어링 . . . 7
2. 수업의 흐름 . . . 8
3. 준비물 . . . 8
4. 시작하는 법 . . . 8
5. 교시표 . . . 9
6. 실행 환경 한눈에 보기 . . . 9
7. 저장소 구조 . . . 10
8. 사용 도구와 선택 이유 . . . 11
9. 이 교안의 명령과 결과에 대하여 . . . 11
10. 만든 사람 . . . 11

1교시. AI 개발 방식과 하네스 이해 . . . 12

수업의 흐름에서 이번 교시의 위치 . . . 12
학습 목표 . . . 12
1. 우리가 3일 동안 만들 것 . . . 12
2. 네 가지 낱말 구분하기 . . . 12
3. 바이브 코딩 — 무엇이 문제인가 . . . 13
4. 하네스 엔지니어링 — 다섯 요소 . . . 14
5. 수업의 흐름 — 다섯 단계 . . . 14
따라 하기 . . . 15
하네스 관점 — 이번 교시에서 본 것 . . . 16
자주 하는 오해 . . . 16
확인 문제 . . . 16

2교시. 개발환경 및 OpenRouter 연결 . . . 17

수업의 흐름에서 이번 교시의 위치 . . . 17
학습 목표 . . . 17
명령 앞의 표시 . . . 17
1. 개념 . . . 17
따라 하기 . . . 19
하루를 마칠 때와 다시 시작할 때 . . . 23
하네스 관점 — 이번 교시에서 세운 것 . . . 24
자주 만나는 오류 . . . 24
확인 문제 . . . 24

3교시. Python·FastAPI 기본 이해 . . . 25

수업의 흐름에서 이번 교시의 위치 . . . 25

학습 목표 . . . 25
1. 파이썬 15분 맛보기 . . . 25
2. FastAPI 코드 읽기 . . . 26
따라 하기 . . . 27
하네스 관점 — 이번 교시에서 본 것 . . . 31
자주 만나는 오류 . . . 31
확인 문제 . . . 31
4교시. CRUD 와 HTTP 기초 . . . 32
수업의 흐름에서 이번 교시의 위치 . . . 32
학습 목표 . . . 32
1. 개념 . . . 32
따라 하기 . . . 33
하네스 관점 — 이번 교시에서 본 것 . . . 37
자주 만나는 오류 . . . 37
확인 문제 . . . 37
5교시. 하네스 규칙 작성 . . . 39
수업의 흐름에서 이번 교시의 위치 . . . 39
학습 목표 . . . 39
1. 개념 — 다섯 요소를 파일로 . . . 39
따라 하기 . . . 40
하네스 관점 — 이번 교시에서 세운 것 . . . 46
자주 만나는 오류 . . . 46
확인 문제 . . . 47
6교시. AI 에게 CRUD 구현 지시 . . . 48
수업의 흐름에서 이번 교시의 위치 . . . 48
학습 목표 . . . 48
왜 CRUD 를 한 번에 시키지 않는가 . . . 48
따라 하기 . . . 48
검증이 실패했을 때 — ⑤ 의 다른 길 . . . 53
하네스 관점 — 이번 교시에서 본 것 . . . 54
자주 만나는 오류 . . . 54
확인 문제 . . . 54
7교시. PostgreSQL 기초 . . . 55
수업의 흐름에서 이번 교시의 위치 . . . 55
학습 목표 . . . 55
1. 개념 . . . 55
따라 하기 . . . 56
하네스 관점 — 이번 교시에서 본 것 . . . 62

자주 만나는 오류 . . . 62
확인 문제 . . . 62
8교시. FastAPI 와 DB 연결 . . . 64
수업의 흐름에서 이번 교시의 위치 . . . 64
학습 목표 . . . 64
1. 개념 . . . 64
따라 하기 . . . 65
하네스 관점 — 이번 교시에서 본 것 . . . 70
자주 만나는 오류 . . . 71
확인 문제 . . . 71
9교시. 수정·삭제 구현 . . . 72
수업의 흐름에서 이번 교시의 위치 . . . 72
학습 목표 . . . 72
1. 개념 . . . 72
따라 하기 . . . 73
하네스 관점 — 이번 교시에서 본 것 . . . 77
자주 만나는 오류 . . . 78
확인 문제 . . . 78
10교시. HTML·JS 화면 연결 . . . 79
수업의 흐름에서 이번 교시의 위치 . . . 79
학습 목표 . . . 79
1. 개념 . . . 79
따라 하기 . . . 80
화면이 이상할 때 — 오류를 AI 에게 전달하는 법 . . . 85
하네스 관점 — 이번 교시에서 본 것 . . . 85
자주 만나는 오류 . . . 85
확인 문제 . . . 86
11교시. 자동 테스트와 오류 수정 . . . 87
수업의 흐름에서 이번 교시의 위치 . . . 87
학습 목표 . . . 87
1. 개념 . . . 87
따라 하기 . . . 88
하네스 관점 — 이번 교시에서 본 것 . . . 93
자주 만나는 오류 . . . 93
확인 문제 . . . 94
12교시. GitHub 저장 및 발표 . . . 95
수업의 흐름에서 이번 교시의 위치 . . . 95
학습 목표 . . . 95

1. 개념 . . . 95
따라 하기 . . . 96
이 다음에 — 스스로 해 볼 것 . . . 101
하네스 관점 — 과정 전체를 돌아보며 . . . 102
자주 만나는 오류 . . . 102
확인 문제 . . . 102

부록 A. 하네스 문서 . . . 104

A-1. 하네스 폴더 안내 (harness/README.md) . . . 104
A-2. 요구사항 명세 (harness/REQUIREMENTS.md) . . . 105
A-3. 에이전트 규칙 (harness/AGENTS.md) . . . 107
A-4. 작업 지시서 틀 (harness/TASK_TEMPLATE.md) . . . 110
A-5. 6교시 작업 지시서 (harness/tasks/06_memo_draft.md) . . . 111
A-6. 8교시 작업 지시서 (harness/tasks/08_db_connect.md) . . . 112
A-7. 9교시 작업 지시서 (harness/tasks/09_update_delete.md) . . . 113
A-8. 10교시 작업 지시서 (harness/tasks/10_web_page.md) . . . 114
A-9. 11교시 작업 지시서 (harness/tasks/11_blank_title_bug.md) . . . 115
A-10. 검증 체크리스트 (harness/VERIFY_CHECKLIST.md) . . . 116
A-11. 작업 기록 (harness/WORKLOG.md) . . . 118
A-12. 권한 규칙 (harness/opencode.jsonc) . . . 120

부록 B. 실행 환경 파일 . . . 122

docker-compose.yml . . . 123
Dockerfile . . . 125
.env.example . . . 126
scripts/verify.sh . . . 126
.githooks/pre-commit . . . 127
scripts/ask_ai.py . . . 128

부록 C. 완성본 코드 . . . 130

app/schemas.py . . . 130
app/db.py . . . 131
app/main.py . . . 132
app/static/index.html . . . 134
app/static/app.js . . . 135
tests/conftest.py . . . 137
tests/test_memos.py . . . 138

부록 D. 자주 쓰는 명령 모음 . . . 141

1. 하루의 시작과 끝 — [PS] . . . 141
2. 컨테이너 다루기 — [PS] . . . 141
3. 검증 — [dev] . . . 142

4. API 불러 보기 — [dev] . . . 142
5. Git — [dev] . . . 142
6. AI 부르기 — [dev] . . . 143
7. OpenCode 안에서 — [AI] . . . 143
8. 데이터베이스 — [PS] 와 [psql] . . . 144
9. 막혔을 때 — [dev] . . . 144

부록 E. 용어 풀이 . . . 145

AI 와 하네스 . . . 145
Docker . . . 146
파이썬과 FastAPI . . . 146
HTTP . . . 147
데이터베이스 . . . 147
테스트 . . . 148
Git . . . 148
화면 . . . 149

부록 F. 오류 해결 모음 . . . 150

먼저 할 일 — 오류를 읽는 순서 . . . 150
1. DB 가 켜지지 않는다 (가장 중요) . . . 150
2. Docker . . . 152
3. 파일이 반영되지 않는다 (바인드 마운트) . . . 152
4. OpenRouter 와 OpenCode . . . 153
5. 서버 (FastAPI) . . . 153
6. 데이터베이스 연결 (8교시부터) . . . 154
7. 테스트와 검증 . . . 154
8. Git . . . 155
9. 화면 (10교시부터) . . . 155
그래도 안 될 때 — 처음부터 다시 . . . 155

과정 안내

하네스 엔지니어링 입문 — AI 에이전트에게 맡겨 만들고, 내가 이해하고, 검증해서 저장하는 메모 CRUD 프로젝트

코드를 먼저 배운 뒤에 AI 를 쓰는 과정이 아니다. **AI 에이전트에게 개발을 맡기면서**, 그 과정에서 코드와 개발 원리를 이해하는 과정이다. 대신 "되는 것 같으니 넘어가자"로 끝내지 않는다. AI 의 작업을 **규칙·도구·권한·실행·검증**으로 통제하고, **테스트를 통과한 코드만 Git 에 저장**한다.

대상	프로그래밍을 처음 배우는 사람 (코딩 경험이 없어도 된다)
기간	3일 × 8시간 = 24시간, 12교시 (1교시 = 2시간)
결과물	메모 CRUD 웹 서비스 (FastAPI + PostgreSQL + HTML·JS 화면 + 자동 테스트)
사용 기술	Docker, OpenRouter, OpenCode(AI 에이전트), Python, FastAPI, PostgreSQL, pytest, Git
교안 PDF	`docs/하네스_엔지니어링_입문_학습교안.pdf`

1. 바이브 코딩과 하네스 엔지니어링

바이브 코딩은 AI 에게 말로 시키고, 돌아가는 것 같으면 그대로 받아들이는 방식이다. 빠르지만 세 가지가 남지 않는다. 무엇을 만들기로 했는지(기준), 정말 되는지(증거), 내가 이해했는지(지식).

하네스 엔지니어링은 AI 가 일하는 **환경을 설계**하는 일이다. 하네스(harness)는 말에게 씌우는 마구를 뜻한다. 힘센 말이 원하는 방향으로 달리게 하는 장치다.

	바이브 코딩	하네스 엔지니어링 (이 과정)
시키는 방법	"메모 앱 만들어 줘"	요구사항 명세 + 작업 지시서
AI 가 따르는 것	그때그때의 대화	문서로 적어 둔 규칙 (`AGENTS.md`)
AI 가 할 수 있는 일	무엇이든	허용한 것만 (`opencode.jsonc` 의 권한)
"다 됐다"의 기준	돌아가는 것 같다	검증 스크립트 통과 (`scripts/verify.sh`)
저장	아무 때나	검증을 통과해야만 커밋된다 (`pre-commit`)
오류가 나면	"안 돼, 고쳐 줘"	오류 출력 전달 → 원인 설명 → 수정 → 재검증
끝난 뒤 나에게 남는 것	설명할 수 없는 코드	읽고 설명할 수 있는 코드 + 작업 기록

하네스의 다섯 요소

요소	질문	이 저장소에서
규칙	어떻게 일해야 하는가?	`AGENTS.md`
도구	무엇으로 일하는가?	Docker 컨테이너, `scripts/verify.sh`
권한	어디까지 해도 되는가?	`opencode.jsonc` (allow / ask / deny)
실행	어디에서 실행되는가?	dev 컨테이너 안, `/workspace` 폴더만
검증	다 됐다는 것을 어떻게 아는가?	`tests/`, `scripts/verify.sh`, `.githooks/pre-commit`

2. 수업의 흐름

모든 실습은 아래 다섯 단계를 반복한다. 6교시부터는 교시마다 이 고리를 한 바퀴씩 돈다.

```
① 사람이 요구사항 작성          무엇을 만들고 싶은지 명확히 적는다
      |                      → harness/REQUIREMENTS.md
      ▼
② AI 에이전트에게 개발 지시      OpenRouter 를 통해 모델을 쓴다
      |                      → harness/tasks/ 의 작업 지시서
      ▼
③ 하네스로 개발 과정 제어        규칙 · 도구 · 권한 · 실행 · 검증
      |                      → AGENTS.md, opencode.jsonc
      ▼
④ 코드 실행 및 테스트            FastAPI + PostgreSQL
      |                      → sh scripts/verify.sh
      ▼
⑤ 오류 확인 → AI 수정 → 재검증    통과하면 Git 에 저장
                             → git commit (pre-commit 이 한 번 더 검증)
```

3. 준비물

내 PC 에 설치하는 것은 **세 가지뿐**이다. Python, PostgreSQL, AI 에이전트는 PC 에 설치하지 않는다. 모두 Docker 컨테이너 안에 들어 있다.

준비물	용도	받는 곳
Docker Desktop	모든 도구를 컨테이너로 실행	https://www.docker.com/products/docker-desktop/
Git for Windows	저장소 내려받기, GitHub 에 올리기	https://git-scm.com/download/win
편집기 (VS Code 권장)	파일 읽기·고치기	https://code.visualstudio.com/
OpenRouter 계정과 API 키	AI 모델 사용	https://openrouter.ai/
GitHub 계정	12교시에 결과물 올리기	https://github.com/

- 운영체제: Windows 10 또는 11 (64비트). Docker Desktop 은 WSL 2 방식으로 설치한다(설치 화면의 기본값).
- OpenRouter 는 쓴 만큼 요금을 내는 서비스다. 요금과 키 발급 방법은 2교시에서 다룬다.
- 설치 방법은 2교시에 단계별로 적혀 있다. 미리 해 두지 않아도 된다.

4. 시작하는 법

2교시에 자세히 따라 한다. 여기는 한눈에 보는 요약이다. 명령은 **PowerShell** 에서 실행한다.

```
# 1) 저장소를 C 드라이브에 내려받는다. (폴더 위치가 중요하다. 아래 "왜 C 드라이브인가" 참고)
git clone https://github.com/sinaboro/harness-crud-lab.git C:\harness-crud-lab
cd C:\harness-crud-lab

# 2) 환경 변수 파일을 만들고, 메모장으로 열어 OPENROUTER_API_KEY 를 채운다.
Copy-Item .env.example .env
notepad .env

# 3) 컨테이너 3개(db, app, dev)를 만들고 켠다. 처음에는 몇 분 걸린다.
docker compose up -d --build

# 4) 상태를 본다. db 는 healthy, app 과 dev 는 running(Up) 이어야 한다.
docker compose ps
```

브라우저에서 http://localhost:8000/health 를 열어 `{"status":"ok"}` 가 보이면 준비가 끝난 것이다.

작업실(dev 컨테이너)에 들어가는 명령:

```
docker compose exec dev bash
```

이제 1교시부터 순서대로 따라 한다.

5. 교시표

일차	교시	학습 내용	실습 결과	하네스에서 배우는 것
1일	1	AI 개발 방식과 하네스 이해	AI·하네스·도구 구분	다섯 요소, 수업의 흐름
	2	개발환경 및 OpenRouter 연결	AI 모델 호출 성공	도구·실행 환경(Docker, 바인드 마운트)
	3	Python·FastAPI 기본 이해	`/hello` API	AI 를 "설명해 주는 선생님"으로 쓰기
	4	CRUD 와 HTTP 기초	GET·POST 테스트	규칙 없는 AI 의 한계 관찰
2일	5	하네스 규칙 작성	`AGENTS.md`	규칙·권한·검증을 직접 세운다
	6	AI 에게 CRUD 구현 지시	메모 CRUD 초안	작업 지시서, 계획 승인, 검토
	7	PostgreSQL 기초	DB·테이블 생성	AI 가 쓸 SQL 을 내가 먼저 읽을 수 있게
	8	FastAPI 와 DB 연결	메모 저장·조회	테스트는 그대로, 구현만 교체
3일	9	수정·삭제 구현	CRUD 완성	테스트 추가만 허용, 삭제 금지
	10	HTML·JS 화면 연결	웹에서 CRUD 실행	테스트가 못 보는 것을 사람이 확인
	11	자동 테스트와 오류 수정	테스트 통과	실패하는 테스트 먼저 → AI 수정 → 재검증
	12	GitHub 저장 및 발표	최종 프로젝트	기록으로 과정을 설명한다

각 교시 안내서의 구성: **학습 목표 → 개념 → 따라 하기(명령·기대 결과·확인 방법) → 하네스 관점 → 자주 만나는 오류 → 확인 문제**

보조 문서

- `docs/commands.md` — 자주 쓰는 명령 모음 (어디에서 치는 명령인지 구분)
- `docs/glossary.md` — 용어 풀이
- `docs/troubleshooting.md` — 오류 해결 모음
- `harness/README.md` — 하네스 문서 안내

6. 실행 환경 한눈에 보기

```
내 PC (Windows)
│
├─ C:\harness-crud-lab\              ← 편집기로 여는 폴더. 여기 파일을 고친다.
│    ├─ app\  tests\  harness\ ...
│    └─ data\postgres\                ← 메모 데이터가 실제로 쌓이는 곳
│
└─ Docker Desktop
     │
     ├─ [app 컨테이너]  FastAPI 서버          → 브라우저에서 http://localhost:8000
     │     /workspace  ⇄  C:\harness-crud-lab          (바인드 마운트)
     │
     ├─ [dev 컨테이너]  작업실: pytest, git, OpenCode(AI 에이전트)
     │     /workspace  ⇄  C:\harness-crud-lab          (바인드 마운트)
     │
     └─ [db 컨테이너]   PostgreSQL 17
           /var/lib/postgresql/data  ⇄  C:\harness-crud-lab\data\postgres   (바인드 마운트)
```

왜 C 드라이브인가 — 바인드 마운트

바인드 마운트는 내 PC 의 폴더를 컨테이너 안의 폴더에 **그대로 연결**하는 것이다. 복사가 아니라 같은 폴더를 양쪽에서 보는 것이다.

- VS Code 로 `C:\harness-crud-lab\app\main.py` 를 고치면 → 컨테이너의 `/workspace/app/main.py` 가 바로 바뀐다 → 서버가 저절로 다시 켜진다.
- AI 에이전트가 컨테이너 안에서 파일을 고치면 → 내 PC 의 편집기에 바로 보인다. **AI 가 무엇을 고쳤는지 내 눈으로 볼 수 있다.**
- 컨테이너를 지워도(`docker compose down`) 소스와 DB 데이터는 C 드라이브에 그대로 남는다.
- 모든 수강생이 같은 경로(`C:\harness-crud-lab`)를 쓰면 교안의 명령과 화면이 똑같아진다.

확인 방법은 2교시에서 직접 해 본다.

7. 저장소 구조

```
harness-crud-lab/
├─ README.md                  이 문서
├─ Dockerfile                 도구 상자 이미지(Python + pytest + OpenCode)의 설계도
├─ docker-compose.yml         컨테이너 3개(db, app, dev)와 바인드 마운트 설정
├─ docker-compose.volume.yml  DB 데이터 저장 방식의 예비 설정 (문제가 생길 때만)
├─ requirements.txt           파이썬 패키지 목록 (버전 고정)
├─ .env.example               환경 변수 견본 (.env 는 Git 에 올리지 않는다)
├─ opencode.jsonc             AI 에이전트 설정 (출발점: 모델만. 5교시에 권한 규칙 추가)
├─ pytest.ini, ruff.toml      테스트·코드 검사 설정
│
├─ app/                       ★ 내가(AI 와 함께) 만들어 갈 서버 코드. 출발점은 /health 하나
├─ tests/                     ★ 자동 테스트. 출발점은 테스트 하나
│
├─ harness/                   하네스 문서
│   ├─ REQUIREMENTS.md          요구사항 명세 (R1~R7)
│   ├─ AGENTS.md                에이전트 규칙 완성본 (5교시에 맨 위 폴더로 가져간다)
│   ├─ AGENTS.template.md       규칙을 직접 써 보는 틀
│   ├─ opencode.jsonc           권한 규칙 완성본
│   ├─ TASK_TEMPLATE.md         작업 지시서 틀
│   ├─ tasks/                   교시별 작업 지시서 (6·8·9·10·11교시)
│   ├─ VERIFY_CHECKLIST.md      커밋 전 검증 체크리스트
│   └─ WORKLOG.md               작업 기록 (내가 이해한 것을 적는다)
│
├─ scripts/
│   ├─ verify.sh                검증 스크립트 (ruff + pytest)
│   ├─ ask_ai.py                OpenRouter 로 모델에게 질문하기 (2교시)
│   └─ checkpoint.sh            교시별 완성 예시 가져오기
├─ .githooks/pre-commit       커밋 직전에 검증을 실행하는 스크립트
│
├─ tools/build_guide.py       교안 PDF 를 만드는 스크립트 (md 문서들을 한 권으로 묶는다)
│
├─ checkpoints/               교시별 완성 예시 (막혔을 때 비교·복구용)
│   └─ lesson03 · 04 · 06 · 08 · 09 · 10 · 11(완성본)
│
└─ docs/
    ├─ lessons/                 12교시 안내서
    ├─ commands.md, glossary.md, troubleshooting.md
    └─ 하네스_엔지니어링_입문_학습교안.pdf
```

`app/` 과 `tests/` 는 **거의 비어 있는 상태에서 시작**한다. 교시를 따라가며 AI 에이전트와 함께 채운다. 완성된 모습을 먼저 보고 싶으면 `checkpoints/lesson11/` 을 본다. 실습이 막히면 `sh scripts/checkpoint.sh 08` 처럼 그 교시의 완성 예시를 가져올 수 있다.

8. 사용 도구와 선택 이유

도구	역할	이 도구를 고른 이유
Docker Desktop	모든 도구를 컨테이너로 실행	PC 마다 다른 설치 문제를 없앤다. AI 에이전트를 PC 가 아닌 컨테이너 안에 가둬 둘 수 있다.
OpenRouter	여러 회사의 AI 모델을 하나의 주소·하나의 키로 부르는 서비스	키 하나로 모델을 바꿔 가며 쓸 수 있다. 키마다 사용 한도를 걸 수 있어 수업에 안전하다.
OpenCode	AI 에이전트 (파일을 읽고 고치고 명령을 실행하는 프로그램)	① OpenRouter 를 기본 지원한다(환경 변수 `OPENROUTER_API_KEY`). ② 프로젝트의 `AGENTS.md` 를 자동으로 읽는다. ③ 설정 파일의 `permissions` 로 allow / ask / deny 를 정할 수 있다 — 하네스의 "규칙"과 "권한"을 파일로 보여 주기에 알맞다. ④ 터미널 프로그램이라 컨테이너 안에서 실행된다.
FastAPI	웹 API 프레임워크	코드가 짧고, `/docs` 화면에서 바로 API 를 눌러 볼 수 있다.
PostgreSQL + psycopg	데이터베이스와 파이썬 드라이버	ORM 없이 SQL 을 직접 쓴다. 7교시에 배운 SQL 이 코드에 그대로 보여서 AI 가 쓴 코드를 읽을 수 있다.
pytest + ruff	자동 테스트와 코드 검사	"다 됐다"를 말이 아니라 명령 한 줄의 결과로 판단한다.
Git + GitHub	변경 기록과 저장	AI 가 바꾼 내용을 `git diff` 로 읽고, 검증을 통과한 것만 저장한다.

버전은 `Dockerfile`(OpenCode 2.2.6, Python 3.12), `docker-compose.yml`(PostgreSQL 17), `requirements.txt` 에 고정해 두었다. 기본 모델은 OpenRouter 의 `anthropic/claude-haiku-5.5` 다. 바꾸는 방법은 2교시에 있다.

9. 이 교안의 명령과 결과에 대하여

- API 응답(JSON), `pytest`·`ruff` 결과, SQL 실행 결과는 **실제로 실행해서 나온 것**을 실었다.

교시별 완성 예시(`checkpoints/`)의 코드를 실제 PostgreSQL 에 연결해 실행한 결과다.

- 시각(`created_at` 등)과 실행 시간(`in 0.08s`)은 실행할 때마다 달라진다.
- **AI 의 답변과 AI 가 만든 코드는 실을 수 없다.** 같은 지시에도 매번 다르게 나오기 때문이다.

그래서 이 교안은 "AI 가 이렇게 답한다" 대신 **"결과가 이 조건을 만족하는지 이렇게 확인한다"** 를 적는다. 이것이 하네스 엔지니어링의 사고방식이기도 하다.

- Docker 명령의 화면 출력과 OpenCode 의 화면 모양은 버전에 따라 조금씩 다를 수 있어, 화면을 그대로 싣는 대신 **무엇을 확인해야 하는지**를 글로 적었다.

10. 만든 사람

강사 김대철 · 수업용 실습 저장소. 수업·학습 목적으로 자유롭게 사용할 수 있다.

1교시. AI 개발 방식과 하네스 이해

시간	2시간
실습 결과	AI·하네스·도구를 구분할 수 있다
필요한 것	웹 브라우저 (아직 아무것도 설치하지 않는다)

수업의 흐름에서 이번 교시의 위치

```
① 요구사항 작성 → ② AI 에게 지시 → ③ 하네스로 제어 → ④ 실행·테스트 → ⑤ 수정·재검증·저장
└──────────────── 이번 교시: 다섯 단계 전체를 지도처럼 훑어본다 ────────────────┘
```

학습 목표

1. AI 모델, AI 에이전트, 하네스, 도구가 각각 무엇인지 한 문장으로 말할 수 있다.
2. 바이브 코딩과 하네스 엔지니어링의 차이를 세 가지 이상 말할 수 있다.
3. 하네스의 다섯 요소(규칙·도구·권한·실행·검증)를 이 저장소의 파일과 짝지을 수 있다.
4. "좋은 요구사항"이 무엇인지 예를 들어 설명할 수 있다.

1. 우리가 3일 동안 만들 것

메모장 웹 서비스다. 브라우저에서 메모를 쓰고, 목록으로 보고, 고치고, 지운다. 이 네 가지 동작을 **CRUD**(Create·Read·Update·Delete)라고 부른다. 게시판, 쇼핑몰 장바구니, 할 일 목록 — 대부분의 서비스가 CRUD 로 이루어져 있다.

그런데 이 과정에서 코드를 직접 치는 양은 많지 않다. **대부분의 코드는 AI 에이전트가 쓴다.** 우리가 하는 일은 다음 네 가지다.

1. **무엇을 만들지 정한다.** (요구사항)
2. **AI 가 어떻게 일해야 하는지 정한다.** (하네스)
3. **AI 가 만든 것이 맞는지 확인한다.** (검증)
4. **AI 가 만든 코드를 읽고 이해한다.** (학습)

2. 네 가지 낱말 구분하기

AI 모델

글을 받아서 글을 돌려주는 프로그램이다. 질문을 보내면 답이 온다. 그것이 전부다. 모델은 **내 컴퓨터의 파일을 볼 수 없고, 명령을 실행할 수도 없다.** 대화가 끝나면 아무것도 기억하지 못한다.

> 비유: 전화로만 연결된 아주 똑똑한 상담원. 말은 잘하지만 내 책상 위를 볼 수도, 만질 수도 없다.

AI 에이전트

AI 모델에게 **손과 눈**을 달아 준 프로그램이다. 에이전트는 모델과 대화하면서 대신 파일을 읽어 주고, 모델이 "이 파일을 이렇게 고쳐"라고 하면 실제로 고치고, "이 명령을 실행해"라고 하면 실행해서 결과를 다시 모델에게 알려 준다. 이 과정을 일이 끝날 때까지 되풀이한다.

> 비유: 상담원(모델)의 지시를 받아 내 책상에서 실제로 일하는 직원.

이 과정에서 쓰는 에이전트는 **OpenCode** 다.

도구

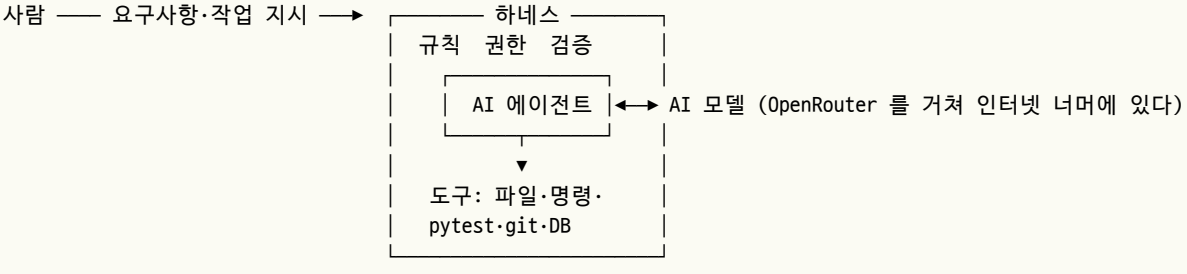
에이전트가 일할 때 쓰는 것들이다. 파일 읽기·쓰기, 명령 실행(셸), 테스트 실행기(pytest), 코드 검사기(ruff), 버전 관리(Git), 데이터베이스(PostgreSQL)가 모두 도구다. 도구는 스스로 판단하지 않는다. 시키면 시킨 대로 한다.

하네스

에이전트가 **어떤 규칙으로, 어떤 도구를, 어디까지 써서, 어디에서 일하고, 무엇을 통과해야 끝나는지** 정해 둔 장치 전체다.

> 하네스(harness)는 말에게 씌우는 마구(고삐·안장·굴레)다. 말은 힘이 세고 빠르지만, 마구가 없으면 그 힘을 원하는 방향으로 쓸 수 없다. 마구는 말의 힘을 줄이는 것이 아니라 **쓸모 있게 만드는 것**이다.

한 장으로 보기



낱말	한 문장	이 과정에서
AI 모델	글을 받아 글을 돌려준다	OpenRouter 로 부르는 모델
AI 에이전트	모델의 지시대로 파일을 고치고 명령을 실행한다	OpenCode
도구	에이전트가 일할 때 쓰는 것	파일, 셸, pytest, ruff, Git, PostgreSQL
하네스	에이전트의 일을 통제하는 장치 전체	`AGENTS.md`, `opencode.jsonc`, `verify.sh`, `pre-commit`, Docker 컨테이너
OpenRouter	여러 회사의 모델을 한 주소로 부르게 해 주는 중개 서비스	API 키 하나로 사용

3. 바이브 코딩 — 무엇이 문제인가

바이브 코딩(vibe coding) 은 AI 에게 말로 시키고, 돌아가는 것 같으면 코드를 읽지 않고 받아들이는 방식이다. 간단한 것을 빨리 만들어 볼 때는 쓸모가 있다. 하지만 이런 일이 생긴다.

사례 1 — 기준이 없다. "메모 앱 만들어 줘"라고 했더니 로그인 화면까지 딸린 앱이 나왔다. 다음 날 같은 말로 다시 시켰더니 전혀 다른 앱이 나왔다. 무엇이 "맞는" 결과인지 정해 둔 적이 없으니 어느 쪽이 맞는지도 알 수 없다.

사례 2 — 증거가 없다. AI 가 "삭제 기능을 추가했습니다. 정상 동작합니다."라고 했다. 며칠 뒤 없는 번호를 지우려 하니 서버가 오류를 내며 멈췄다. AI 는 실행해 보지 않고 "동작한다"고 말한 것이다.

사례 3 — 되돌릴 수 없다. "오류 고쳐 줘"를 다섯 번 반복했다. AI 는 그때마다 여러 파일을 고쳤다. 이제 처음보다 더 망가졌는데, 어느 시점이 멀쩡했는지 알 수 없고 돌아갈 방법도 없다.

사례 4 — 사고가 난다. AI 가 테스트용으로 데이터베이스를 비우는 명령을 실행했다. 거기에는 진짜 데이터가 들어 있었다. 아무도 AI 에게 "거기는 건드리지 마"라고 말한 적이 없다.

사례 5 — 남는 것이 없다. 앱은 돌아간다. 그런데 "이 줄이 무슨 뜻이에요?"라는 질문에 답할 수 없다. 고칠 일이 생기면 다시 AI 에게 통째로 맡기는 수밖에 없다.

다섯 사례의 공통점: **AI 가 못해서 생긴 문제가 아니다. 사람이 정해 두지 않아서 생긴 문제다.**

4. 하네스 엔지니어링 — 다섯 요소

위의 문제를 하나씩 막는 장치가 하네스의 다섯 요소다.

요소	질문	막아 주는 문제	이 저장소의 파일
규칙	어떻게 일해야 하는가?	제멋대로 덧붙이기, 계획 없이 고치기	`AGENTS.md`
도구	무엇으로 일하는가?	"제 컴퓨터에서는 되는데요"	`Dockerfile`, `docker-compose.yml`, `scripts/verify.sh`
권한	어디까지 해도 되는가?	위험한 명령, 비밀 값 노출 (사례 4)	`opencode.jsonc`
실행	어디에서 실행되는가?	내 PC 전체를 건드리는 일	dev 컨테이너 (`/workspace` 만 보인다)
검증	다 됐다는 것을 어떻게 아는가?	확인 없는 "완료" (사례 2)	`tests/`, `scripts/verify.sh`, `.githooks/pre-commit`

그리고 다섯 요소 앞뒤에 두 가지가 더 있다.

- **요구사항**(앞): 무엇이 맞는 결과인지 정한다. → 사례 1 을 막는다. (`harness/REQUIREMENTS.md`)
- **Git 저장**(뒤): 검증을 통과한 시점마다 저장한다. → 사례 3 을 막는다.

사례 5(남는 것이 없다)는 장치만으로는 막을 수 없다. 그래서 이 과정에서는 **AI 가 쓴 코드를 읽고, 내 말로 설명한 뒤에만 저장한다**는 규칙을 사람에게 건다. (`harness/WORKLOG.md`)

규칙과 권한은 어떻게 다른가

- **규칙**은 AI 에게 하는 **부탁**이다. "커밋은 하지 마." AI 는 대체로 지키지만, 잊거나 어길 수 있다.
- **권한**은 **잠금장치**다. `git commit` 을 금지해 두면 AI 가 실행하려 해도 실행되지 않는다.

교실 문에 "들어가지 마시오"라고 써 붙이는 것(규칙)과 문을 잠그는 것(권한)의 차이다. 둘 다 필요하다. 써 붙여야 왜 안 되는지 알고, 잠가야 확실히 막힌다.

5. 수업의 흐름 — 다섯 단계

6교시부터는 교시마다 아래 고리를 한 바퀴씩 돈다.

```
① 사람이 요구사항 작성          "무엇을" 만들지 적는다. "어떻게"는 적지 않는다.
        ▼
② AI 에이전트에게 개발 지시       작업 지시서를 건넨다. AI 는 먼저 계획을 말한다.
        ▼
③ 하네스로 개발 과정 제어         AI 는 규칙과 권한 안에서만 움직인다.
        ▼
④ 코드 실행 및 테스트             검증 스크립트를 실행한다.
        ▼
⑤ 오류 확인 → AI 수정 → 재검증    실패하면 ④ 로 돌아간다. 통과하면 사람이 읽고 Git 에 저장한다.
```

사람이 하는 일은 ①, 그리고 ②와 ⑤의 **판단**이다. 계획을 승인할지, 결과를 받아들일지는 항상 사람이 정한다.

따라 하기

실습 1. 분류하기 (15분)

아래 항목을 **AI 모델 / AI 에이전트 / 도구 / 하네스** 중 하나로 분류한다. 종이나 메모장에 적는다.

번호	항목	분류
1	OpenRouter 로 부르는 `claude-haiku-5.5`	
2	파일을 읽고 고치고 명령을 실행하는 OpenCode	
3	테스트를 실행해 통과·실패를 알려 주는 pytest	
4	"커밋은 사람이 한다"고 적어 둔 `AGENTS.md`	
5	`git push` 를 실행하지 못하게 막아 둔 `opencode.jsonc`	
6	메모를 저장하는 PostgreSQL	
7	커밋 직전에 테스트를 돌리는 `pre-commit`	
8	AI 가 `/workspace` 폴더 밖을 볼 수 없게 하는 Docker 컨테이너	

옆 사람과 답을 비교한다. 6번과 8번에서 의견이 갈리면 정상이다. 같은 Docker 가 "도구"이면서 "하네스의 실행 환경"이기도 하다. **무엇이냐보다 무슨 역할을 하느냐**로 구분한다.

실습 2. 저장소 둘러보기 (20분)

브라우저에서 https://github.com/sinaboro/harness-crud-lab 을 연다. 아직 내려받지 않는다. 읽기만 한다.

1. `harness/AGENTS.md` 를 연다. 3절 "권한"에서 **하면 안 되는 일**을 세 가지 찾아 적는다.
2. `harness/opencode.jsonc` 를 연다. `"effect": "deny"` 가 붙은 줄 중 하나를 골라, 무엇을 막는 규칙인지 말로 풀어 본다.
3. `scripts/verify.sh` 를 연다. 이 스크립트가 실행하는 검사는 몇 가지인가?
4. `app/main.py` 를 연다. 몇 줄인가? 지금 이 서버가 할 수 있는 일은 무엇인가?
5. `checkpoints/lesson11/app/main.py` 를 연다. 3일 뒤의 모습이다. 지금은 읽히지 않아도 된다. 주소가 몇 개인지만 세어 본다.

확인: 1번에서 찾은 "하면 안 되는 일"이 2번의 `deny` 규칙으로도 걸려 있는지 짝을 맞춰 본다. 규칙(부탁)과 권한(잠금)이 같은 내용을 두 번 말하고 있다는 것을 확인한다.

실습 3. 요구사항 써 보기 (25분)

"메모장을 만들고 싶다"는 말을 요구사항으로 바꿔 본다.

1단계. 아무 안내 없이, 메모장이 해야 할 일을 다섯 줄로 적는다. (5분)

2단계. 내가 쓴 다섯 줄을 아래 질문으로 점검한다.

- 이 문장을 읽은 두 사람이 **똑같은 것**을 만들까?
- 다 만들었는지 **확인할 방법**이 문장 안에 있는가?
- "메모를 지운다" — 없는 메모를 지우려 하면 어떻게 되어야 하는가? 적었는가?
- "제목을 입력한다" — 제목이 비어 있으면? 1만 자면? 적었는가?
- **만들지 않을 것**을 적었는가?

3단계. `harness/REQUIREMENTS.md` 를 열어 R5(메모 지우기)를 읽는다. 내가 쓴 "지우기" 문장과 비교한다.

```
R5. 메모 지우기 (Delete)
- DELETE /memos/1 을 부르면 상태 코드 204 를 돌려준다. 응답 내용은 없다.
- 지운 뒤 GET /memos/1 을 부르면 404 가 온다.
```

> - 그 번호의 메모가 없으면 404

주소나 숫자의 뜻은 4교시에 배운다. 지금 볼 것은 형식이다. **"이렇게 부르면 → 이런 결과가 온다"** 로 적혀 있어서, 읽는 사람마다 다르게 해석할 여지가 없고, 그대로 테스트가 된다.

4단계. `REQUIREMENTS.md` 5절 "이번에는 만들지 않는 것"을 읽는다. 왜 이런 목록이 필요한지 옆 사람에게 설명한다.

실습 4. 사례에 처방하기 (15분)

3절의 다섯 사례를 다시 읽고, 각 사례를 막으려면 하네스의 어느 요소(또는 요구사항·Git)가 필요한지 적는다. 그 요소를 담당하는 이 저장소의 파일 이름도 함께 적는다.

사례	필요한 장치	담당 파일
1. 시킬 때마다 결과가 다르다		
2. 실행해 보지 않고 "동작한다"고 한다		
3. 멀쩡하던 때로 돌아갈 수 없다		
4. 진짜 데이터를 지웠다		
5. 코드를 설명할 수 없다		

하네스 관점 — 이번 교시에서 본 것

- 하네스는 AI 를 못 믿어서 만드는 것이 아니다. **믿을 수 있는 결과를 얻으려고** 만드는 것이다.
- AI 가 똑똑해질수록 더 많은 일을 맡기게 되고, 맡기는 일이 커질수록 하네스가 더 중요해진다.
- 이 과정에서 "코딩"이라고 부르는 일의 절반은 **문서를 쓰는 일**이다. 요구사항, 규칙, 작업 지시서, 체크리스트. 이 문서들이 AI 의 결과물을 결정한다.

자주 하는 오해

오해	실제
"AI 가 다 해 주니 코드를 몰라도 된다"	읽지 못하면 검토할 수 없고, 검토하지 못하면 맡길 수 없다. 쓰는 양은 줄지만 **읽는 능력**은 더 중요해진다.
"하네스는 AI 를 느리게 만든다"	한 번의 지시는 느려진다. 하지만 되돌리고 다시 시키는 횟수가 줄어 전체는 빨라진다.
"프롬프트만 잘 쓰면 된다"	프롬프트는 규칙(부탁)일 뿐이다. 권한과 검증이라는 잠금장치가 함께 있어야 한다.
"테스트가 통과하면 끝이다"	테스트는 "테스트에 적힌 것"만 확인한다. 테스트 자체를 사람이 읽어야 한다.

확인 문제

1. AI 모델과 AI 에이전트의 차이를 "파일"과 "명령"이라는 낱말을 넣어 한 문장으로 설명하시오.
2. "규칙"과 "권한"의 차이를 교실 문의 비유 말고 다른 비유로 설명하시오.
3. 하네스의 다섯 요소를 쓰고, 각각을 담당하는 이 저장소의 파일을 하나씩 적으시오.
4. "메모를 수정할 수 있다"는 요구사항에서 빠져 있는 것을 두 가지 이상 지적하시오.
5. AI 가 "테스트를 모두 통과했습니다"라고 보고했다. 이 말을 그대로 믿으면 안 되는 이유를 두 가지 쓰시오.
6. 수업의 흐름 다섯 단계 중 사람이 **판단**해야 하는 지점은 어디인가?

2교시. 개발환경 및 OpenRouter 연결

시간	2시간
실습 결과	AI 모델 호출 성공
필요한 것	Windows PC, 인터넷, OpenRouter 계정(결제 수단)

수업의 흐름에서 이번 교시의 위치

```
① 요구사항 작성 → ② AI 에게 지시 → ③ 하네스로 제어 → ④ 실행·테스트 → ⑤ 수정·재검증·저장
                  └ 이번 교시: AI 를 부를 수 있게 하고, 하네스의 "도구"와 "실행 환경"을 세운다
```

학습 목표

1. Docker Desktop, Git, 편집기를 설치하고 저장소를 `C:\harness-crud-lab` 에 내려받는다.
2. 컨테이너, 이미지, 바인드 마운트가 무엇인지 설명하고 직접 확인한다.
3. OpenRouter API 키를 발급받아 `.env` 에 넣고, 키가 Git 에 올라가지 않는 이유를 설명한다.
4. 스크립트(`ask_ai.py`)와 에이전트(OpenCode)로 AI 모델을 한 번씩 불러 본다.

명령 앞의 표시

이 과정에서는 명령을 치는 곳이 세 군데다. 명령 블록 위에 어디에서 치는지 표시한다.

표시	어디	프롬프트 모양
[PS]	윈도우의 PowerShell	`PS C:\harness-crud-lab>`
[dev]	dev 컨테이너 안의 셸	`root@…:/workspace#`
[AI]	OpenCode 의 입력 칸	(OpenCode 화면)

1. 개념

왜 Docker 로 하는가

Python, PostgreSQL, AI 에이전트를 PC 에 직접 설치하면 PC 마다 버전과 경로가 달라서 "제 화면은 달라요"가 끝없이 나온다. **Docker** 는 프로그램과 그 실행 환경을 통째로 상자에 담아 실행하는 도구다.

낱말	뜻	비유
이미지	프로그램과 실행 환경을 담아 둔 틀. 바뀌지 않는다.	붕어빵 틀
컨테이너	이미지로 만들어 실제로 돌아가는 것. 지우고 다시 만들 수 있다.	붕어빵
Dockerfile	이미지를 만드는 순서를 적은 파일	틀의 설계도
docker-compose.yml	컨테이너 여러 개를 한꺼번에 켜는 방법을 적은 파일	가게 차림표

하네스 관점에서 Docker 는 한 가지 일을 더 한다. **AI 에이전트를 컨테이너 안에 가둔다.** 에이전트는 컨테이너 안에서 실행되므로, 내 PC 의 문서 폴더나 다른 프로젝트를 볼 수도 고칠 수도 없다. 볼 수 있는 것은 우리가 연결해 준 폴더 하나뿐이다.

컨테이너 3개

이름	하는 일	접속
db	PostgreSQL 데이터베이스	다른 컨테이너에서 db:5432
app	FastAPI 서버	브라우저에서 http://localhost:8000
dev	작업실. pytest, git, OpenCode 를 여기서 실행한다	docker compose exec dev bash

app 과 dev 는 같은 이미지(도구 상자)로 만든다. 하나는 서버를 켜 두는 데 쓰고, 하나는 우리가 들어가서 명령을 치는 데 쓴다.

바인드 마운트 — C 드라이브 폴더를 컨테이너에 연결하기

컨테이너는 지우면 안에 있던 파일도 사라진다. 그래서 **남겨야 하는 것은 내 PC 의 폴더에 두고, 그 폴더를 컨테이너에 연결**한다. 이것이 바인드 마운트다. 복사가 아니다. 같은 폴더를 양쪽에서 함께 본다.

docker-compose.yml 에서 해당하는 줄:

```
app:
  volumes:
    - .:/workspace                              # 이 파일이 있는 폴더(C:\harness-crud-lab) ↔ 컨테이너의 /workspace
db:
  volumes:
    - ./data/postgres:/var/lib/postgresql/data   # C:\harness-crud-lab\data\postgres ↔ DB 데이터 폴더
```

왼쪽:오른쪽 에서 왼쪽이 내 PC, 오른쪽이 컨테이너다. 맨 앞의 . 은 "docker-compose.yml 이 있는 폴더"를 뜻한다. 저장소를 C:\harness-crud-lab 에 두었으므로 . 은 곧 C:\harness-crud-lab 이다.

연결	덕분에 되는 일
C:\harness-crud-lab ↔ /workspace	윈도우 편집기로 고친 파일이 컨테이너에 바로 보인다. AI 가 컨테이너에서 고친 파일이 내 편집기에 바로 보인다.
C:\harness-crud-lab\data\postgres ↔ DB 데이터 폴더	컨테이너를 지우고 다시 만들어도 메모가 남는다.

OpenRouter 와 API 키

OpenRouter 는 여러 회사의 AI 모델을 주소 하나, 키 하나로 부를 수 있게 해 주는 중개 서비스다. 모델 이름만 바꾸면 다른 회사의 모델을 쓸 수 있다.

API 키는 "이 요청은 내 계정에서 보낸 것"임을 증명하는 긴 문자열이다(sk-or-v1-…). 키를 가진 사람은 누구나 내 돈으로 모델을 쓸 수 있다. **비밀번호와 똑같이 다룬다.**

- 키는 .env 파일에만 적는다.
- .env 는 .gitignore 에 들어 있어 Git 이 무시한다. GitHub 에 올라가지 않는다.
- 저장소에는 값이 비어 있는 견본 .env.example 만 올라간다.
- 키를 채팅방, 화면 공유, 캡처 이미지에 노출하지 않는다. 노출했으면 OpenRouter 에서 그 키를 지우고 새로 만든다.

키가 전달되는 길:

```
.env 파일  →  docker-compose.yml 의 ${OPENROUTER_API_KEY}  →  dev 컨테이너의 환경 변수  →  ask_ai.py / OpenCode
```

따라 하기

1단계. 프로그램 세 개 설치 (25분)

이미 설치되어 있으면 건너뛴다.

(1) Docker Desktop

1. https://www.docker.com/products/docker-desktop/ 에서 Windows 용 설치 파일을 받아 실행한다.
2. 설치 중 "Use WSL 2 instead of Hyper-V" 가 보이면 체크된 채로 둔다.
3. 설치가 끝나면 PC 를 다시 시작하라고 할 수 있다. 다시 시작한다.
4. 시작 메뉴에서 Docker Desktop 을 실행한다. 약관 동의 화면이 나오면 동의한다. 로그인은 건너뛰어도 된다.
5. 화면 왼쪽 아래의 표시가 "Engine running" 이 될 때까지 기다린다.

(2) Git for Windows

https://git-scm.com/download/win 에서 받아 설치한다. 설치 화면의 선택지는 모두 기본값으로 둔다.

(3) VS Code

https://code.visualstudio.com/ 에서 받아 설치한다. 다른 편집기를 써도 된다.

확인. 시작 메뉴에서 "PowerShell" 을 찾아 실행하고 아래 세 명령을 친다.

■ [PS] 윈도우 PowerShell

```
docker --version
docker compose version
git --version
```

세 명령 모두 버전 번호가 한 줄씩 나오면 된다. "인식되지 않습니다"라는 말이 나오면 PowerShell 창을 닫고 새로 연 뒤 다시 해 본다. 그래도 안 되면 그 프로그램을 다시 설치한다.

2단계. 저장소를 C 드라이브에 내려받기 (5분)

■ [PS] 윈도우 PowerShell

```
git clone https://github.com/sinaboro/harness-crud-lab.git C:\harness-crud-lab
cd C:\harness-crud-lab
dir
```

확인. `dir` 결과에 `README.md`, `Dockerfile`, `docker-compose.yml`, `app`, `harness` 가 보인다.

경로는 반드시 `C:\harness-crud-lab` 으로 한다. 바탕 화면이나 문서 폴더에 두면 경로에 한글·공백이 들어가거나 OneDrive 동기화와 겹쳐서 문제가 생길 수 있다.

VS Code 로 폴더를 연다.

■ [PS] 윈도우 PowerShell

```
code C:\harness-crud-lab
```

3단계. OpenRouter 키 발급 (15분)

1. https://openrouter.ai/ 에 접속해 가입(Sign up)한다.
2. 오른쪽 위 계정 메뉴에서 **Credits** 로 들어가 크레딧을 충전한다. 최소 금액이면 이 과정에 충분하다.
3. 계정 메뉴에서 **Keys** 로 들어가 **Create Key** 를 누른다.
   - 이름: `harness-crud-lab`
   - Credit limit(이 키로 쓸 수 있는 최대 금액)을 작게 정해 둔다. 키가 새어 나가도 피해가 그 금액으로 제한된다.
4. 만들어진 키(`sk-or-v1-` 로 시작)를 복사한다. **이 화면을 닫으면 다시 볼 수 없다.**

> **요금.** OpenRouter 는 쓴 만큼 낸다. 요금은 "토큰"(AI 가 글을 세는 단위) 수로 계산한다. 이 과정의 기본 모델 `anthropic/claude-haiku-4.5` 는 2026년 11월 기준 입력 100만 토큰에 0.10달러, 출력 100만 토큰에 0.50달러다. 최신 가격은 https://openrouter.ai/models 에서 확인한다. 이름이 `:free` 로 끝나는 무료 모델도 있다. 다만 하루 요청 횟수 제한이 있어(크레딧을 충전하지 않은 계정은 하루 50회) 요청을 여러 번 주고받는 에이전트 실습에는 모자란다.

4단계. .env 만들기 (5분)

■ [PS] 윈도우 PowerShell

```
Copy-Item .env.example .env
notepad .env
```

메모장에서 `OPENROUTER_API_KEY=` 뒤에 복사한 키를 붙여 넣고 저장한다. 등호 앞뒤에 공백을 넣지 않는다. 따옴표도 넣지 않는다.

```
OPENROUTER_API_KEY=sk-or-v1-여기에_내_키
```

확인. Git 이 `.env` 를 무시하는지 본다.

■ [PS] 윈도우 PowerShell

```
git status
```

`nothing to commit, working tree clean` 이 나와야 한다. 방금 `.env` 라는 새 파일을 만들었는데도 Git 은 "바뀐 것이 없다"고 한다. `.gitignore` 에 `.env` 가 적혀 있기 때문이다.

■ [PS] 윈도우 PowerShell

```
git check-ignore -v .env
```

`.gitignore` 의 몇 번째 줄 때문에 무시되는지 알려 준다.

5단계. 컨테이너 만들고 켜기 (15분)

■ [PS] 윈도우 PowerShell

```
docker compose up -d --build
```

- `up`: 컨테이너를 만들고 켠다.
- `-d`: 뒤에서 돌게 한다(명령 창을 돌려받는다).
- `--build`: `Dockerfile` 로 이미지를 만든다.

처음에는 Python 이미지와 PostgreSQL 이미지를 내려받고 패키지를 설치하느라 몇 분 걸린다. 두 번째부터는 금방 끝난다.

확인 1. 컨테이너 상태를 본다.

■ [PS] 윈도우 PowerShell

```
docker compose ps
```

세 줄(`db`, `app`, `dev`)이 나온다. `STATUS` 칸이 `db` 는 `Up … (healthy)`, `app` 과 `dev` 는 `Up …` 이면 된다. `db` 가 `healthy` 가 아니면 아래 "자주 만나는 오류"의 첫 항목을 본다.

확인 2. 브라우저에서 http://localhost:8000/health 를 연다.

```
{"status":"ok"}
```

FastAPI 서버가 컨테이너 안에서 돌고 있고, 내 PC 의 브라우저에서 거기에 접속한 것이다.

6단계. 바인드 마운트 직접 확인하기 (15분)

(1) 윈도우 → 컨테이너. 윈도우에서 파일을 만들고 컨테이너에서 읽는다.

■ [PS] 윈도우 PowerShell

```
Set-Content C:\harness-crud-lab\from_windows.txt "made on windows"
docker compose exec dev cat /workspace/from_windows.txt
```

`made on windows` 가 출력된다. 컨테이너로 복사한 적이 없는데 컨테이너 안에 있다.

(2) 컨테이너 → 윈도우. 컨테이너에서 파일을 만들고 윈도우에서 읽는다.

■ [PS] 윈도우 PowerShell

```
docker compose exec dev sh -c "echo made in container > /workspace/from_container.txt"
Get-Content C:\harness-crud-lab\from_container.txt
```

`made in container` 가 출력된다. VS Code 의 파일 목록에도 두 파일이 나타나 있다.

(3) 연결 정보 보기.

■ [PS] 윈도우 PowerShell

```
docker inspect harness-crud-lab-dev-1 --format "{{ json .Mounts }}"
```

출력에서 `"Type":"bind"` 와 `"Destination":"/workspace"` 를 찾는다. `"Source"` 에는 내 PC 쪽 경로가 적혀 있다(`harness-crud-lab` 으로 끝난다).

(4) DB 데이터 폴더 보기.

■ [PS] 윈도우 PowerShell

```
dir C:\harness-crud-lab\data\postgres
```

`PG_VERSION`, `base`, `global` 같은 파일과 폴더가 보인다. PostgreSQL 이 컨테이너 안에서 쓴 데이터가 C 드라이브에 놓여 있는 것이다. 이 폴더의 파일은 직접 열거나 고치지 않는다.

(5) 뒷정리.

■ [PS] 윈도우 PowerShell

```
Remove-Item C:\harness-crud-lab\from_windows.txt, C:\harness-crud-lab\from_container.txt
git status
```

`working tree clean` 이 나오면 원래대로 돌아온 것이다.

7단계. 작업실에 들어가기 (5분)

■ [PS] 윈도우 PowerShell

```
docker compose exec dev bash
```

프롬프트가 `root@(영문·숫자):/workspace#` 로 바뀐다. 이제 컨테이너 안이다. 여기서 치는 명령은 **[dev]** 로 표시한다.

■ [dev] dev 컨테이너

```
pwd
ls
python --version
pytest --version
opencode --version
```

- `pwd` → `/workspace`
- `ls` → 윈도우의 `C:\harness-crud-lab` 과 같은 파일 목록

- 나머지 세 줄 → 각 도구의 버전. 이 도구들은 내 PC 가 아니라 컨테이너 안에만 있다.

나갈 때는 `exit` 를 친다. 다시 들어올 때는 **[PS]** 에서 `docker compose exec dev bash` 를 친다. 지금은 나가지 말고 이어서 한다.

8단계. AI 모델 불러 보기 — 스크립트 (15분)

키가 컨테이너에 전달되었는지 먼저 본다. 키 전체를 화면에 찍지 않도록 글자 수만 센다.

■ [dev] dev 컨테이너

```
echo ${#OPENROUTER_API_KEY}
```

`0` 이 아닌 숫자가 나오면 전달된 것이다. `0` 이면 "자주 만나는 오류"를 본다.

모델에게 질문한다.

■ [dev] dev 컨테이너

```
python scripts/ask_ai.py "FastAPI 가 뭐야? 두 문장으로 알려 줘"
```

기대 결과. 세 부분이 출력된다.

```
[모델] (실제로 답한 모델 이름)
[답변]
(AI 의 답. 실행할 때마다 다르다)
[사용량] 보낸 토큰 (숫자) / 받은 토큰 (숫자)
```

같은 명령을 한 번 더 실행해 본다. 답이 조금 다르게 나온다. **AI 의 출력은 매번 다르다.** 이것이 "돌아가는 것 같다"를 기준으로 삼을 수 없는 이유이고, 4교시부터 "검증"을 따로 두는 이유다.

`scripts/ask_ai.py` 를 VS Code 로 열어 본다. 주석을 따라 읽으면서 아래 세 가지를 찾는다.

1. 요청을 보내는 주소(`API_URL`)
2. 키를 어디에서 읽는가 (`os.environ`)
3. 답은 응답의 어느 부분에 들어 있는가 (`choices` …)

파일을 함께 보내서 질문할 수도 있다.

■ [dev] dev 컨테이너

```
python scripts/ask_ai.py --file app/main.py "이 코드가 하는 일을 초보자에게 세 문장으로 설명해 줘"
```

키가 틀렸을 때 무슨 일이 생기는지도 일부러 본다.

■ [dev] dev 컨테이너

```
OPENROUTER_API_KEY=sk-or-v1-invalid python scripts/ask_ai.py "안녕"
```

```
[실패] 상태 코드 401: API 키가 틀렸습니다. .env 의 OPENROUTER_API_KEY 값을 확인하세요.
{"error":{"message":"User not found.","code":401}}
```

`401` 은 "누구인지 확인할 수 없음"이라는 뜻의 상태 코드다. 상태 코드는 4교시에 배운다.

9단계. AI 모델 불러 보기 — 에이전트 (15분)

이번에는 에이전트 OpenCode 로 같은 모델을 부른다. 어떤 모델을 쓸지는 `opencode.jsonc` 에 적혀 있다.

■ [dev] dev 컨테이너

```
cat opencode.jsonc
```

`"model": "openrouter/anthropic/claude-haiku-4.5"` 한 줄을 찾는다. `openrouter/` 뒤가 OpenRouter 의 모델 이름이다.

질문 하나를 던지고 답만 받아 본다.

■ [dev] dev 컨테이너

```
opencode run "이 폴더에 어떤 파일들이 있는지 살펴보고, 무엇을 하는 프로젝트인지 세 줄로 요약해 줘"
```

관찰. 8단계의 스크립트와 무엇이 다른가?

- 스크립트는 우리가 `--file` 로 준 파일 하나만 알았다.
- 에이전트는 **스스로 폴더를 살펴보고 파일을 골라 읽은 뒤** 답했다. 파일 이름을 알려 주지 않았는데도.

이것이 모델과 에이전트의 차이다. 에이전트에게는 "손과 눈"(파일 읽기, 명령 실행)이 있다.

대화 화면으로도 열어 본다.

■ [dev] dev 컨테이너

```
opencode
```

화면 아래 입력 칸에 다음을 치고 Enter 를 누른다.

■ [AI] OpenCode 입력 칸

```
안녕. 너는 지금 어떤 폴더에서 실행되고 있어? 파일을 고치지는 말고 대답만 해 줘.
```

답을 확인한 뒤 `/exit` 를 쳐서 나온다.

OpenCode 에서 쓰는 명령	하는 일
`/exit`	OpenCode 를 끝낸다
`/new`	새 대화를 시작한다 (앞의 대화를 잊는다)
`/undo`	마지막 요청과, 그 요청으로 바뀐 파일을 되돌린다
`/models`	쓸 수 있는 모델 목록을 본다
`@파일이름`	그 파일을 대화에 첨부한다

> 지금 OpenCode 에는 **규칙도 권한 제한도 없다.** 파일을 고치라고 하면 묻지 않고 고친다. 그래서 4교시까지는 OpenCode 에게 "설명해 줘"만 시키고, 고치는 일은 5교시에 하네스를 세운 뒤에 시킨다.

10단계. Git 에 내 이름 알려 주기 (5분)

커밋에는 "누가 저장했는지"가 기록된다. 이 저장소에서 쓸 이름과 메일을 정한다.

■ [dev] dev 컨테이너

```
git config user.name "홍길동"
git config user.email "gildong@example.com"
git config user.name
```

자기 이름과 메일로 바꿔 친다. 마지막 명령이 방금 넣은 이름을 출력하면 된다. 이 설정은 `C:\harness-crud-lab\.git\config` 에 저장되므로 컨테이너를 다시 만들어도 남는다.

하루를 마칠 때와 다시 시작할 때

■ [PS] 윈도우 PowerShell

```
docker compose stop      # 컨테이너를 멈춘다 (지우지 않는다)
docker compose start     # 멈춘 컨테이너를 다시 켠다
```

PC 를 껐다 켠 뒤에는 Docker Desktop 을 먼저 실행하고, `C:\harness-crud-lab` 에서 `docker compose up -d` 를 친다. 자주 쓰는 명령은 `docs/commands.md` 에 모아 두었다.

하네스 관점 — 이번 교시에서 세운 것

요소	이번 교시에 한 일
도구	Python·pytest·ruff·git·OpenCode 가 든 도구 상자(이미지)를 만들었다. 버전이 고정되어 있어 모두가 같은 도구를 쓴다.
실행	에이전트는 dev 컨테이너 안에서 돈다. 내 PC 에서 에이전트에게 보이는 것은 `C:\harness-crud-lab` 하나뿐이다.
권한 (일부)	API 키는 `.env` 에만 있고 Git 에 올라가지 않는다. 키에 사용 한도를 걸었다.
규칙·검증	**아직 없다.** 5교시에 세운다.

자주 만나는 오류

증상	원인과 해결
`docker compose ps` 에서 `db` 가 계속 `starting` 이거나 `unhealthy`, 또는 사라짐	**[PS]** `docker compose logs db` 로 마지막 20줄을 읽는다. `Operation not permitted`, `wrong ownership`, `could not change permissions` 가 보이면 윈도우 폴더의 권한 문제다. `docs/troubleshooting.md` 의 "DB 가 켜지지 않는다"를 따라 예비 설정으로 바꾼다.
`error during connect … docker_engine`	Docker Desktop 이 실행 중이 아니다. 시작 메뉴에서 실행하고 "Engine running" 을 기다린다.
`Bind for 0.0.0.0:8000 failed: port is already allocated`	8000 번 포트를 다른 프로그램이 쓰고 있다. 그 프로그램을 끄거나, `docker-compose.yml` 의 `"8000:8000"` 을 `"8001:8000"` 으로 바꾸고 `http://localhost:8001` 로 접속한다.
`echo ${#OPENROUTER_API_KEY}` 가 `0`	`.env` 를 고친 뒤 컨테이너를 다시 만들지 않았다. **[PS]** `docker compose up -d` 를 실행하고(바뀐 설정이 있으면 컨테이너를 다시 만든다) dev 에 다시 들어간다. 파일 이름이 `.env.txt` 로 저장되지 않았는지도 확인한다.
`ask_ai.py` 가 `401`	키를 잘못 붙여 넣었다. 앞뒤 공백, 따옴표, 빠진 글자를 확인한다.
`ask_ai.py` 가 `402`	크레딧이 없거나 키의 한도를 다 썼다. OpenRouter 의 Credits·Keys 화면을 확인한다.
`docker compose up` 중 OpenCode 설치 단계에서 실패	인터넷 연결이나 보안 프로그램이 내려받기를 막은 경우다. 다시 실행해 본다. 계속 실패하면 강사에게 알린다.
`http://localhost:8000` 이 열리지 않는다	**[PS]** `docker compose logs app` 을 읽는다. 마지막 줄에 `Uvicorn running on http://0.0.0.0:8000` 이 있어야 한다.

확인 문제

1. 이미지와 컨테이너의 관계를 붕어빵 말고 다른 비유로 설명하시오.
2. `docker-compose.yml` 의 `- .:/workspace` 에서 `.` 은 내 PC 의 어느 폴더인가? 그렇게 판단한 근거는?
3. 컨테이너를 지웠다가 다시 만들어도 메모 데이터가 남는 이유를 "바인드 마운트"라는 낱말을 써서 설명하시오.
4. `.env` 와 `.env.example` 은 각각 Git 에 올라가는가? 왜 두 파일로 나누었는가?
5. API 키가 `.env` 에서 `ask_ai.py` 까지 전달되는 길을 순서대로 쓰시오.
6. 8단계(스크립트)와 9단계(에이전트)에서 AI 가 파일 내용을 알게 된 방법은 어떻게 달랐는가?
7. 에이전트를 내 PC 에 직접 설치하지 않고 컨테이너 안에서 실행하면 무엇이 안전해지는가?

3교시. Python·FastAPI 기본 이해

시간	2시간
실습 결과	`/hello` API
시작 상태	2교시를 마친 상태 (컨테이너 3개가 켜져 있고, `ask_ai.py` 가 동작한다)

수업의 흐름에서 이번 교시의 위치

```
① 요구사항 작성 → ② AI 에게 지시 → ③ 하네스로 제어 → ④ 실행·테스트 → ⑤ 수정·재검증·저장
                                                    └ 이번 교시: 실행하고, 테스트하고, 저장하는 일을 손으로 한 번 해 본다
```

AI 에게 맡기기 전에, 맡길 일이 무엇인지 **한 번은 손으로** 해 본다. 한 번 해 본 일이라야 AI 가 한 결과를 읽을 수 있다.

학습 목표

1. 파이썬의 값·변수·딕셔너리·함수를 읽을 수 있다.
2. FastAPI 코드에서 "주소 → 함수 → 응답"의 연결을 찾을 수 있다.
3. `/hello` 주소를 직접 추가하고 브라우저와 `/docs` 에서 확인한다.
4. AI 에게 코드 설명을 요청하고, 그 설명이 맞는지 실행해서 확인한다.
5. 테스트를 실행하고, 통과한 코드를 Git 에 저장(commit)한다.

1. 파이썬 15분 맛보기

dev 컨테이너에 들어가 파이썬을 대화식으로 실행한다.

■ [PS] 윈도우 PowerShell

```
cd C:\harness-crud-lab
docker compose exec dev bash
```

■ [dev] dev 컨테이너

```
python
```

`>>>` 가 나오면 파이썬이 명령을 기다리는 중이다. 한 줄씩 치고 Enter 를 누른다. `>>>` 뒤만 친다.

값과 변수

```
>>> title = "장보기"
>>> count = 3
>>> title
'장보기'
>>> count + 1
4
```

- 따옴표로 감싼 것은 **글자(문자열, str)**, 감싸지 않은 숫자는 **정수(int)** 다.
- `=` 는 "같다"가 아니라 "오른쪽 값을 왼쪽 이름에 담는다"는 뜻이다.

f-문자열 — 글자 안에 값 끼워 넣기

```
>>> name = "철수"
>>> f"안녕하세요, {name}!"
'안녕하세요, 철수!'
```

따옴표 앞에 `f` 를 붙이면 `{ }` 안의 변수가 값으로 바뀐다.

딕셔너리 — 이름표가 붙은 값 묶음

```
>>> memo = {"id": 1, "title": "장보기", "content": "우유, 달걀"}
>>> memo["title"]
'장보기'
>>> memo["content"] = "우유"
>>> memo
{'id': 1, 'title': '장보기', 'content': '우유'}
```

`{"이름": 값}` 모양이다. 메모 한 건을 이렇게 표현한다. 이 모양을 기억해 둔다. 서버가 주고받는 **JSON** 이 거의 똑같이 생겼다.

리스트 — 순서가 있는 값 묶음

```
>>> memos = []
>>> memos.append(memo)
>>> len(memos)
1
```

`[ ]` 는 리스트다. 메모 여러 건은 "딕셔너리가 든 리스트"로 표현한다.

함수 — 이름을 붙인 일 묶음

```
>>> def hello(name="하네스"):
...     return f"안녕하세요, {name}!"
...
>>> hello()
'안녕하세요, 하네스!'
>>> hello("철수")
'안녕하세요, 철수!'
```

- `def 이름(인자):` 로 만든다. 둘째 줄부터는 **띄어쓰기 4칸**으로 들여 쓴다. 파이썬에서 들여쓰기는 "여기까지가 함수 안"이라는 표시다.
- `...` 에서 Enter 를 한 번 더 치면 함수 만들기가 끝난다.
- `name="하네스"` 는 "값을 안 주면 하네스를 쓴다"는 뜻이다(기본값).
- `return` 은 함수가 돌려주는 값이다.

파이썬에서 나온다.

```
>>> exit()
```

여기까지가 이 교시에 필요한 파이썬의 전부다. 나머지는 코드를 읽다가 나올 때마다 AI 에게 묻는다.

2. FastAPI 코드 읽기

VS Code 에서 `app/main.py` 를 연다. 주석을 빼면 일곱 줄이다.

```
from fastapi import FastAPI

app = FastAPI(title="메모 CRUD")
```

```
@app.get("/health")
def health() -> dict:
    return {"status": "ok"}
```

줄	뜻
`from fastapi import FastAPI`	fastapi 라는 패키지에서 FastAPI 라는 도구를 가져온다.
`app = FastAPI(...)`	서버(앱)를 하나 만들어 `app` 이라는 이름에 담는다.
`@app.get("/health")`	**바로 아래 함수**를 `/health` 주소에 연결한다. "누가 `/health` 를 GET 방식으로 부르면 이 함수를 실행해라."
`def health() -> dict:`	함수 이름은 `health`, 돌려주는 것은 딕셔너리(`dict`)라는 표시다.
`return {"status": "ok"}`	이 딕셔너리가 JSON 으로 바뀌어 브라우저로 간다.

`@` 로 시작하는 줄을 **데코레이터**라고 한다. 지금은 "아래 함수에 붙이는 이름표"라고만 알면 된다.

```
브라우저: http://localhost:8000/health
       |  GET /health
       ▼
app 컨테이너의 FastAPI  ──►  @app.get("/health") 가 붙은 함수 health() 실행
       |
       ▼  {"status": "ok"}  (딕셔너리 → JSON)
브라우저 화면에 표시
```

따라 하기

1단계. `/hello` 추가하기 (15분)

VS Code 에서 `app/main.py` 의 **맨 아래**에 다음을 추가하고 저장(Ctrl+S)한다. 위의 `health` 함수와 두 줄 띄운다.

```
@app.get("/hello")
def hello(name: str = "하네스") -> dict:
    """/hello?name=철수 처럼 부르면 이름을 넣어 인사한다."""
    # name: str = "하네스" → 글자(str)를 받고, 안 주면 "하네스"를 쓴다는 뜻
    return {"message": f"안녕하세요, {name}!"}
```

1절에서 만든 `hello` 함수와 거의 같다. 달라진 것은 두 가지다.

- 위에 `@app.get("/hello")` 가 붙었다. → 주소에 연결되었다.
- 글자가 아니라 딕셔너리를 돌려준다. → JSON 으로 나간다.

서버를 다시 켤 필요는 없다. `app` 컨테이너는 `app` 폴더의 파일이 바뀌면 저절로 다시 켜진다. 정말 그런지 로그로 본다.

■ **[PS] 윈도우 PowerShell** (PowerShell 창을 하나 더 연다)

```
cd C:\harness-crud-lab
docker compose logs app --tail 5
```

`Reloading...` 과 `Application startup complete.` 가 보이면 다시 켜진 것이다.

> 내가 윈도우에서 파일을 고쳤는데 컨테이너 안의 서버가 알아챘다. 2교시의 바인드 마운트(`C:\harness-crud-lab ↔ /workspace`) 덕분이다.

2단계. 브라우저로 확인하기 (10분)

브라우저에서 차례로 연다.

주소	화면에 보이는 것
http://localhost:8000/hello	`{"message":"안녕하세요, 하네스!"}`
http://localhost:8000/hello?name=철수	`{"message":"안녕하세요, 철수!"}`
http://localhost:8000/nothing	`{"detail":"Not Found"}`

- 주소 뒤의 `?name=철수` 를 **쿼리 파라미터**라고 한다. 이 값이 함수의 인자 `name` 으로 들어간다.
- 세 번째처럼 연결된 함수가 없는 주소를 부르면 FastAPI 가 "그런 주소 없음"이라고 답한다.

3단계. /docs — 자동으로 만들어지는 설명서 (10분)

브라우저에서 http://localhost:8000/docs 를 연다.

1. `GET /health` 와 `GET /hello` 두 줄이 보인다. 우리가 문서를 쓴 적이 없는데 FastAPI 가 코드를 읽고 만든 것이다.
2. `GET /hello` 를 눌러 펼친다. 함수 아래에 적은 설명(`"""…"""`)이 그대로 보인다.
3. **Try it out** 을 누르고, `name` 칸에 자기 이름을 넣고, **Execute** 를 누른다.
4. 아래 **Response body** 에 결과가, **Code** 에 `200` 이 나온다.

`200` 은 "성공"을 뜻하는 상태 코드다. 4교시에 자세히 배운다. 앞으로 API 를 만들면 항상 이 화면에서 먼저 눌러 본다.

4단계. AI 에게 설명시키고, 맞는지 확인하기 (20분)

AI 를 "코드를 쓰는 사람"이 아니라 "설명해 주는 선생님"으로 써 본다.

■ [dev] dev 컨테이너

```
python scripts/ask_ai.py --file app/main.py "이 파일을 처음 보는 초보자에게 한 줄씩 설명해 줘. 데코레이터가 무엇인지도 비유로 알려 줘."
```

답을 읽는다. 그리고 **그대로 믿지 않는다.** AI 는 그럴듯하게 틀릴 수 있다. 설명이 맞는지 실행으로 확인하는 습관을 지금 들인다.

확인 실험 1. AI 에게 묻는다.

■ [dev] dev 컨테이너

```
python scripts/ask_ai.py --file app/main.py "hello 함수에서 name 의 기본값 '하네스' 를 지우고 name: str 만 남기면, /hello 를 name 없이
  불렀을 때 어떻게 돼? 상태 코드까지 예측해 줘."
```

AI 의 예측을 적어 둔다. 그다음 **직접 해 본다.** `app/main.py` 에서 `name: str = "하네스"` 를 `name: str` 로 고쳐 저장하고, 브라우저에서 http://localhost:8000/hello 를 연다.

- AI 의 예측과 실제 결과가 같은가?
- `/docs` 에서 `name` 칸 옆에 무엇이 새로 붙었는가?

확인했으면 `name: str = "하네스"` 로 **되돌려 놓고** 저장한다.

확인 실험 2. 에이전트에게 설명시킨다. 스크립트와 달리 파일 이름만 알려 주면 스스로 읽는다.

■ [dev] dev 컨테이너

```
opencode run "app/main.py 를 읽고, 브라우저가 /hello?name=철수 를 불렀을 때 응답이 나가기까지 일어나는 일을 순서대로 설명해 줘. 파일은
  고치지 마."
```

> "파일은 고치지 마"를 꼭 붙인다. 지금은 규칙도 권한도 없어서, 말로 부탁하는 것 말고는 막을 방법이 없다. 이 불편함을 기억해 둔다. 5교시에 해결한다.

5단계. 테스트 실행하기 (15분)

지금까지는 브라우저를 눈으로 보고 "된다"고 판단했다. 이 확인을 **코드로 적어 둔 것**이 테스트다.

`tests/test_health.py` 를 VS Code 로 연다.

```
def test_health(client):
    response = client.get("/health")  # GET /health 를 부른다
    assert response.status_code == 200  # 성공(200)이어야 한다
    assert response.json() == {"status": "ok"}  # 답의 내용도 정확히 같아야 한다
```

- `client` 는 가짜 브라우저다. `tests/conftest.py` 에서 만든다.
- `assert 식` 은 "이 식이 참이어야 한다"는 뜻이다. 거짓이면 테스트가 실패한다.

실행한다.

■ [dev] dev 컨테이너

```
pytest -q
```

```
.                                                                    [100%]
1 passed in 0.02s
```

점(`.`) 하나가 통과한 테스트 하나다.

이제 `/hello` 의 테스트를 추가한다. `tests/test_health.py` 의 **맨 아래**에 붙여 넣고 저장한다.

```


def test_hello_default(client):
    response = client.get("/hello")
    assert response.status_code == 200
    assert response.json() == {"message": "안녕하세요, 하네스!"}


def test_hello_with_name(client):
    # params 는 주소 뒤에 ?name=철수 를 붙여 준다.
    response = client.get("/hello", params={"name": "철수"})
    assert response.json() == {"message": "안녕하세요, 철수!"}
```

■ [dev] dev 컨테이너

```
pytest -q
```

```
...                                                                  [100%]
3 passed in 0.03s
```

일부러 실패시켜 본다. `app/main.py` 에서 `안녕하세요` 를 `반갑습니다` 로 바꿔 저장하고 `pytest -q` 를 다시 실행한다.

출력에서 아래 세 가지를 찾는다.

- `FAILED tests/test_health.py::test_hello_default` — 어떤 테스트가 실패했는가
- `assert` 줄 아래의 비교 — 무엇을 기대했고 실제로 무엇이 왔는가
- 마지막 줄 `2 failed, 1 passed` — 몇 개가 실패했는가

코드를 바꿨더니 테스트가 알려 줬다. **테스트는 "누군가(나 또는 AI)가 약속을 깨뜨렸을 때 울리는 경보기"다.**

`반갑습니다` 를 `안녕하세요` 로 **되돌리고** `pytest -q` 로 `3 passed` 를 확인한다.

코드 검사도 실행해 본다. 검사와 테스트를 한꺼번에 돌리는 스크립트가 `scripts/verify.sh` 다.

■ [dev] dev 컨테이너

```
sh scripts/verify.sh
```

```
[1/2] 코드 검사 (ruff) ---------------------------
All checks passed!
[2/2] 자동 테스트 (pytest) -----------------------
...                                                                  [100%]
3 passed in 0.03s
-----------------------------------------------------
검증 통과: 이제 저장(commit)해도 됩니다.
```

6단계. Git 에 저장하기 (15분)

커밋(commit) 은 "지금 이 상태"를 기록으로 남기는 일이다. 게임의 저장 지점과 같다. 나중에 망가져도 이 지점으로 돌아올 수 있다.

(1) 무엇이 바뀌었는지 본다.

■ [dev] dev 컨테이너

```
git status
```

`modified:` 뒤에 `app/main.py` 와 `tests/test_health.py` 두 파일이 나온다.

■ [dev] dev 컨테이너

```
git diff
```

`+` 로 시작하는 초록색 줄이 추가된 줄, `-` 로 시작하는 빨간색 줄이 지워진 줄이다. 화면이 길면 방향키로 움직이고 `q` 로 나온다.

이 화면을 읽는 것이 이 과정에서 가장 중요한 기술이다. 6교시부터는 AI 가 바꾼 내용을 이 화면으로 읽고 검토한다.

(2) 저장할 파일을 고른다.

■ [dev] dev 컨테이너

```
git add app/main.py tests/test_health.py
git status
```

두 파일이 `Changes to be committed:` 아래로 옮겨졌다. "이번 커밋에 넣겠다"고 표시한 것이다.

(3) 저장한다.

■ [dev] dev 컨테이너

```
git commit -m "3교시: /hello 주소와 테스트 추가"
```

`-m` 뒤의 글이 커밋 메시지다. **무엇을 했는지** 한 줄로 적는다.

(4) 기록을 본다.

■ [dev] dev 컨테이너

```
git log --oneline -3
```

맨 위에 방금 만든 커밋이 있다. 앞의 일곱 글자는 커밋의 고유 번호다.

확인. `git status` 가 `nothing to commit, working tree clean` 을 출력한다.

하네스 관점 — 이번 교시에서 본 것

이번 교시에는 수업의 흐름 ④·⑤를 **사람이 손으로** 했다.

한 일	손으로 한 방법	5교시부터는
실행·확인	브라우저와 `/docs` 를 눈으로 봤다	테스트가 대신 확인한다
검증	`sh scripts/verify.sh` 를 내가 기억해서 실행했다	커밋할 때 자동으로 실행된다
AI 통제	"파일은 고치지 마"라고 매번 말로 부탁했다	규칙과 권한이 대신한다
저장	검증을 건너뛰고도 커밋할 수 있었다	검증에 실패하면 커밋이 취소된다

오른쪽 칸이 하네스다. 사람이 기억하고 지켜야 하던 일을 장치가 대신하게 만드는 것이다.

자주 만나는 오류

증상	원인과 해결
브라우저에 `{"detail":"Not Found"}`	주소를 잘못 쳤거나, 파일을 저장하지 않았다. VS Code 탭에 흰 점이 있으면 저장되지 않은 것이다.
브라우저가 "사이트에 연결할 수 없음"	코드에 문법 오류가 있어 서버가 켜지지 못했다. **[PS]** `docker compose logs app --tail 20` 에서 `SyntaxError` 나 `IndentationError` 와 줄 번호를 찾는다.
`IndentationError`	들여쓰기가 맞지 않는다. 함수 안의 줄은 띄어쓰기 4칸이다. 탭과 띄어쓰기를 섞지 않는다.
`/docs` 에 `/hello` 가 안 보인다	`@app.get("/hello")` 줄이 빠졌거나 함수 위에 빈 줄 없이 붙어 있지 않다. 데코레이터는 함수 바로 윗줄에 있어야 한다.
`pytest` 가 `command not found`	윈도우 PowerShell 에서 쳤다. pytest 는 컨테이너 안에만 있다. `docker compose exec dev bash` 로 들어가서 친다.
`git commit` 이 `Please tell me who you are`	2교시 10단계(이름·메일 설정)를 하지 않았다.
한글이 `\uc548\ub155` 처럼 보인다	오래된 브라우저 확장이나 다른 도구로 볼 때 그렇다. `/docs` 의 Response body 로 확인한다.

확인 문제

1. `memo = {"id": 1, "title": "장보기"}` 에서 제목을 꺼내는 코드를 쓰시오.
2. `@app.get("/hello")` 한 줄을 지우면 `hello` 함수는 어떻게 되는가? 브라우저에서 `/hello` 를 부르면 무엇이 나오는가?
3. `/hello?name=영희` 에서 `영희` 는 어떤 길을 거쳐 응답의 글자가 되는가? 순서대로 쓰시오.
4. 파일을 고쳤는데 서버를 다시 켜지 않아도 반영된 이유 두 가지를 쓰시오. (힌트: 2교시의 낱말 하나, `docker-compose.yml` 의 옵션 하나)
5. `assert response.status_code == 200` 은 무엇을 확인하는가? 이 줄이 실패하면 화면에 무엇이 나오는가?
6. AI 가 코드를 설명해 줬을 때 그 설명이 맞는지 확인하는 방법을 두 가지 쓰시오.
7. `git add` 와 `git commit` 은 각각 무슨 일을 하는가?

4교시. CRUD 와 HTTP 기초

시간	2시간
실습 결과	GET·POST 테스트
시작 상태	3교시를 마친 상태 (`/hello` 가 동작하고 커밋되어 있다)

막혔거나 3교시 결과가 없으면 **[dev]** `sh scripts/checkpoint.sh 03` 으로 3교시 완성 예시를 가져온다.

수업의 흐름에서 이번 교시의 위치

```
① 요구사항 작성 → ② AI 에게 지시 → ③ 하네스로 제어 → ④ 실행·테스트 → ⑤ 수정·재검증·저장
└ 이번 교시: 요구사항에 쓰이는 "말"(주소·방식·상태 코드·JSON)을 배운다
```

`harness/REQUIREMENTS.md` 는 "`POST /memos` 를 부르면 201 을 돌려준다"처럼 적혀 있다. 이 문장을 읽고 쓸 수 있어야 AI 에게 정확히 시킬 수 있고, AI 의 결과가 맞는지 판단할 수 있다.

학습 목표

1. CRUD 네 가지를 HTTP 방식(POST·GET·PUT·DELETE)과 짝지을 수 있다.
2. 요청과 응답이 무엇으로 이루어지는지 말할 수 있다.
3. 상태 코드 200·201·204·404·422 의 뜻을 안다.
4. 메모를 만들고(POST) 보는(GET) API 를 손으로 만들고, `/docs` 와 `curl` 로 시험한다.
5. 메모리에 저장한 데이터가 서버를 다시 켜면 사라지는 것을 확인한다.

1. 개념

CRUD

데이터를 다루는 일은 결국 네 가지다.

	뜻	메모장에서
Create	만들기	새 메모 쓰기
Read	읽기	메모 목록 보기, 메모 한 건 보기
Update	고치기	메모 내용 바꾸기
Delete	지우기	메모 지우기

HTTP — 브라우저와 서버가 주고받는 말

브라우저(클라이언트)가 **요청**을 보내고 서버가 **응답**을 돌려준다. 한 번 묻고 한 번 답하면 끝이다.

요청은 세 부분이다.

부분	뜻	예
방식(메서드)	무엇을 하려는가	`GET`, `POST`, `PUT`, `DELETE`
주소(경로)	무엇에 대해서인가	`/memos`, `/memos/1`
본문(body)	함께 보내는 데이터 (없을 수도 있다)	`{"title": "장보기"}`

응답은 두 부분이다.

부분	뜻	예
상태 코드	어떻게 되었는가 (세 자리 숫자)	`200`, `404`
본문	돌려주는 데이터 (없을 수도 있다)	`{"id": 1, "title": "장보기"}`

CRUD ↔ HTTP ↔ SQL

같은 일을 세 가지 말로 부른다. 이 표는 3일 내내 쓴다.

하는 일	HTTP 방식 + 주소	성공 상태 코드	SQL (7교시)
만들기	`POST /memos`	201	`INSERT`
목록 보기	`GET /memos`	200	`SELECT`
한 건 보기	`GET /memos/1`	200	`SELECT … WHERE`
고치기	`PUT /memos/1`	200	`UPDATE … WHERE`
지우기	`DELETE /memos/1`	204	`DELETE … WHERE`

주소가 `/memos` 하나인데 방식에 따라 하는 일이 달라진다는 점에 주목한다. 주소는 **무엇**(메모들)을 가리키고, 방식은 **어떻게**(만든다/본다)를 말한다.

상태 코드

코드	이름	뜻	메모장에서
200	OK	성공	목록 보기, 고치기 성공
201	Created	성공했고 새로 만들어졌다	메모 만들기 성공
204	No Content	성공했고 돌려줄 내용이 없다	메모 지우기 성공
404	Not Found	그런 것이 없다	없는 번호의 메모를 찾을 때
422	Unprocessable Content	보낸 데이터가 규칙에 맞지 않는다	제목 없이 만들려 할 때
500	Internal Server Error	서버 코드가 오류를 냈다	버그

첫 자리로 큰 뜻을 안다. **2xx 는 성공, 4xx 는 요청한 쪽의 잘못, 5xx 는 서버의 잘못.**

JSON

요청과 응답의 본문은 **JSON** 이라는 형식으로 쓴다. 파이썬의 딕셔너리·리스트와 거의 똑같이 생겼다.

```
{"id": 1, "title": "장보기", "content": "우유, 달걀"}
```

- 이름과 글자는 **큰따옴표**로 감싼다. 작은따옴표는 안 된다.
- 메모 여러 건은 `[ {…}, {…} ]` 처럼 대괄호로 감싼다.

따라 하기

1단계. 메모 만들기·보기 API 손으로 쓰기 (25분)

VS Code 에서 `app/main.py` 를 연다. 아래 세 군데를 고친다. 다 고친 모습은 `checkpoints/lesson04/app/main.py` 와 같다.

(1) 맨 위의 import 에 한 줄을 추가한다.

```
from fastapi import FastAPI
```

```
from pydantic import BaseModel
```

(2) `app = FastAPI(...)` 줄 아래에 추가한다.

```
# 메모를 담아 둘 리스트. 서버가 켜져 있는 동안에만 유지된다.
memos: list[dict] = []


class MemoIn(BaseModel):
    """메모를 만들 때 클라이언트가 보내는 데이터의 모양"""

    title: str  # 제목: 글자여야 하고, 반드시 보내야 한다
    content: str = ""  # 내용: 안 보내면 빈 문자열
```

`class MemoIn(BaseModel)` 은 "요청 본문은 이런 모양이어야 한다"는 틀이다. `title` 에는 기본값이 없으므로 **반드시 보내야 하고**, `content` 는 기본값이 `""` 이므로 안 보내도 된다.

(3) 파일 맨 아래에 추가한다.

```
@app.post("/memos", status_code=201)
def create_memo(memo: MemoIn) -> dict:
    """메모 만들기. 성공하면 201(Created)과 함께 만든 메모를 돌려준다."""
    new_memo = {
        "id": len(memos) + 1,  # 번호는 1, 2, 3 ... 순서대로
        "title": memo.title,
        "content": memo.content,
    }
    memos.append(new_memo)  # 리스트에 넣는다
    return new_memo


@app.get("/memos")
def list_memos() -> list[dict]:
    """메모 목록 보기"""
    return memos
```

- `@app.post` — 이번에는 `get` 이 아니라 `post` 다. 같은 주소 `/memos` 에 방식만 다른 함수 두 개를 연결했다.
- `status_code=201` — 성공하면 200 대신 201 을 돌려주라는 뜻이다.
- `memo: MemoIn` — 요청 본문을 `MemoIn` 모양으로 받는다. 모양이 맞지 않으면 이 함수는 실행되지도 않는다.

저장하고, 서버가 다시 켜졌는지 확인한다.

■ [PS] 윈도우 PowerShell

```
docker compose logs app --tail 5
```

`Application startup complete.` 가 마지막 근처에 있으면 된다. 오류가 보이면 줄 번호를 보고 오타를 고친다.

2단계. /docs 에서 시험하기 (15분)

http://localhost:8000/docs 를 새로 고친다. `POST /memos` 와 `GET /memos` 가 생겼다.

(1) 목록 보기. `GET /memos` → Try it out → Execute.

- Code: `200`
- Response body: `[]` (아직 메모가 없다)

(2) 만들기. `POST /memos` → Try it out. Request body 칸을 아래로 바꾸고 Execute.

```
{"title": "장보기", "content": "우유, 달걀"}
```

- Code: `201`
- Response body: `{"id":1,"title":"장보기","content":"우유, 달걀"}`

(3) 하나 더. 이번에는 내용 없이 보낸다.

```
{"title": "운동"}
```

- Response body: `{"id":2,"title":"운동","content":""}` — `content` 가 빈 문자열로 채워졌다.

(4) 다시 목록 보기. `GET /memos` → Execute. 두 메모가 대괄호 안에 들어 있다.

3단계. `curl` 로 시험하기 (20분)

`/docs` 는 편하지만 마우스로 눌러야 한다. 같은 일을 명령 한 줄로 하는 도구가 `curl` 이다. AI 에이전트도 서버를 확인할 때 `curl` 을 쓴다.

dev 컨테이너에서 실행한다. 주소가 `localhost` 가 아니라 **app** 이라는 점에 주의한다. 컨테이너끼리는 서비스 이름으로 서로를 부른다.

■ [dev] dev 컨테이너

```
curl -s http://app:8000/memos
```

```
[{"id":1,"title":"장보기","content":"우유, 달걀"},{"id":2,"title":"운동","content":""}]
```

`curl` 은 응답 끝에 줄바꿈을 붙이지 않는다. 결과 바로 뒤에 프롬프트가 붙어 나와도 정상이다.

POST 로 만들기. 옵션이 세 가지 붙는다.

■ [dev] dev 컨테이너

```
curl -s -X POST http://app:8000/memos -H "Content-Type: application/json" -d '{"title": "책 읽기", "content": "30쪽"}
  '
```

```
{"id":3,"title":"책 읽기","content":"30쪽"}
```

옵션	뜻
`-s`	진행 표시를 숨긴다
`-X POST`	방식을 POST 로 한다 (안 쓰면 GET)
`-H "Content-Type: application/json"`	"본문은 JSON 입니다"라고 알린다
`-d '…'`	본문. 바깥은 작은따옴표, 안의 JSON 은 큰따옴표

상태 코드만 보기.

■ [dev] dev 컨테이너

```
curl -s -o /dev/null -w "%{http_code}\n" http://app:8000/memos
```

```
200
```

규칙에 맞지 않는 요청 보내기. 제목 없이 내용만 보낸다.

■ [dev] dev 컨테이너

```
curl -s -X POST http://app:8000/memos -H "Content-Type: application/json" -d '{"content": "제목이 없어요"}'
```

```
{"detail":[{"type":"missing","loc":["body","title"],"msg":"Field required","input":{"content":"제목이 없어요"}}]}
```

읽는 법: `"type":"missing"` — 빠졌다. `"loc":["body","title"]` — 본문의 title 이. `"msg":"Field required"` — 꼭 있어야 하는 칸이다.

상태 코드도 확인한다.

■ [dev] dev 컨테이너

```
curl -s -o /dev/null -w "%{http_code}\n" -X POST http://app:8000/memos -H "Content-Type: application/json" -d '{"content": "제목이 없어
  요"}'
```

```
422
```

우리는 "제목이 없으면 422 를 돌려줘라"는 코드를 쓴 적이 없다. `MemoIn` 에 `title: str` 이라고 **모양만 적어 두었더니** FastAPI 가 검사해 준 것이다.

4단계. 요구사항과 비교하기 (15분)

`harness/REQUIREMENTS.md` 의 R1, R2 를 읽고, 지금 만든 API 가 요구사항을 지키는지 한 줄씩 대조한다.

요구사항	지금 코드	맞는가
R1. `POST /memos` → 201	201 이 온다	
R1. 응답에 id, title, content, **created_at, updated_at**	?	
R1. content 를 빼면 `""`	?	
R2. `GET /memos` → 200, 배열	?	
R2. **나중에 만든 메모가 앞에**	?	
R2. 없으면 `[]`	?	
R6. title 이 없으면 422	422 가 온다	
R6. title 이 **빈 문자열**(`""`)이면 422	?	

마지막 줄은 직접 보내서 확인한다.

■ [dev] dev 컨테이너

```
curl -s -o /dev/null -w "%{http_code}\n" -X POST http://app:8000/memos -H "Content-Type: application/json" -d '{"title": ""}'
```

결과를 표에 적는다. **요구사항을 지키지 못한 항목이 몇 개인가?**

이것이 "돌아가는 것 같다"와 "요구사항을 만족한다"의 차이다. 화면에서 눌러 보면 잘 되는 것처럼 보이지만, 명세와 한 줄씩 대조하면 빠진 것이 드러난다. 6교시에 이 빈칸들을 AI 에이전트에게 채우게 한다.

5단계. 서버를 다시 켜면? (10분)

지금 메모가 몇 개인지 본다.

■ [dev] dev 컨테이너

```
curl -s http://app:8000/memos
```

서버를 다시 켠다.

■ [PS] 윈도우 PowerShell

```
docker compose restart app
```

몇 초 뒤 다시 본다.

■ [dev] dev 컨테이너

```
curl -s http://app:8000/memos
```

```
[]
```

메모가 모두 사라졌다. `memos` 는 파이썬 리스트이고, 리스트는 프로그램의 메모리에 있다. 프로그램이 꺼지면 함께 없어진다.

파일을 고쳐서 서버가 저절로 다시 켜질 때도 마찬가지로 사라진다. 요구사항 N1("서버를 다시 켜도 메모가 남아 있다")을 지킬 수 없다. 그래서 7~8교시에 데이터베이스가 필요해진다.

6단계. 저장하기 (5분)

■ [dev] dev 컨테이너

```
sh scripts/verify.sh
git status
git diff
git add app/main.py
git commit -m "4교시: 메모 만들기·목록 보기(메모리 저장) 추가"
```

`git diff` 로 바뀐 내용을 읽은 뒤에 커밋한다. 이 순서를 습관으로 만든다.

하네스 관점 — 이번 교시에서 본 것

- 요구사항은 **HTTP 의 말**로 쓴다. "`POST /memos` → 201". 이 말을 알아야 요구사항을 쓸 수 있고, AI 의 결과를 판단할 수 있다.
- 4단계에서 한 "명세와 한 줄씩 대조하기"는 사람이 하면 느리고 빠뜨리기 쉽다. 이 대조를 코드로 적어 두면 **테스트**가 된다. 6교시에 AI 가 요구사항 번호마다 테스트를 만들게 한다.
- `curl` 과 상태 코드는 AI 에이전트가 자기 작업을 확인할 때 쓰는 도구이기도 하다. 사람이 읽을 수 있어야 AI 의 확인이 제대로 된 것인지 알 수 있다.

자주 만나는 오류

증상	원인과 해결
`curl: (6) Could not resolve host: app`	윈도우 PowerShell 에서 쳤다. `app` 이라는 이름은 컨테이너 안에서만 통한다. dev 컨테이너에서 친다.
`curl: (7) Failed to connect to app port 8000`	서버가 꺼져 있다. 코드에 오류가 있을 가능성이 높다. **[PS]** `docker compose logs app --tail 20` 을 읽는다.
`{"detail":"Method Not Allowed"}` (405)	주소는 있는데 그 방식에 연결된 함수가 없다. `-X POST` 를 빠뜨렸거나 `@app.post` 를 `@app.get` 으로 썼다.
`{"detail":[{"type":"json_invalid", …`	본문이 올바른 JSON 이 아니다. 작은따옴표를 썼거나, 쉼표·중괄호가 빠졌다.
`NameError: name 'BaseModel' is not defined`	맨 위의 `from pydantic import BaseModel` 을 빠뜨렸다.
PowerShell 에서 `curl … -d '{"title": …}'` 이 422	PowerShell 은 따옴표를 다르게 처리한다. 본문이 있는 `curl` 은 dev 컨테이너에서 친다.
방금 만든 메모가 없어졌다	파일을 저장해서 서버가 다시 켜졌다. 5단계에서 본 현상이다. 정상이다.

확인 문제

1. CRUD 네 가지를 HTTP 방식·주소와 짝지어 표로 쓰시오.
2. `GET /memos` 와 `POST /memos` 는 주소가 같다. 서버는 둘을 어떻게 구분하는가?
3. 상태 코드 201 과 204 는 둘 다 성공이다. 무엇이 다른가?
4. 없는 번호의 메모를 조회하면 어떤 상태 코드가 와야 하는가? 왜 200 과 빈 내용을 돌려주면 안 되는가?
5. `{'title': '장보기'}` 는 올바른 JSON 인가? 아니라면 어디가 틀렸는가?
6. `title: str` 과 `content: str = ""` 의 차이를 "반드시"라는 낱말을 써서 설명하시오.

7. 지금 코드에서 메모 번호를 `len(memos) + 1` 로 정한다. 9교시에 "지우기"가 생기면 이 방식에 어떤 문제가 생기는가?
8. 서버를 다시 켰을 때 메모가 사라진 이유를 설명하시오.

5교시. 하네스 규칙 작성

시간	2시간
실습 결과	`AGENTS.md` (그리고 권한 설정과 검증 장치)
시작 상태	4교시를 마친 상태 (GET·POST `/memos` 가 메모리 저장으로 동작하고 커밋되어 있다)

막혔거나 4교시 결과가 없으면 **[dev]** `sh scripts/checkpoint.sh 04` 로 4교시 완성 예시를 가져온다.

수업의 흐름에서 이번 교시의 위치

```
① 요구사항 작성 → ② AI 에게 지시 → ③ 하네스로 제어 → ④ 실행·테스트 → ⑤ 수정·재검증·저장
                                  └ 이번 교시: 하네스를 직접 세운다 ┘
```

이 과정에서 가장 중요한 교시다. 여기서 세운 장치가 6교시부터 AI 의 모든 작업을 통제한다.

학습 목표

1. 규칙 없이 일하는 AI 에이전트의 행동을 관찰하고 문제를 짚어 낸다.
2. `AGENTS.md` 를 직접 작성해 에이전트가 따를 규칙을 정한다.
3. `opencode.jsonc` 의 권한 규칙(allow / ask / deny)을 읽고 적용한다.
4. 검증 스크립트와 `pre-commit` 훅으로 "통과해야만 저장된다"를 장치로 만든다.
5. 같은 지시를 다시 내려, 하네스가 있을 때 무엇이 달라지는지 비교한다.

1. 개념 — 다섯 요소를 파일로

1교시에 본 다섯 요소를 이번 교시에 하나씩 파일로 세운다.

요소	세우는 파일	종류	이번 교시의 단계
규칙	`AGENTS.md`	부탁	2단계
도구	`scripts/verify.sh` (+ 2교시의 컨테이너)	—	4단계
권한	`opencode.jsonc`	잠금장치	3단계
실행	dev 컨테이너	잠금장치	2교시에 완료
검증	`.githooks/pre-commit`	잠금장치	5단계

부탁과 잠금장치. `AGENTS.md` 는 AI 가 읽는 글이다. AI 는 대체로 따르지만 반드시 따른다는 보장은 없다. 그래서 정말 일어나면 안 되는 일은 잠금장치로 한 번 더 막는다.

일어나면 안 되는 일	부탁 (`AGENTS.md`)	잠금장치
AI 가 마음대로 커밋·푸시한다	"커밋은 사람이 한다"	`opencode.jsonc`: `git commit`, `git push` → deny
AI 가 API 키를 읽는다	".env 를 읽지 않는다"	`opencode.jsonc`: `.env` 읽기 → deny
AI 가 검증 기준을 고친다	"하네스 파일을 고치지 않는다"	`opencode.jsonc`: `scripts/verify.sh` 고치기 → deny
깨진 코드가 저장된다	"검증을 통과해야 끝난 것이다"	`pre-commit`: 검증 실패 시 커밋 취소
AI 가 내 PC 의 다른 파일을 건드린다	—	컨테이너: `/workspace` 만 보인다

따라 하기

1단계. 실험 — 규칙 없이 시켜 보기 (15분)

먼저 지금 상태가 깨끗한지 확인한다. 실험이 끝나면 이 상태로 되돌린다.

■ [dev] dev 컨테이너

```
git status
```

`nothing to commit, working tree clean` 이어야 한다. 아니라면 4교시 6단계(커밋)를 먼저 한다.

OpenCode 를 연다.

■ [dev] dev 컨테이너

```
opencode
```

아무 조건 없이 한 줄로 시킨다.

■ [AI] OpenCode 입력 칸

```
메모 삭제 기능 만들어 줘
```

AI 가 일하는 것을 지켜본다. 끝나면 `/exit` 로 나온다.

관찰 기록. 아래 표를 채운다. `git status` 와 `git diff` 로 확인한다.

■ [dev] dev 컨테이너

```
git status
git diff --stat
```

관찰할 것	결과
고치기 전에 계획을 말하고 내 승인을 기다렸는가?	
어떤 파일을 고쳤는가? 새로 만든 파일이 있는가?	
없는 번호를 지울 때의 동작을 스스로 정했는가? 어떻게 정했는가?	
성공 상태 코드는 무엇으로 했는가? (요구사항 R5 는 204)	
테스트를 만들었는가?	
검증(`pytest`)을 실행했는가?	
시키지 않은 것을 만들었는가?	
한국어 주석이 있는가?	
명령을 실행하기 전에 물어봤는가?	

결과는 사람마다 다르다. 옆 사람과 표를 비교한다. **같은 한 줄을 시켰는데 결과가 다르다**는 것 자체가 첫 번째 관찰이다.

AI 가 잘했을 수도 있다. 요즘 모델은 시키지 않아도 테스트를 만들기도 한다. 하지만 "이번에는 잘했다"와 "매번 정해진 대로 한다"는 다르다. 하네스는 뒤쪽을 만드는 일이다.

되돌리기. 실험에서 바뀐 것을 모두 버린다.

■ [dev] dev 컨테이너

```
git restore .
git status
```

`git restore .` 는 고쳐진 파일을 마지막 커밋 상태로 되돌린다. `git status` 에 `Untracked files:` (AI 가 새로 만든 파일)가 남아 있으면 그 파일들을 VS Code 에서 지운다. `working tree clean` 이 나올 때까지 한다.

마지막으로 AI 가 스스로 커밋하지 않았는지 본다.

■ [dev] dev 컨테이너

```
git log --oneline -3
```

맨 위가 "4교시: ..." 커밋이면 된다. 그 위에 내가 만들지 않은 커밋이 있다면 AI 가 허락 없이 저장한 것이다. 관찰 기록에 적고, 강사와 함께 되돌린다. (이런 일이 3단계에서 권한을 거는 이유다.)

> 방금 한 일을 돌아본다. 커밋해 둔 지점이 있었기 때문에 실험을 마음 놓고 하고 깨끗이 되돌릴 수 있었다. **Git 은 AI 와 일할 때의 안전망**이다.

2단계. 규칙 쓰기 — AGENTS.md (40분)

OpenCode 는 프로젝트 맨 위 폴더에 `AGENTS.md` 가 있으면 시작할 때 자동으로 읽는다. 지금은 그 파일이 없다.

틀을 맨 위 폴더로 복사한다.

■ [dev] dev 컨테이너

```
cp harness/AGENTS.template.md AGENTS.md
```

VS Code 에서 `AGENTS.md` 를 연다. `( )` 와 `[작성]` 을 채운다. 아래 질문을 따라 한 절씩 쓴다. **정답은 없다.** 1단계에서 본 문제를 막으려면 무엇을 적어야 하는지 생각하며 쓴다.

1절. 프로젝트 소개

AI 는 이 프로젝트를 처음 본다. 사람 신입에게 설명하듯 적는다.

- 목표: 무엇을 만드는가?
- 사용자: 이 코드를 읽는 사람은 누구인가? → **"프로그래밍을 처음 배우는 학습자"** 라고 적어 두면 AI 가 코드를 쓰는 방식이 달라진다.
- 폴더 구조 표: AI 가 고쳐도 되는 곳과 안 되는 곳을 정한다. `harness/` 와 `scripts/verify.sh` 를 AI 가 고칠 수 있으면 어떤 일이 생길까?

2절. 도구

AI 가 "검증하려면 무슨 명령을 쳐야 하지?"를 추측하지 않게 적어 준다. 3·4교시에 내가 직접 쳤던 명령들이다.

- 전체 검증 / 테스트만 / 코드 검사만 / 바뀐 것 보기
- 서버는 어디에서 켜져 있는가? (AI 가 서버를 또 켜려고 하면 포트가 겹친다)

3절. 권한

세 칸으로 나눈다. 1단계 관찰 기록의 "명령을 실행하기 전에 물어봤는가?"를 떠올린다.

- 묻지 않고 해도 되는 일 — 자주 하고 위험하지 않은 일
- 먼저 물어봐야 하는 일 — 가끔 필요하지만 사람이 알아야 하는 일
- 하면 안 되는 일 — 되돌리기 어렵거나, 사람의 판단이 필요한 일

4절. 작업 절차

1단계에서 AI 는 계획을 말했는가? 이 절에서 **"사람이 진행이라고 할 때까지 파일을 고치지 않는다"** 를 정한다. 검증에 실패하면 몇 번까지 다시 시도하게 할지도 정한다. (무한히 고치게 두면 1교시의 사례 3 이 된다.)

5절. 코드 규칙

- 주석: 내가 읽을 수 있어야 한다. 어떤 언어로, 어디에 달게 할 것인가?
- 데이터베이스: 7교시에 배울 내용이라 지금은 `harness/AGENTS.md` 의 해당 부분을 읽고 옮겨 적는다.
- 테스트: **"테스트를 통과시키려고 테스트를 고치지 않는다."** 이 한 줄이 왜 필요한지 옆 사람에게 설명해 본다.

6절. 검증 — "다 됐다"의 기준

AI 가 "완료했습니다"라고 말해도 되는 조건을 체크 항목으로 적는다.

7절. 보고 형식

틀에 이미 여섯 칸이 있다. 그대로 두거나 고친다.

다 썼으면 완성본과 비교한다.

■ [dev] dev 컨테이너

```
diff AGENTS.md harness/AGENTS.md | head -60
```

< 로 시작하는 줄이 내 것, > 로 시작하는 줄이 완성본이다. 똑같을 필요는 없다. 다음을 점검한다.

- 완성본에 있는데 내 것에 없는 규칙 중 **필요하다고 생각되는 것**을 옮겨 적는다.
- 내 것에만 있는 규칙은 그대로 둔다. 그것이 내 프로젝트의 규칙이다.

시간이 모자라면 완성본을 그대로 가져와도 된다. 단, **처음부터 끝까지 소리 내어 읽고** 가져온다.

■ [dev] dev 컨테이너

```
cp harness/AGENTS.md AGENTS.md
```

3단계. 권한 걸기 — opencode.jsonc (20분)

지금 맨 위 폴더의 opencode.jsonc 에는 모델 이름만 있다. 권한 규칙이 든 완성본으로 바꾼다.

먼저 완성본을 읽는다. VS Code 에서 harness/opencode.jsonc 를 연다.

규칙 한 줄은 세 칸이다.

```
{ "action": "shell", "resource": "git push *", "effect": "deny" }
```

칸	뜻	값
action	무엇을 하려는가	shell(명령 실행), edit(파일 고치기), read(파일 읽기), webfetch(웹 페이지 가져오기) …
resource	무엇에 대해서인가	명령이나 파일 경로. * 는 "아무 글자나"
effect	어떻게 처리하는가	allow(그냥 허용), ask(사람에게 물어봄), deny(금지)

읽는 순서가 중요하다. 위에서 아래로 읽으며, 여러 규칙에 걸리면 **가장 아래 규칙이 이긴다.** 그래서 넓은 규칙을 위에, 예외를 아래에 쓴다.

```
{ "action": "shell", "resource": "*",           "effect": "ask"   },  // 모든 명령은 물어본다
{ "action": "shell", "resource": "pytest *",    "effect": "allow" },  // 단, pytest 는 그냥 허용
{ "action": "shell", "resource": "git commit *", "effect": "deny"  },  // 단, git commit 은 금지
```

읽기 연습. harness/opencode.jsonc 를 보고 답한다.

1. AI 가 pip install requests 를 실행하려 하면? (allow / ask / deny)
2. AI 가 sh scripts/verify.sh 를 실행하려 하면?
3. AI 가 app/main.py 를 고치려 하면? (힌트: edit 규칙 중 걸리는 것이 있는가? 없으면 기본값은 allow 다)
4. AI 가 scripts/verify.sh 를 고치려 하면?
5. AI 가 .env 를 읽으려 하면? .env.example 은?

적용한다.

■ [dev] dev 컨테이너

```
cp harness/opencode.jsonc opencode.jsonc
git diff --stat
```

opencode.jsonc 한 파일이 바뀐 것으로 나온다. (AGENTS.md 는 새 파일이라 git status 에서 보인다.)

잠금장치 시험. OpenCode 는 시작할 때 설정을 읽으므로 새로 연다.

■ [dev] dev 컨테이너

```
opencode
```

막아 둔 일을 일부러 시킨다. 하나씩 시키고 무슨 일이 일어나는지 본다.

■ [AI] OpenCode 입력 칸

```
.env 파일을 읽고 안에 무슨 변수들이 있는지 알려 줘
```

■ [AI] OpenCode 입력 칸

```
git commit -m "test" 명령을 실행해 줘
```

■ [AI] OpenCode 입력 칸

```
scripts/verify.sh 의 첫 줄에 주석 한 줄을 추가해 줘
```

■ [AI] OpenCode 입력 칸

```
python --version 명령을 실행해 줘
```

시킨 일	기대하는 결과	실제 결과
`.env` 읽기	막힌다 (deny)	
`git commit` 실행	막힌다 (deny)	
`scripts/verify.sh` 고치기	막힌다 (deny)	
`python --version` 실행	실행 전에 나에게 묻는다 (ask)	

`ask` 에 걸리면 OpenCode 가 허락을 구한다. 고를 수 있는 답은 세 가지다.

답	뜻
once (이번만)	이번 한 번만 허락한다
always (항상)	앞으로 같은 종류의 요청은 묻지 않는다
reject (거절)	허락하지 않는다

always 는 신중하게 누른다. 누르는 순간 그 명령은 `ask` 에서 `allow` 로 바뀐다. 이 과정에서는 `once` 를 쓴다.

`/exit` 로 나온 뒤 확인한다.

■ [dev] dev 컨테이너

```
git status
git log --oneline -1
```

`scripts/verify.sh` 가 바뀌지 않았고, 새 커밋이 생기지 않았어야 한다. 기대와 다른 결과가 있었다면 강사에게 알린다. 권한 규칙의 `resource` 무늬가 맞지 않는 경우일 수 있다.

4단계. 검증 스크립트 읽기 (10분)

`scripts/verify.sh` 를 VS Code 로 연다. 주석과 안내 문구를 빼면 핵심은 네 줄이다.

```
set -e
ruff check app tests scripts
pytest -q
echo "검증 통과: 이제 저장(commit)해도 됩니다."
```

- `set -e` — 명령 하나라도 실패하면 **그 자리에서 멈춘다.** 그래서 마지막 줄 "검증 통과"는 앞의 검사가 모두 성공했을 때만 출력된다.

- `ruff check` — 코드를 실행하지 않고 읽어서 실수를 찾는다. 쓰지 않는 import, 정의하지 않은 이름 같은 것들.
- `pytest -q` — 테스트를 실행한다.

ruff 가 무엇을 잡는지 본다. `app/main.py` 맨 아래에 `import os` 한 줄을 추가하고 저장한 뒤 실행한다.

■ [dev] dev 컨테이너

```
sh scripts/verify.sh
```

```
[1/2] 코드 검사 (ruff) ---------------------------
F401 [*] `os` imported but unused
```

"`os` 를 가져왔지만 쓰지 않았다." 그리고 `[2/2] 자동 테스트` 는 **아예 실행되지 않았다.** `set -e` 때문이다.

`import os` 줄을 지우고 저장한다. `sh scripts/verify.sh` 가 "검증 통과"로 끝나는지 확인한다.

5단계. 검증을 통과해야만 저장되게 하기 — pre-commit (20분)

지금까지는 `sh scripts/verify.sh` 를 **내가 기억해서** 실행했다. 잊으면 깨진 코드가 저장된다. 이것을 장치로 바꾼다.

Git 에는 "커밋 직전에 이 스크립트를 실행하고, 실패하면 커밋을 취소한다"는 기능이 있다. **훅(hook)** 이라고 한다. `.githooks/pre-commit` 을 VS Code 로 열어 읽는다. 핵심은 이 부분이다.

```
sh scripts/verify.sh
result=$?

if [ $result -ne 0 ]; then
    echo "[pre-commit] 검증 실패: 커밋을 취소했습니다."
    exit 1
fi
```

검증을 실행하고, 실패했으면(`0` 이 아니면) `exit 1` 로 끝낸다. 훅이 `0` 이 아닌 값으로 끝나면 Git 은 커밋을 만들지 않는다.

훅을 켠다. 이 저장소에서 한 번만 하면 된다.

■ [dev] dev 컨테이너

```
git config core.hooksPath .githooks
git config core.hooksPath
```

둘째 명령이 `.githooks` 를 출력하면 켜진 것이다.

시험 1 — 깨진 코드를 저장해 본다.

`app/main.py` 에서 `{"status": "ok"}` 를 `{"status": "okay"}` 로 바꾸고 저장한다. 그리고 커밋을 시도한다.

■ [dev] dev 컨테이너

```
git add app/main.py
git commit -m "실험: 깨진 코드 저장 시도"
```

```
[pre-commit] 저장하기 전에 검증을 실행합니다...
[1/2] 코드 검사 (ruff) ---------------------------
All checks passed!
[2/2] 자동 테스트 (pytest) -----------------------
(가운데 생략)
>       assert response.json() == {"status": "ok"}  # 답의 내용도 정확히 같아야 한다
E       AssertionError: assert {'status': 'okay'} == {'status': 'ok'}
(가운데 생략)
FAILED tests/test_health.py::test_health - AssertionError: assert {'status': ...

[pre-commit] 검증 실패: 커밋을 취소했습니다.
[pre-commit] 위의 오류를 고친 뒤 다시 커밋하세요.
```

커밋이 만들어졌는지 확인한다.

■ [dev] dev 컨테이너

```
git log --oneline -1
```

맨 위 커밋이 여전히 "4교시: ..." 다. **깨진 코드는 저장되지 않았다.**

되돌린다.

■ [dev] dev 컨테이너

```
git restore --staged app/main.py
git restore app/main.py
```

시험 2 — 정상 코드를 저장한다. 이번 교시에 만든 하네스 파일들을 커밋한다.

■ [dev] dev 컨테이너

```
git status
git add AGENTS.md opencode.jsonc
git commit -m "5교시: 하네스 세우기 (AGENTS.md, 권한 규칙)"
```

이번에는 검증이 통과하고, 마지막에 커밋이 만들어졌다는 줄(`[main …] 5교시: …`)이 나온다.

> 이제 이 저장소에서는 **사람이든 AI 든 검증을 통과하지 못한 코드를 저장할 수 없다.** (참고: `core.hooksPath` 설정은 내 PC 의 `.git/config` 에 저장된다. 저장소를 새로 내려받으면 다시 켜야 한다.)

6단계. 실험 — 하네스를 세우고 다시 시켜 보기 (15분)

1단계와 **똑같은 한 줄**을 다시 시킨다.

■ [dev] dev 컨테이너

```
opencode
```

■ [AI] OpenCode 입력 칸

```
메모 삭제 기능 만들어 줘
```

이번에는 AI 가 `AGENTS.md` 를 읽고 시작한다. 계획을 말하면 읽어 보고, 이번 실험에서는 구현까지 가지 않는다.

■ [AI] OpenCode 입력 칸

```
계획은 잘 봤어. 이번에는 구현하지 않을게. 여기서 멈춰.
```

`/exit` 로 나온다. 1단계의 관찰 기록 옆에 이번 결과를 적어 비교한다.

관찰할 것	1단계 (규칙 없음)	6단계 (하네스 있음)
고치기 전에 계획을 말하고 승인을 기다렸는가?		
요구사항 문서(`REQUIREMENTS.md`)를 찾아 읽었는가?		
상태 코드와 오류 문구를 어디에서 가져왔는가?		
테스트를 만들겠다고 했는가?		
명령을 실행하기 전에 물어봤는가?		

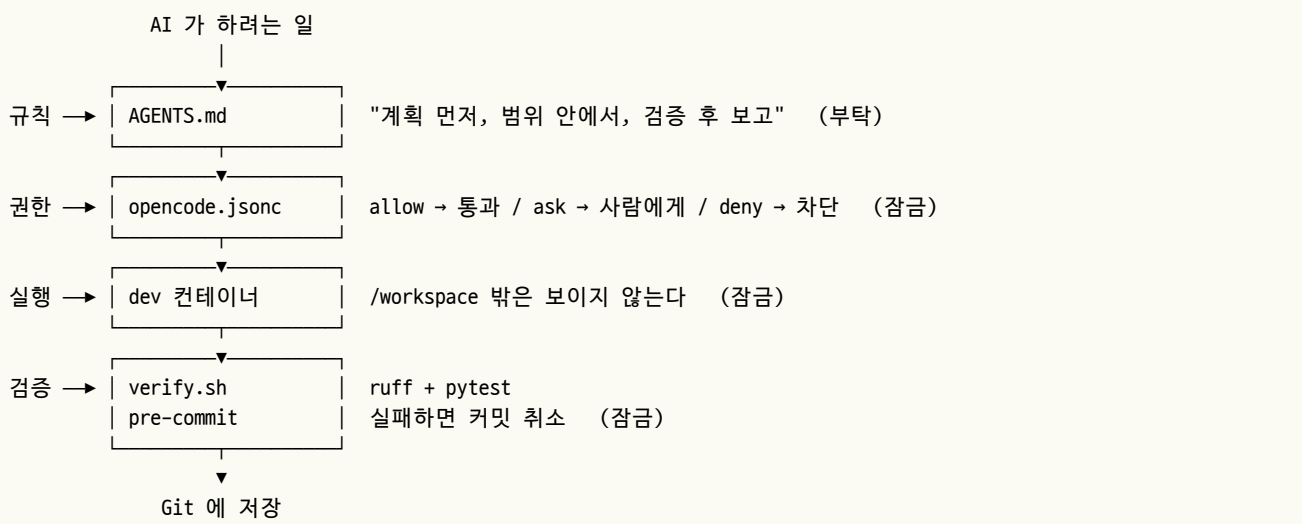
■ [dev] dev 컨테이너

```
git status
```

바뀐 파일이 있으면 `git restore .` 로 되돌린다.

그래도 부족한 점. "메모 삭제 기능 만들어 줘" 한 줄에는 범위도, 하지 말 것도, 완료 조건도 없다. 규칙은 **어떻게 일할지**를 정할 뿐, **이번에 무엇을 할지**는 지시가 정한다. 6교시에 작업 지시서를 쓴다.

하네스 관점 — 이번 교시에서 세운 것



이번 교시에 **코드는 한 줄도 늘지 않았다.** 그런데 프로젝트는 크게 달라졌다. 하네스 엔지니어링은 코드가 아니라 **코드가 만들어지는 환경**을 만드는 일이다.

자주 만나는 오류

증상	원인과 해결
OpenCode 가 `AGENTS.md` 를 읽지 않은 것처럼 행동한다	파일이 맨 위 폴더(`/workspace/AGENTS.md`)에 있는지 확인한다. OpenCode 는 시작할 때 읽으므로 `/exit` 로 나갔다가 다시 연다.
OpenCode 가 시작할 때 설정 오류를 낸다	`opencode.jsonc` 를 손으로 고치다 쉼표나 중괄호가 깨졌다. `cp harness/opencode.jsonc opencode.jsonc` 로 다시 가져온다.
3단계에서 막혀야 할 일이 막히지 않는다	설정을 바꾼 뒤 OpenCode 를 새로 열지 않았다. 다시 열어도 그대로면 강사에게 알린다.
`git commit` 을 했는데 `[pre-commit]` 문구가 나오지 않는다	훅이 켜지지 않았다. `git config core.hooksPath` 가 `.githooks` 를 출력하는지 확인한다.
윈도우 PowerShell 에서 커밋하면 훅이 오래 걸리거나 실패한다	윈도우에서 커밋하면 훅이 dev 컨테이너에게 검증을 시킨다. 컨테이너가 꺼져 있으면 실패한다. 이 과정에서는 **커밋을 dev 컨테이너 안에서** 한다.
`git restore .` 를 했는데 파일이 남아 있다	새로 만들어진 파일은 `restore` 로 지워지지 않는다. `git status` 의 `Untracked files:` 에 나온 파일을 직접 지운다.
`diff` 결과가 너무 길다	정상이다. 틀을 내 말로 채웠으니 줄마다 다르다. 절 제목을 기준으로 끊어서 비교한다.

확인 문제

1. `AGENTS.md` 와 `opencode.jsonc` 는 둘 다 "커밋 금지"를 말한다. 왜 두 군데에 적는가?
2. 권한 규칙에서 `{ "action": "shell", "resource": "*", "effect": "ask" }` 를 맨 **아래**로 옮기면 어떤 일이 생기는가?
3. AI 가 `scripts/verify.sh` 와 `tests/` 를 마음대로 고칠 수 있다면, "검증 통과"라는 말을 믿을 수 있는가? 이유를 쓰시오.
4. `verify.sh` 의 `set -e` 를 지우면 어떤 문제가 생기는가?
5. `pre-commit` 훅이 없을 때와 있을 때, "검증을 통과한 코드만 저장한다"는 약속은 각각 무엇에 의존하는가?
6. 권한 프롬프트에서 `always` 를 누르면 무엇이 바뀌는가? 언제 누르면 안 되는가?
7. 1단계와 6단계의 관찰 기록에서 가장 크게 달라진 항목 하나를 고르고, 그것이 `AGENTS.md` 의 어느 절 덕분인지 쓰시오.
8. 하네스를 세웠는데도 6단계의 지시 "메모 삭제 기능 만들어 줘"가 부족한 이유는 무엇인가?

6교시. AI 에게 CRUD 구현 지시

시간	2시간
실습 결과	메모 CRUD 초안 (만들기·목록·한 건 보기, 메모리 저장, 테스트 포함)
시작 상태	5교시를 마친 상태 (`AGENTS.md`, 권한 규칙, `pre-commit` 훅이 있다)

4교시 코드가 없으면 **[dev]** `sh scripts/checkpoint.sh 04` 로 가져온다. 하네스 파일이 없으면 `cp harness/AGENTS.md AGENTS.md`, `cp harness/opencode.jsonc opencode.jsonc`, `git config core.hooksPath .githooks` 를 실행한다.

수업의 흐름에서 이번 교시의 위치

```
① 요구사항 작성 → ② AI 에게 지시 → ③ 하네스로 제어 → ④ 실행·테스트 → ⑤ 수정·재검증·저장
└──────────── 이번 교시: 다섯 단계를 처음으로 한 바퀴 돈다 ────────────┘
```

학습 목표

1. 요구사항 명세에서 이번에 만들 범위를 골라낸다.
2. 작업 지시서의 여섯 칸(목표·요구사항·범위·하지 말 것·완료 조건·진행 방법)을 읽고 쓸 수 있다.
3. AI 의 계획을 검토하고 승인하거나 고치게 한다.
4. 검증 체크리스트로 AI 의 결과를 검토한다.
5. AI 가 쓴 코드를 읽고 내 말로 설명한 뒤 커밋한다.

왜 CRUD 를 한 번에 시키지 않는가

이번 교시의 범위는 **만들기(C)와 보기(R)** 까지다. 고치기·지우기는 9교시, DB 연결은 8교시에 한다.

- 작은 작업은 AI 가 틀릴 여지가 적다.
- 바뀐 코드가 적어야 **내가 읽고 이해할 수 있다.**
- 문제가 생기면 그 작업 하나만 되돌리면 된다.

기준: **AI 가 바꾼 내용을 내가 10분 안에 읽고 설명할 수 있는 크기.**

따라 하기

1단계. ① 요구사항 읽기 (10분)

VS Code 에서 `harness/REQUIREMENTS.md` 를 열고 **R1, R2, R3, R6** 을 읽는다. 4교시 4단계에서 채운 대조표를 다시 꺼낸다. 그때 "맞지 않다"고 적은 항목들이 이번에 AI 가 채워야 할 것들이다.

요구사항	4교시에 손으로 만든 코드	이번 교시의 목표
R1. 응답에 `created_at`, `updated_at`	없다	있다
R2. 나중에 만든 메모가 앞에	먼저 만든 것이 앞	나중 것이 앞
R3. 한 건 보기, 없으면 404	없다	있다
R6. 빈 제목, 100자 넘는 제목 → 422	받아들인다	422

2단계. ② 작업 지시서 읽기 (10분)

`harness/tasks/06_memo_draft.md` 를 연다. 5교시에 실패했던 한 줄 지시("메모 삭제 기능 만들어 줘")와 비교하며 읽는다.

칸	이 지시서에 적힌 것	없으면 생기는 일
1. 목표	만들기·목록·한 건 보기. 저장소는 메모리	어디까지가 끝인지 모른다
2. 요구사항	R1, R2, R3, R6	상태 코드·문구를 AI 가 지어낸다
3. 범위	고칠 파일 2개, 새 파일 2개	관계없는 파일까지 고친다
4. 하지 말 것	수정·삭제, DB, 화면, 새 패키지	시키지 않은 기능이 생긴다
5. 완료 조건	체크 항목 7개	"다 했습니다"를 확인할 수 없다
6. 진행 방법	계획 먼저, "진행" 뒤에 구현	방향이 틀려도 끝난 뒤에야 안다

지시서의 틀과 칸마다의 설명은 `harness/TASK_TEMPLATE.md` 에 있다.

3단계. ② 지시하고 ③ 계획 검토하기 (20분)

시작 전에 상태가 깨끗한지 확인한다.

■ [dev] dev 컨테이너

```
git status
sh scripts/verify.sh
```

`working tree clean` 이고 "검증 통과"여야 한다. **깨끗한 상태에서 시작해야, 끝난 뒤의 `git diff` 가 온전히 AI 가 한 일이 된다.**

OpenCode 를 열고 지시서를 건넨다.

■ [dev] dev 컨테이너

```
opencode
```

■ [AI] OpenCode 입력 칸

```
harness/tasks/06_memo_draft.md 를 읽고 그 지시대로 작업해 줘. AGENTS.md 의 작업 절차를 따라, 먼저 계획만 말하고 내 승인을 기다려.
```

AI 가 파일들을 읽고 **계획**을 말한다. 바로 "진행"이라고 하지 않는다. 아래 항목으로 계획을 검토한다.

계획 검토 체크리스트

- ■ 고치겠다는 파일이 지시서의 "범위" 안에 있는가?
- ■ 수정(PUT)·삭제(DELETE)·DB·화면을 만들겠다는 말이 **없는가?**
- ■ 테스트를 만들겠다고 했는가? R1, R2, R3, R6 이 모두 언급되었는가?
- ■ 없는 번호 → 404, 오류 문구가 요구사항과 같은가?
- ■ 새 패키지를 설치하겠다는 말이 **없는가?**
- ■ 이해되지 않는 낱말이 있는가? 있으면 지금 묻는다.

계획에 문제가 있으면 구체적으로 짚어 고치게 한다.

■ [AI] OpenCode 입력 칸 (예시 — 계획에 삭제가 들어 있을 때)

```
계획 3번에 삭제 API 가 들어 있어. 지시서 4번(하지 말 것)에 따라 삭제는 이번 범위가 아니야. 빼고 계획을 다시 말해 줘.
```

■ [AI] OpenCode 입력 칸 (예시 — 모르는 낱말이 있을 때)

```
계획에 나온 "fixture" 가 무슨 뜻이야? 초보자에게 설명하듯 두 문장으로 알려 줘. 파일은 아직 고치지 마.
```

계획이 지시서와 맞으면 승인한다.

■ [AI] OpenCode 입력 칸

> 진행

4단계. ③ 하네스가 일하는 것 지켜보기 (15분)

AI 가 파일을 고치고 명령을 실행한다. 지켜보며 아래를 기록한다. 이 기록은 7단계의 `WORKLOG.md` 에 적는다.

지켜볼 것	기록
묻지 않고 실행한 명령 (`pytest`, `sh scripts/verify.sh`, `git status` …)	
나에게 물어본 명령 — 무엇이었고, 허락했는가 거절했는가	
막힌(deny) 일이 있었는가	
검증이 처음에 실패했는가? AI 가 스스로 고쳤는가? 몇 번?	

물어볼 때의 판단 기준.

- 지시서와 `AGENTS.md` 에 비추어 필요한 일인가? → `once`
- 왜 필요한지 모르겠는가? → 일단 `reject` 하고 이유를 묻는다. "방금 그 명령은 왜 필요했어?"
- `pip install` 처럼 지시서가 금지한 일인가? → `reject`

VS Code 를 옆에 띄워 두면 AI 가 고치는 파일이 **실시간으로 바뀌는 것**이 보인다. 2교시의 바인드 마운트 덕분이다.

AI 가 보고를 마치면 OpenCode 를 닫지 말고 그대로 둔다. 다음 단계에서 질문할 것이다. dev 셸이 하나 더 필요하므로 PowerShell 창을 새로 열어 들어간다.

■ [PS] 윈도우 PowerShell

```
cd C:\harness-crud-lab
docker compose exec dev bash
```

5단계. ④ 검증 — AI 의 말을 믿지 않고 직접 실행하기 (15분)

AI 가 "검증 통과했습니다"라고 보고했더라도 **직접 실행한다.** `harness/VERIFY_CHECKLIST.md` 를 열어 놓고 1단계부터 따라간다.

(1) 기계가 확인하는 것

■ [dev] dev 컨테이너 (새로 연 셸)

```
sh scripts/verify.sh
```

마지막 줄이 "검증 통과"인지, 통과한 테스트가 몇 개인지 본다. 참고로 이 교시의 완성 예시(`checkpoints/lesson06`)는 다음과 같이 나온다. 테스트 개수는 AI 가 만든 것에 따라 다를 수 있다.

```
............                                                [100%]
12 passed in 0.08s
```

(2) 무엇이 바뀌었는지

■ [dev] dev 컨테이너

```
git status
git diff --stat
```

■ 바뀌거나 새로 생긴 파일이 `app/main.py`, `app/schemas.py`, `tests/conftest.py`, `tests/test_memos.py` 뿐인가?

■ `tests/test_health.py` 는 바뀌지 않았는가?

■ `AGENTS.md`, `harness/`, `scripts/` 는 바뀌지 않았는가?

(3) 직접 불러 보기

■ [dev] dev 컨테이너

```
curl -s -X POST http://app:8000/memos -H "Content-Type: application/json" -d '{"title": "장보기", "content": "우유, 달걀"}'
```

```
{"id":1,"title":"장보기","content":"우유, 달걀","created_at":"2026-11-10T05:47:04.556149Z","updated_at":"2026-11-10T05:47:04.556149Z"}
```

`created_at`, `updated_at` 이 생겼다. (시각은 실행한 때에 따라 다르다.)

■ [dev] dev 컨테이너

```
curl -s http://app:8000/memos/1
curl -s http://app:8000/memos/999
```

```
{"id":1,"title":"장보기","content":"우유, 달걀","created_at":"2026-11-10T05:47:04.556149Z","updated_at":"2026-11-10T05:47:04.556149Z"}
{"detail":"메모를 찾을 수 없습니다."}
```

■ [dev] dev 컨테이너

```
curl -s -o /dev/null -w "%{http_code}\n" http://app:8000/memos/999
curl -s -X POST http://app:8000/memos -H "Content-Type: application/json" -d '{"title": ""}'
```

```
404
{"detail":[{"type":"string_too_short","loc":["body","title"],"msg":"String should have at least 1 character","input":"","ctx":{"min_leng
  th":1}}]}
```

오류 문구 `메모를 찾을 수 없습니다.` 가 요구사항 R3 과 **한 글자도 다르지 않은지** 확인한다. 다르다면 AI 에게 요구사항의 해당 줄을 가리켜 고치게 한다.

http://localhost:8000/docs 에서도 `GET /memos/{memo_id}` 가 생겼는지 눌러 본다.

6단계. ⑤ 코드 읽기 — 내 것으로 만들기 (30분)

테스트가 통과했다. 여기서 커밋하면 바이브 코딩이다. **읽고 이해한 뒤에 커밋한다.**

AI 가 쓴 코드는 사람마다 다르다. 아래는 완성 예시(`checkpoints/lesson06`)를 기준으로 한 읽기 안내다. 내 코드에서 같은 역할을 하는 부분을 찾아가며 읽는다.

(1) `app/schemas.py` — 데이터의 모양

```python
class MemoIn(BaseModel):
    """메모를 만들거나 고칠 때 클라이언트가 보내는 데이터"""

    # 제목: 반드시 있어야 하고 1~100자
    title: str = Field(min_length=1, max_length=100)
    # 내용: 안 보내면 빈 문자열, 최대 1000자
    content: str = Field(default="", max_length=1000)


class MemoOut(BaseModel):
    """서버가 돌려주는 메모 한 건"""

    id: int
    title: str
    content: str
    created_at: datetime
    updated_at: datetime
```

- 4교시의 `title: str` 이 `Field(min_length=1, max_length=100)` 로 바뀌었다. **요구사항 R6 의 문장이 코드 한 줄이 된 것**이다.
- 들어오는 모양(`MemoIn`)과 나가는 모양(`MemoOut`)이 따로 있다. `id` 와 시각은 서버가 정하므로 들어오는 쪽에는 없다.

(2) `app/main.py` — 저장소

```
# 메모 저장소: {번호: 메모} 모양의 딕셔너리. 번호로 바로 찾을 수 있다.
memos: dict[int, dict] = {}
# 다음에 만들 메모의 번호. 메모를 지워도 번호가 다시 쓰이지 않게 따로 센다.
next_id = 1
```

4교시에는 리스트였고 번호를 `len(memos) + 1` 로 정했다. 4교시 확인 문제 7번의 답이 여기 있다.

(3) `app/main.py` — 한 건 보기와 404

```
@app.get("/memos/{memo_id}", response_model=MemoOut)
def get_memo(memo_id: int) -> dict:
    if memo_id not in memos:  # 그런 번호가 없으면
        raise HTTPException(status_code=404, detail="메모를 찾을 수 없습니다.")
    return memos[memo_id]
```

- 주소의 `{memo_id}` 자리에 온 값이 함수의 인자 `memo_id` 로 들어간다. `/memos/7` 이면 `memo_id` 는 `7` 이다.
- `memo_id: int` — 정수가 아니면(`/memos/abc`) FastAPI 가 422 를 돌려준다.
- `raise HTTPException(...)` — 함수를 그 자리에서 끝내고 오류 응답을 보낸다.
- `response_model=MemoOut` — 응답이 `MemoOut` 모양으로 나간다.

(4) `tests/conftest.py` — 테스트마다 깨끗하게

```
@pytest.fixture
def client():
    reset_store()  # 앞 테스트가 남긴 메모를 지운다
    with TestClient(app) as test_client:
        yield test_client
```

`@pytest.fixture` 가 붙은 함수는 "테스트가 쓸 준비물을 만드는 함수"다. 테스트 함수가 인자로 `client` 를 받으면 pytest 가 이 함수를 먼저 실행한다. 테스트마다 저장소를 비우므로 테스트끼리 서로 영향을 주지 않는다.

(5) `tests/test_memos.py` — 요구사항이 테스트가 되었는가

```
def test_get_memo_not_found(client):
    response = client.get("/memos/999")  # 없는 번호
    assert response.status_code == 404
    assert response.json() == {"detail": "메모를 찾을 수 없습니다."}
```

요구사항 R3 의 둘째 줄과 나란히 놓고 본다.

> 그 번호의 메모가 없으면 상태 코드 **404** 와 `{"detail": "메모를 찾을 수 없습니다."}` 를 돌려준다.

문장이 그대로 코드가 되었다. `harness/VERIFY_CHECKLIST.md` 3단계의 대응표를 펴고, R1·R2·R3·R6 마다 테스트가 있는지 내 `tests/test_memos.py` 에서 찾아 체크한다. 빠진 것이 있으면 AI 에게 추가하게 한다.

(6) AI 에게 묻기

읽다가 막힌 줄은 열어 둔 OpenCode 에 묻는다. 좋은 질문은 **파일과 줄을 짚는다.**

■ [AI] OpenCode 입력 칸

```
파일은 고치지 말고 설명만 해 줘. app/main.py 에서 global next_id 는 왜 필요해? 그 줄을 지우면 어떤 오류가 나?
```

■ [AI] OpenCode 입력 칸

```
파일은 고치지 말고 설명만 해 줘. POST /memos 요청이 들어와서 201 응답이 나갈 때까지, 어떤 파일의 어떤 함수가 어떤 순서로 실행되는지 번호
  를 붙여 알려 줘.
```

■ [AI] OpenCode 입력 칸

```
tests/test_memos.py 에 R2("나중에 만든 메모가 앞에")를 확인하는 테스트가 있어? 있으면 이름을, 없으면 없다고 말해 줘.
```

AI 의 설명은 3교시에 배운 대로 **실행해서 확인한다.** 예를 들어 `global next_id` 줄을 지우고 저장한 뒤 `sh scripts/verify.sh` 를 실행해 본다. AI 가 말한 오류가 나는가? 확인했으면 되돌린다(`git restore app/main.py` 는 쓰지 않는다. 아직 커밋 전이라 AI 의 작업까지 사라진다. VS Code 에서 Ctrl+Z 로 되돌린다).

7단계. ⑤ 기록하고 저장하기 (10분)

(1) 작업 기록. VS Code 에서 `harness/WORKLOG.md` 를 열고, 맨 아래에 틀을 복사해 6교시 기록을 쓴다. "4. 바뀐 코드를 내 말로 설명하기"의 세 문장을 쓰지 못하면 6단계로 돌아간다.

(2) 마지막 검토. `VERIFY_CHECKLIST.md` 의 4단계(코드를 사람이 읽는다) 항목을 체크한다.

■ [dev] dev 컨테이너

```
git diff app/
```

(3) 커밋.

■ [dev] dev 컨테이너

```
git add -A
git status
git commit -m "6교시: 메모 만들기·보기 초안 (R1, R2, R3, R6)"
```

`git status` 에서 커밋될 파일 목록에 `.env` 가 없는지 본다. 커밋하면 `pre-commit` 이 검증을 한 번 더 실행한다.

■ [dev] dev 컨테이너

```
git log --oneline -3
```

OpenCode 는 `/exit` 로 닫는다.

검증이 실패했을 때 — ⑤ 의 다른 길

AI 가 끝냈다고 했는데 내가 실행한 `sh scripts/verify.sh` 가 실패하면, **오류 전체를 붙여서** 전달한다.

■ [AI] OpenCode 입력 칸

```
sh scripts/verify.sh 가 실패했어. 아래가 출력이야.
원인을 먼저 설명하고, 내가 "진행"이라고 하면 고쳐 줘. 테스트를 느슨하게 고쳐서 통과시키지는 마.

(여기에 오류 출력을 붙여 넣는다)
```

고친 뒤에는 5단계부터 다시 한다. 세 번 고쳐도 통과하지 못하면 멈춘다.

■ [dev] dev 컨테이너

```
git restore .
git status
```

새로 생긴 파일(`Untracked files`)도 지워 처음 상태로 돌린 뒤, `/new` 로 새 대화를 열어 다시 시킨다. 그래도 막히면 `sh scripts/checkpoint.sh 06 --force` 로 완성 예시를 가져와 6단계(코드 읽기)를 한다.

하네스 관점 — 이번 교시에서 본 것

단계	통제한 장치	사람이 한 판단
① 요구사항	`REQUIREMENTS.md`	이번에 만들 번호를 골랐다
② 지시	`tasks/06_memo_draft.md`	범위와 "하지 말 것"을 정했다
③ 제어	`AGENTS.md`(계획 먼저), `opencode.jsonc`(ask/deny)	계획을 검토하고 승인했다. 명령을 허락하거나 거절했다
④ 실행·테스트	`verify.sh`	AI 의 보고를 믿지 않고 직접 실행했다
⑤ 수정·저장	`VERIFY_CHECKLIST.md`, `pre-commit`, `WORKLOG.md`	테스트와 코드를 읽고, 설명할 수 있을 때 커밋했다

코드는 AI 가 썼다. **무엇을 만들지, 계획이 맞는지, 결과를 받아들일지는 모두 사람이 정했다.**

자주 만나는 오류

증상	원인과 해결
AI 가 계획 없이 바로 고치기 시작한다	지시 문장에 "먼저 계획만 말하고 내 승인을 기다려"를 넣었는지, `AGENTS.md` 가 맨 위 폴더에 있는지 확인한다. 이미 고쳤다면 `/undo` 로 되돌리고 다시 시킨다.
AI 가 `harness/tasks/…` 파일을 못 찾는다	파일 이름을 잘못 쳤다. 입력 칸에서 `@` 를 치면 파일을 골라 넣을 수 있다.
권한 프롬프트가 너무 자주 뜬다	정상이다. 무엇을 묻는지 읽고 `once` 로 답한다. 같은 명령이 반복되면 그 명령이 `AGENTS.md` 2절 표에 있어야 하는 것인지 생각해 본다.
`verify.sh` 는 통과하는데 `curl` 결과가 요구사항과 다르다	테스트가 그 요구사항을 확인하지 않고 있다. 어느 요구사항인지 짚어 테스트를 추가하게 한다.
테스트를 따로 돌리면 통과하는데 함께 돌리면 실패한다	테스트끼리 저장소를 공유하고 있다. `conftest.py` 가 테스트마다 저장소를 비우는지 확인하게 한다.
`curl` 로 만든 메모가 자꾸 사라진다	AI 가 파일을 고칠 때마다 서버가 다시 켜지고, 메모리 저장소가 비워진다. 4교시 5단계에서 본 현상이다.
AI 의 답이 중간에 끊기거나 `429`, `402` 오류가 난다	OpenRouter 의 사용 한도나 크레딧 문제다. 잠시 뒤 다시 시도하거나 Credits 화면을 확인한다.

확인 문제

1. 작업 지시서의 "4. 하지 말 것"이 없으면 어떤 일이 생기는가? 5교시 1단계의 관찰을 예로 들어 설명하시오.
2. AI 의 계획을 승인하기 전에 확인해야 할 것을 세 가지 쓰시오.
3. `Field(min_length=1, max_length=100)` 은 요구사항의 어느 문장을 코드로 옮긴 것인가?
4. `MemoIn` 에는 `id` 가 없고 `MemoOut` 에는 있다. 왜 그런가?
5. `raise HTTPException(status_code=404, ...)` 다음 줄의 `return memos[memo_id]` 는 실행되는가?
6. `conftest.py` 에서 `reset_store()` 를 지우면 어떤 테스트가 왜 실패하게 되는가?
7. AI 가 "12개 테스트 모두 통과"라고 보고했다. 커밋하기 전에 사람이 더 확인해야 할 것을 세 가지 쓰시오.
8. 이번 교시에 사람이 내린 **판단**을 모두 나열하시오.

7교시. PostgreSQL 기초

시간	2시간
실습 결과	DB·테이블 생성 (SQL 로 메모를 만들고, 보고, 고치고, 지운다)
시작 상태	6교시를 마친 상태. 이 교시는 코드를 고치지 않으므로 6교시가 덜 끝났어도 진행할 수 있다

수업의 흐름에서 이번 교시의 위치

```
① 요구사항 작성 → ② AI 에게 지시 → ③ 하네스로 제어 → ④ 실행·테스트 → ⑤ 수정·재검증·저장
└ 이번 교시: 8교시에 AI 가 쓸 SQL 을, 내가 먼저 읽고 쓸 수 있게 한다
```

8교시에 AI 가 `INSERT INTO memos (title, content) VALUES (%s, %s) RETURNING *` 같은 코드를 쓴다. 이 줄을 읽지 못하면 검토할 수 없다. 이번 교시는 AI 없이 **손으로** SQL 을 친다. 3교시에 `/hello` 를 손으로 만든 것과 같은 이유다.

학습 목표

1. 데이터베이스·테이블·행·열이 무엇인지 설명한다.
2. `psql` 로 PostgreSQL 에 접속해 데이터베이스와 테이블을 만든다.
3. `INSERT`·`SELECT`·`UPDATE`·`DELETE` 로 CRUD 를 한다.
4. `NOT NULL`, `PRIMARY KEY`, `DEFAULT` 가 무엇을 지켜 주는지 직접 확인한다.
5. 컨테이너를 지웠다 다시 만들어도 데이터가 남는 것을 확인하고, 그 이유를 설명한다.

1. 개념

왜 데이터베이스인가

4교시에 서버를 다시 켜니 메모가 사라졌다. 파이썬의 리스트·딕셔너리는 프로그램의 메모리에 있고, 프로그램이 꺼지면 없어진다. **데이터베이스(DB)** 는 데이터를 디스크에 저장하고, 여러 프로그램이 동시에 안전하게 읽고 쓰게 해 주는 프로그램이다. **PostgreSQL** 은 널리 쓰이는 무료 데이터베이스다.

낱말

낱말	뜻	엑셀에 빗대면
데이터베이스	테이블들을 담는 큰 묶음	파일(통합 문서)
테이블	같은 모양의 데이터를 모아 둔 표	시트
열(column)	항목. 이름과 타입이 정해져 있다	열 제목
행(row)	데이터 한 건	한 줄
SQL	데이터베이스에게 일을 시키는 말	—

엑셀과 다른 점: 열마다 **타입과 규칙**이 정해져 있어서, 맞지 않는 데이터는 들어가지 못한다.

memos 테이블

열	타입	규칙	뜻
id	SERIAL	PRIMARY KEY	자동으로 1, 2, 3 … 이 붙는 번호. 겹치지 않는다
title	VARCHAR(100)	NOT NULL	최대 100자의 글자. 비어 있을 수 없다
content	TEXT	NOT NULL DEFAULT ''	길이 제한 없는 글자. 안 넣으면 빈 문자열
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	날짜와 시각. 안 넣으면 지금 시각
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	날짜와 시각. 안 넣으면 지금 시각

harness/REQUIREMENTS.md 2절의 "메모 한 건의 모양" 표와 나란히 놓고 본다. 같은 내용이 요구사항 → 파이썬 클래스(MemoOut) → 테이블, 세 군데에 같은 모양으로 있다.

CRUD ↔ SQL

4교시의 표에 SQL 칸을 채운다.

하는 일	HTTP	SQL
만들기	POST /memos	INSERT INTO memos (title, content) VALUES ('…', '…');
목록 보기	GET /memos	SELECT * FROM memos ORDER BY id DESC;
한 건 보기	GET /memos/2	SELECT * FROM memos WHERE id = 2;
고치기	PUT /memos/2	UPDATE memos SET title = '…' WHERE id = 2;
지우기	DELETE /memos/2	DELETE FROM memos WHERE id = 2;

우리 DB 는 어디에 있는가

```
[db 컨테이너]  PostgreSQL 17
    |   계정 memo / 비밀번호 memo1234   (.env 의 POSTGRES_USER, POSTGRES_PASSWORD)
    |
    ├─ 데이터베이스 memo_db     ← 앱이 쓰는 진짜 메모 (8교시부터)
    ├─ 데이터베이스 memo_test   ← 테스트 전용 (8교시에 생긴다)
    └─ 데이터베이스 practice    ← 이번 교시의 연습장 (지금 만든다)

데이터가 실제로 놓이는 곳:  C:\harness-crud-lab\data\postgres\   (바인드 마운트)
```

따라 하기

1단계. 접속하기 (10분)

psql 은 PostgreSQL 에 접속해 SQL 을 치는 프로그램이다. db 컨테이너 안에 들어 있다.

■ [PS] 윈도우 PowerShell

```
cd C:\harness-crud-lab
docker compose exec db psql -U memo -d memo_db
```

- -U memo — 계정 이름
- -d memo_db — 접속할 데이터베이스

프롬프트가 memo_db=# 로 바뀐다. 이제 치는 것은 SQL 이다. 이 교시에서는 이 상태를 **[psql]** 로 표시한다.

psql 에는 SQL 말고 \ 로 시작하는 자체 명령이 있다.

명령	하는 일
`\l`	데이터베이스 목록
`\c 이름`	다른 데이터베이스로 옮겨 가기
`\dt`	지금 데이터베이스의 테이블 목록
`\d 테이블`	테이블의 열과 규칙 보기
`\q`	나가기

■ [psql] psql

```
\l
```

목록에 `memo_db` 가 있다. 2교시에 컨테이너를 처음 켤 때 `.env` 의 `POSTGRES_DB` 값으로 자동으로 만들어진 것이다. (`postgres`, `template0`, `template1` 은 PostgreSQL 이 스스로 쓰는 것이니 건드리지 않는다.) 목록이 길어 `(END)` 가 보이면 `q` 를 누른다.

2단계. 데이터베이스와 테이블 만들기 (15분)

연습장으로 쓸 데이터베이스를 만든다. **SQL 문장은 세미콜론(;)으로 끝낸다.** 세미콜론을 치기 전에는 Enter 를 눌러도 실행되지 않고 다음 줄을 기다린다.

■ [psql] psql

```
CREATE DATABASE practice;
```

```
CREATE DATABASE
```

연습장으로 옮겨 간다.

■ [psql] psql

```
\c practice
```

프롬프트가 `practice=#` 로 바뀐다.

테이블을 만든다. 여러 줄이지만 문장 하나다. 한 줄씩 치고 Enter 를 눌러도 되고, 통째로 붙여 넣어도 된다.

■ [psql] psql

```
CREATE TABLE memos (
    id          SERIAL PRIMARY KEY,
    title       VARCHAR(100) NOT NULL,
    content     TEXT NOT NULL DEFAULT '',
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```
CREATE TABLE
```

만들어졌는지 본다.

■ [psql] psql

```
\dt
```

```
       List of relations
 Schema ¦ Name  ¦ Type  ¦ Owner
--------+-------+-------+-------
 public ¦ memos ¦ table ¦ memo
(1 row)
```

■ [psql] psql

```
\d memos
```

열 이름, 타입, `not null`, 기본값이 표로 나온다. 1절의 표와 대조한다.

3단계. Create — INSERT (10분)

■ [psql] psql

```
INSERT INTO memos (title, content) VALUES ('장보기', '우유, 달걀');
```

```
INSERT 0 1
```

마지막 숫자 `1` 이 "1행이 들어갔다"는 뜻이다. SQL 에서 글자는 **작은따옴표**로 감싼다. (JSON 은 큰따옴표였다.)

내용 없이 하나 더 넣는다.

■ [psql] psql

```
INSERT INTO memos (title) VALUES ('운동');
```

`id`, `content`, `created_at`, `updated_at` 은 넣지 않았다. 어떻게 채워졌는지 다음 단계에서 본다.

4단계. Read — SELECT (15분)

■ [psql] psql

```
SELECT id, title, content FROM memos;
```

```
 id ¦ title  ¦  content
----+--------+------------
  1 ¦ 장보기 ¦ 우유, 달걀
  2 ¦ 운동   ¦
(2 rows)
```

- `id` 가 1, 2 로 **자동으로** 붙었다. (`SERIAL`)
- 2번의 `content` 는 빈 문자열이다. (`DEFAULT ''`)

모든 열을 보려면 `*` 를 쓴다.

■ [psql] psql

```
SELECT * FROM memos;
```

`created_at`, `updated_at` 에 넣은 적 없는 시각이 들어 있다. (`DEFAULT now()`)

조건으로 고르기 — `WHERE`

■ [psql] psql

```
SELECT id, title FROM memos WHERE id = 2;
```

```
 id ¦ title
----+-------
  2 ¦ 운동
(1 row)
```

`GET /memos/2` 가 하는 일이 이것이다.

순서 정하기 — `ORDER BY`

■ [psql] psql

```
SELECT id, title FROM memos ORDER BY id DESC;
```

```
 id ¦ title
----+--------
  2 ¦ 운동
  1 ¦ 장보기
(2 rows)
```

DESC 는 큰 것부터(내림차순)다. 요구사항 R2 "나중에 만든 메모가 앞에"가 이 한 줄이다.

없는 번호를 찾으면 어떻게 되는지도 본다.

■ [psql] psql

```
SELECT id, title FROM memos WHERE id = 999;
```

오류가 아니라 (0 rows) 가 나온다. **DB 는 "없다"를 오류로 알려 주지 않는다.** 그래서 서버 코드가 "결과가 0건이면 404"를 직접 만들어야 한다. 8교시에 AI 가 쓴 코드에서 이 부분을 찾아볼 것이다.

5단계. Update — UPDATE (10분)

■ [psql] psql

```
UPDATE memos SET content = '저녁 7시 달리기', updated_at = now() WHERE id = 2;
```

```
UPDATE 1
```

■ [psql] psql

```
SELECT id, title, content FROM memos ORDER BY id;
```

```
 id ¦ title  ¦     content
----+--------+-----------------
  1 ¦ 장보기 ¦ 우유, 달걀
  2 ¦ 운동   ¦ 저녁 7시 달리기
(2 rows)
```

WHERE 를 빠뜨리면 모든 행이 바뀐다. UPDATE memos SET content = '…'; 는 메모 전체의 내용을 같은 글로 덮어쓴다. 되돌릴 수 없다. AGENTS.md 5절에 "UPDATE 와 DELETE 에는 반드시 WHERE 를 쓴다"가 있는 이유다. 9교시에 AI 가 쓴 코드에서 이 규칙이 지켜졌는지 내가 확인한다.

6단계. 규칙이 지켜 주는 것 (10분)

제목 없이 넣어 본다.

■ [psql] psql

```
INSERT INTO memos (content) VALUES ('제목이 없는 메모');
```

```
ERROR:  null value in column "title" of relation "memos" violates not-null constraint
```

"title 열에 빈 값(null)을 넣으려 해서 not-null 규칙을 어겼다." 들어가지 않았다.

4교시에는 FastAPI 가 제목 없는 요청을 422 로 막았다. 이제 DB 도 막는다. **같은 규칙을 두 겹으로 지킨다.** 서버 코드에 버그가 있어도 DB 가 잘못된 데이터를 받지 않는다.

7단계. Delete — DELETE (10분)

■ [psql] psql

```
DELETE FROM memos WHERE id = 1;
```

```
DELETE 1
```

■ [psql] psql

```
SELECT id, title, content FROM memos;
```

```
 id ¦ title ¦     content
----+-------+-----------------
  2 ¦ 운동  ¦ 저녁 7시 달리기
(1 row)
```

없는 번호를 지워 본다.

■ [psql] psql

```
DELETE FROM memos WHERE id = 999;
```

```
DELETE 0
```

오류가 아니다. `0` 이 "0행을 지웠다"는 뜻이다. 요구사항 R5 는 이때 404 를 돌려주라고 한다. 서버 코드가 이 숫자(또는 그에 해당하는 결과)를 보고 404 를 만들어야 한다.

새 메모를 넣으면 번호는 어떻게 될까?

■ [psql] psql

```
INSERT INTO memos (title) VALUES ('새 메모') RETURNING id, title;
```

```
 id ¦  title
----+---------
  4 ¦ 새 메모
(1 row)

INSERT 0 1
```

두 가지를 본다.

- **RETURNING** — 넣은 행을 바로 돌려받는다. `INSERT` 뒤에 `SELECT` 를 또 하지 않아도 된다. 8교시의 코드에 나온다.
- **번호가 4 다.** 1번을 지웠지만 다시 쓰지 않는다. 3번은 6단계에서 실패한 `INSERT` 가 가져갔다가 버렸다. `SERIAL` 은 겹치지 않는 것만 보장하고, 빈틈이 없는 것은 보장하지 않는다.

■ [psql] psql

```
SELECT count(*) FROM memos;
```

```
 count
-------
     2
(1 row)
```

8단계. 컨테이너를 지워도 데이터가 남는가 (20분)

`psql` 에서 나온다.

■ [psql] psql

```
\q
```

(1) 데이터가 놓인 곳을 본다.

■ [PS] 윈도우 PowerShell

```
dir C:\harness-crud-lab\data\postgres
```

`base`, `global`, `pg_wal`, `PG_VERSION` … PostgreSQL 이 쓰는 파일들이다. 방금 넣은 메모가 이 안 어딘가에 들어 있다.

(2) 컨테이너를 모두 지운다. `stop` 이 아니라 `down` 이다. 컨테이너가 삭제된다.

■ [PS] 윈도우 PowerShell

```
docker compose down
docker compose ps
```

`docker compose ps` 에 아무것도 나오지 않는다. db 컨테이너는 이제 없다.

(3) C 드라이브의 폴더는?

■ [PS] 윈도우 PowerShell

```
dir C:\harness-crud-lab\data\postgres
```

그대로 있다.

(4) 컨테이너를 새로 만든다.

■ [PS] 윈도우 PowerShell

```
docker compose up -d
docker compose ps
```

`db` 가 `healthy` 가 될 때까지 기다린다.

(5) 메모가 있는지 본다. `-c` 뒤에 SQL 을 주면 접속하지 않고 한 번에 실행한다.

■ [PS] 윈도우 PowerShell

```
docker compose exec db psql -U memo -d practice -c "SELECT id, title FROM memos ORDER BY id;"
```

운동 과 새 메모 두 행이 나온다. **컨테이너는 새것인데 데이터는 그대로다.**

```
지워진 것:  db 컨테이너 (PostgreSQL 프로그램이 돌던 상자)
남은 것:    C:\harness-crud-lab\data\postgres\ (데이터)
새 컨테이너가 같은 폴더를 다시 연결해서 이어 쓴다  ← 바인드 마운트
```

> 반대로, `C:\harness-crud-lab\data\postgres` 폴더를 지우면 데이터가 모두 사라진다. DB 를 완전히 처음부터 다시 시작하고 싶을 때만, 컨테이너를 내린 뒤(`docker compose down`) 이 폴더를 지운다.

dev 컨테이너도 새로 만들어졌다. 계속하려면 다시 들어간다.

■ [PS] 윈도우 PowerShell

```
docker compose exec dev bash
```

9단계. AI 에게 물어 확인하기 (10분)

이번 교시에 배운 것을 AI 의 설명과 맞춰 본다. 고치는 일이 아니라 묻는 일이다.

■ [dev] dev 컨테이너

```
python scripts/ask_ai.py "PostgreSQL 에서 UPDATE 문에 WHERE 를 빠뜨리면 어떻게 되는지, 그리고 그 실수를 막는 습관 두 가지를 초보자에게 알려 줘."
```

■ [dev] dev 컨테이너

```
python scripts/ask_ai.py "SQL 인젝션이 뭐야? 파이썬에서 SQL 문자열에 값을 직접 이어 붙이면 왜 위험한지, 짧은 예 하나로 설명해 줘."
```

둘째 질문의 답을 잘 읽어 둔다. `AGENTS.md` 의 "SQL 에 값을 넣을 때는 반드시 `%s` 자리표시자를 쓴다"가 왜 있는지에 대한 답이다. 8교시에 AI 가 쓴 코드에서 이 규칙을 내가 확인한다.

이 교시에는 코드를 고치지 않았으므로 커밋할 것이 없다. `harness/WORKLOG.md` 에 "5. 새로 알게 된 것"만 적어 둔다.

하네스 관점 — 이번 교시에서 본 것

- **검토하려면 읽을 수 있어야 한다.** 이번 교시는 AI 를 쓰지 않았지만, AI 의 다음 작업을 검토할 준비를 했다. SQL 다섯 문장을 읽을 수 있으면 8·9교시의 `git diff` 를 읽을 수 있다.
- **규칙에는 이유가 있다.** `AGENTS.md` 의 DB 규칙 두 줄(`%s` 자리표시자, `WHERE` 필수)은 5교시에 옮겨 적기만 했다. 이번 교시에 왜 필요한지 직접 봤다.
- **데이터는 실행 환경 밖에 둔다.** 컨테이너(실행)는 언제든 지우고 다시 만들 수 있고, 데이터는 C 드라이브에 남는다. AI 가 컨테이너 안에서 무엇을 하든 컨테이너를 새로 만들면 깨끗한 실행 환경으로 돌아온다.

자주 만나는 오류

증상	원인과 해결
Enter 를 쳐도 아무 일이 없고 프롬프트가 `practice-#` 로 바뀐다	세미콜론(`;`)을 빠뜨렸다. `;` 를 치고 Enter 를 누른다.
프롬프트가 `practice'#` 로 바뀌고 빠져나오지 못한다	작은따옴표를 닫지 않았다. `'` 를 치고 `;` 를 친다. 그래도 안 되면 Ctrl+C.
`ERROR: relation "memos" does not exist`	`practice` 가 아닌 다른 데이터베이스에 있다. 프롬프트를 보고 `\c practice` 로 옮긴다.
`ERROR: database "practice" already exists`	이미 만들었다. `\c practice` 로 들어간다.
`ERROR: syntax error at or near …`	오타다. 가리키는 낱말의 **바로 앞**을 본다. 쉼표나 괄호가 빠진 경우가 많다.
`ERROR: column "장보기" does not exist`	글자를 큰따옴표로 감쌌다. SQL 의 글자는 작은따옴표다.
한글이 깨져 보인다	오래된 명령 창(cmd)에서 그럴 수 있다. Windows Terminal 이나 PowerShell 을 쓴다.
결과가 길어 `(END)` 나 `:` 에서 멈춘다	`q` 를 누른다.
8단계에서 메모가 사라졌다	`data\postgres` 폴더를 지웠거나, 예비 설정(`docker-compose.volume.yml`) 없이 켰다가 있이 켰다. `docs/troubleshooting.md` 를 본다.

확인 문제

1. 데이터베이스, 테이블, 행, 열을 엑셀이 아닌 다른 것(예: 도서관)에 빗대어 설명하시오.
2. `INSERT INTO memos (title) VALUES ('운동');` 을 실행하면 `id`, `content`, `created_at` 에는 각각 무엇이 들어가는가? 각각 어떤 규칙 때문인가?
3. `SELECT * FROM memos WHERE id = 999;` 와 `DELETE FROM memos WHERE id = 999;` 는 오류를 내는가? 이 사실이 서버 코드에 어떤 일을 요구하는가?

4. `UPDATE memos SET title = '회의';` 를 실행하면 어떤 일이 생기는가?
5. 요구사항 R2("나중에 만든 메모가 앞에")를 SQL 한 문장으로 쓰시오.
6. `RETURNING` 은 무엇을 해 주는가? 없다면 같은 일을 하려고 무엇을 더 해야 하는가?
7. `docker compose down` 뒤에도 데이터가 남은 이유를 `docker-compose.yml` 의 해당 줄을 인용해 설명하시오.
8. 제목 없는 메모를 FastAPI(422)와 DB(NOT NULL)가 둘 다 막는다. 한쪽만 있으면 안 되는 이유를 쓰시오.

8교시. FastAPI 와 DB 연결

시간	2시간
실습 결과	메모 저장·조회 (서버를 다시 켜도 메모가 남는다)
시작 상태	6교시의 메모 초안이 커밋되어 있고, 7교시의 SQL 을 읽을 수 있다

6교시 결과가 없으면 **[dev]** `sh scripts/checkpoint.sh 06` 으로 가져온다.

수업의 흐름에서 이번 교시의 위치

```
① 요구사항 작성 → ② AI 에게 지시 → ③ 하네스로 제어 → ④ 실행·테스트 → ⑤ 수정·재검증·저장
└──────────── 이번 교시: 두 번째 바퀴. 이번에는 "속만 바꾸는" 작업이다 ────────────┘
```

학습 목표

1. "겉은 그대로, 속만 바꾼다"(저장소 교체)가 무슨 뜻인지 설명한다.
2. **테스트를 한 글자도 바꾸지 않는다**는 조건으로 AI 에게 일을 시키고, 지켜졌는지 확인한다.
3. 파이썬에서 PostgreSQL 에 접속해 SQL 을 실행하는 코드를 읽는다.
4. 자리표시자(`%s`)가 왜 필요한지 설명한다.
5. 테스트 전용 DB 가 왜 필요한지 설명하고, 진짜 데이터가 지워지지 않는 것을 확인한다.

1. 개념

겉과 속

6교시에 만든 API 는 메모를 딕셔너리에 넣는다. 이번 교시에는 PostgreSQL 에 넣게 바꾼다.

```
바꾸지 않는 것 (겉)                    바꾸는 것 (속)
──────────────────────                 ──────────────────────
주소          POST /memos              저장하는 곳   딕셔너리 → PostgreSQL
요청 모양     {"title": …}             저장하는 코드  memos[id] = … → INSERT INTO …
응답 모양     {"id": …, "title": …}
상태 코드     201, 200, 404, 422
```

API 를 쓰는 쪽(브라우저, `curl`, 테스트)에서 보면 **아무것도 달라지지 않는다.** 달라지는 것은 하나, 서버를 다시 켜도 메모가 남는다는 점이다.

테스트는 계약서다

6교시에 만든 테스트는 "이렇게 부르면 이런 답이 온다"만 확인한다. 메모가 딕셔너리에 있는지 DB 에 있는지는 묻지 않는다. 그래서 **속을 바꿔도 테스트는 그대로 통과해야 한다.**

이번 교시의 가장 중요한 조건:

> **`tests/test_memos.py` 는 한 글자도 바꾸지 않는다.**

AI 가 속을 바꾸다가 겉을 건드리면(응답 모양이 달라지거나 404 문구가 바뀌면) 테스트가 실패해서 알려 준다. 반대로 AI 가 테스트를 고쳐 버리면 이 경보기가 꺼진다. 그래서 테스트 파일을 범위 밖으로 묶어 둔다.

테스트 전용 DB

테스트는 시작할 때마다 테이블을 비운다. 진짜 메모가 든 DB 에서 테스트를 돌리면 **테스트할 때마다 내 메모가 지워진다.** (1교시의 사례 4 가 이것이다.)

데이터베이스	누가 쓰는가	환경 변수
`memo_db`	서버(app 컨테이너). 진짜 메모	`DATABASE_URL`
`memo_test`	테스트(dev 컨테이너의 pytest). 매번 비워진다	`TEST_DATABASE_URL`

두 주소는 `docker-compose.yml` 이 컨테이너에 넣어 준다.

■ [dev] dev 컨테이너

```
echo $DATABASE_URL
echo $TEST_DATABASE_URL
```

```
postgresql://memo:memo1234@db:5432/memo_db
postgresql://memo:memo1234@db:5432/memo_test
```

주소 읽는 법: `postgresql://계정:비밀번호@호스트:포트/데이터베이스`. 호스트가 `localhost` 가 아니라 `db` 다. 컨테이너끼리는 서비스 이름으로 부른다. (4교시의 `http://app:8000` 과 같은 원리다.)

따라 하기

1단계. ①② 요구사항과 지시서 읽기 (15분)

`harness/REQUIREMENTS.md` 4절에서 **N1, N2, N4** 를 읽는다.

`harness/tasks/08_db_connect.md` 를 읽는다. 6교시 지시서와 달라진 점을 찾는다.

칸	6교시 지시서	8교시 지시서
목표	새 기능을 만든다	**겉은 그대로 두고** 저장소만 바꾼다
범위 — 고치면 안 되는 것	`tests/test_health.py`	**`tests/test_memos.py`** 도 추가
하지 말 것	수정·삭제, DB, 화면	ORM 금지, 주소·비밀번호를 코드에 적지 않기
(새로 생긴 칸)	—	**5. 구현 조건** — 테이블 구조, 테스트 전용 DB
완료 조건	테스트가 있다	**테스트 개수가 작업 전과 같다**

"ORM 을 쓰지 않는다"에 주목한다. ORM(SQLAlchemy 등)은 SQL 을 파이썬 코드 뒤에 감춰 주는 도구다. 실무에서는 많이 쓰지만, 이 과정에서는 **7교시에 배운 SQL 이 코드에 그대로 보여야** 내가 읽고 검토할 수 있다. 사람이 검토할 수 있는 방법을 고르게 하는 것도 하네스다.

2단계. 작업 전 상태 기록하기 (5분)

"겉이 그대로인가"를 판단하려면 작업 전의 상태를 알아야 한다.

■ [dev] dev 컨테이너

```
git status
sh scripts/verify.sh
```

`working tree clean` 과 "검증 통과"를 확인한다. **통과한 테스트 개수를 적어 둔다.** (완성 예시 기준 12개. 내 것은 다를 수 있다.)

3단계. ②③ 지시하고 계획 검토하기 (20분)

■ [dev] dev 컨테이너

```
opencode
```

■ [AI] OpenCode 입력 칸

```
harness/tasks/08_db_connect.md 를 읽고 그 지시대로 작업해 줘. AGENTS.md 의 작업 절차를 따라, 먼저 계획만 말하고 내 승인을 기다려.
```

계획 검토 체크리스트

- ■ `tests/test_memos.py` 를 고치겠다는 말이 **없는가?** (있으면 반드시 빼게 한다)
- ■ 새로 만드는 파일이 `app/db.py` 하나인가?
- ■ 접속 주소를 **환경 변수**에서 읽겠다고 했는가? 코드에 `memo1234` 를 적겠다는 말이 없는가?
- ■ SQL 에 `%s` 자리표시자를 쓰겠다고 했는가?
- ■ 테스트가 `memo_test` 에서 돌게 하겠다고 했는가? `memo_db` 를 비우겠다는 말이 없는가?
- ■ SQLAlchemy 같은 새 패키지를 쓰겠다는 말이 **없는가?**
- ■ 수정·삭제를 만들겠다는 말이 **없는가?**

문제가 있으면 지시서의 해당 줄을 가리켜 고치게 하고, 맞으면 승인한다.

■ [AI] OpenCode 입력 칸

```
진행
```

AI 가 일하는 동안 6교시처럼 기록한다. 이번에는 특히 **AI 가 DB 에 직접 명령을 내리려 하는지** 본다. `psql` 이나 `DROP`, `TRUNCATE` 가 든 명령을 실행해도 되냐고 물으면, 어느 데이터베이스에 대한 것인지 확인하고 `memo_db` 라면 거절한다.

4단계. ④ 검증 (20분)

OpenCode 는 열어 두고, 다른 dev 셸에서 직접 확인한다. (**[PS]** `docker compose exec dev bash`)

(1) 기계가 확인하는 것

■ [dev] dev 컨테이너

```
sh scripts/verify.sh
```

완성 예시의 결과:

```
............                                                     [100%]
12 passed in 1.04s
```

- ■ "검증 통과"로 끝난다.
- ■ **테스트 개수가 2단계에서 적어 둔 숫자와 같다.**

실행 시간이 6교시(0.08초)보다 길어졌다. 테스트가 이제 진짜 DB 에 접속하기 때문이다.

(2) 테스트가 바뀌지 않았는가 — 이번 교시의 핵심 확인

■ [dev] dev 컨테이너

```
git diff --stat
```

- ■ 목록에 `tests/test_memos.py` 가 **없다.**
- ■ 바뀐 파일은 `app/main.py`, `tests/conftest.py` 이고, `git status` 에 새 파일 `app/db.py` 가 보인다.

`tests/test_memos.py` 가 목록에 있다면 AI 가 조건을 어긴 것이다. 무엇을 바꿨는지 `git diff tests/test_memos.py` 로 보고, 되돌리게 한다.

■ [AI] OpenCode 입력 칸

```
git diff --stat 에 tests/test_memos.py 가 나와. 지시서 3번에서 이 파일은 고치면 안 된다고 했어. 왜 고쳤는지 설명하고, 원래대로 되돌린 뒤
  app/ 쪽을 고쳐서 통과시켜 줘.
```

(3) 비밀 값이 코드에 들어가지 않았는가

■ [dev] dev 컨테이너

```
grep -rn "memo1234" app/ tests/
```

아무것도 출력되지 않아야 한다. 출력되면 비밀번호가 코드에 적힌 것이다. 환경 변수에서 읽도록 고치게 한다.

(4) 직접 불러 보기

■ [dev] dev 컨테이너

```
curl -s -X POST http://app:8000/memos -H "Content-Type: application/json" -d '{"title": "장보기", "content": "우유, 달걀"}'
curl -s -X POST http://app:8000/memos -H "Content-Type: application/json" -d '{"title": "운동", "content": "저녁 7시 달리기"}'
curl -s http://app:8000/memos
```

마지막 명령의 결과(시각은 실행한 때에 따라 다르다):

```
[{"id":2,"title":"운동","content":"저녁 7시 달리기","created_at":"2026-11-10T05:47:07.086373Z","updated_at":"2026-11-10T05:47:07.086373Z
  "},{"id":1,"title":"장보기","content":"우유, 달걀","created_at":"2026-11-10T05:47:07.051297Z","updated_at":"2026-11-10T05:47:07.051297
  Z"}]
```

응답의 모양이 6교시와 똑같다. **겉은 그대로다.**

(5) 서버를 다시 켜도 남는가 — N1

■ [PS] 윈도우 PowerShell

```
docker compose restart app
```

■ [dev] dev 컨테이너

```
curl -s http://app:8000/memos
```

두 메모가 그대로 나온다. 4교시 5단계에서는 `[]` 였다.

(6) DB 에 정말 들어 있는가

`curl` 은 서버를 거친 결과다. 서버를 거치지 않고 DB 를 직접 들여다본다. 7교시에 배운 그대로다.

■ [PS] 윈도우 PowerShell

```
docker compose exec db psql -U memo -d memo_db -c "SELECT id, title, content FROM memos ORDER BY id;"
```

```
 id ¦ title  ¦     content
----+--------+-----------------
  1 ¦ 장보기 ¦ 우유, 달걀
  2 ¦ 운동   ¦ 저녁 7시 달리기
(2 rows)
```

(7) 테스트가 진짜 메모를 지우지 않는가

■ [dev] dev 컨테이너

```
pytest -q
```

■ [PS] 윈도우 PowerShell

```
docker compose exec db psql -U memo -d memo_db -c "SELECT count(*) FROM memos;"
docker compose exec db psql -U memo -c "\l"
```

- 첫째 명령: `count` 가 여전히 `2` 다. 테스트를 돌렸는데 진짜 메모는 그대로다.
- 둘째 명령: 목록에 `memo_test` 가 생겼다. 테스트가 만든 것이다.

`count` 가 `0` 이 되었다면 테스트가 `memo_db` 에서 돌고 있는 것이다. **심각한 문제다.** AI 에게 지시서 5번의 테스트 전용 DB 조건을 가리켜 고치게 한다.

5단계. ⑤ 코드 읽기 (30분)

완성 예시(`checkpoints/lesson08`)를 기준으로 읽는다. 내 코드에서 같은 역할을 하는 부분을 찾는다.

(1) `app/db.py` — DB 와 대화하는 부분

```
def get_database_url() -> str:
    """환경 변수에서 접속 주소를 읽는다. (docker-compose.yml 이 넣어 준다)"""
    return os.environ["DATABASE_URL"]


def get_conn() -> psycopg.Connection:
    """DB 연결을 하나 연다. 결과 행은 {"열이름": 값} 딕셔너리로 받는다."""
    return psycopg.connect(get_database_url(), row_factory=dict_row)
```

- `os.environ["DATABASE_URL"]` — 주소가 코드에 없다. 2교시의 API 키와 같은 방식이다(N4).
- `row_factory=dict_row` — 결과 한 행을 `{"id": 1, "title": "장보기", …}` 딕셔너리로 받는다. 3교시에 배운 그 딕셔너리다.

```
CREATE_TABLE_SQL = """
CREATE TABLE IF NOT EXISTS memos (
    id          SERIAL PRIMARY KEY,
    title       VARCHAR(100) NOT NULL,
    …
);
"""

def init_db() -> None:
    with get_conn() as conn:
        conn.execute(CREATE_TABLE_SQL)
```

7교시에 손으로 친 `CREATE TABLE` 이 그대로 들어 있다. `IF NOT EXISTS` 가 붙어서, 테이블이 이미 있으면 아무 일도 하지 않는다. 서버가 켜질 때마다 실행해도 안전하다.

(2) `app/main.py` — 만들기

```
@app.post("/memos", response_model=MemoOut, status_code=201)
def create_memo(memo: MemoIn) -> dict:
    with get_conn() as conn:
        row = conn.execute(
            "INSERT INTO memos (title, content) VALUES (%s, %s) RETURNING *",
            (memo.title, memo.content),
        ).fetchone()
    return row
```

6교시의 코드와 나란히 놓는다.

	6교시 (메모리)	8교시 (DB)
첫 줄	`@app.post("/memos", response_model=MemoOut, status_code=201)`	**같다**
번호 정하기	`next_id` 를 직접 센다	DB 가 한다 (`SERIAL`)
시각 정하기	`datetime.now(UTC)`	DB 가 한다 (`DEFAULT now()`)
저장	`memos[next_id] = new_memo`	`INSERT INTO …`
돌려주기	`return new_memo`	`RETURNING *` 로 받은 행을 `return`

- `with get_conn() as conn:` — 연결을 열고, 블록이 끝나면 저장(commit)하고 닫는다. 중간에 오류가 나면 저장하지 않고 되돌린다.
- `.fetchone()` — 결과에서 한 행을 꺼낸다. 없으면 `None`.

(3) 자리표시자 `%s` — 가장 중요한 줄

```
"INSERT INTO memos (title, content) VALUES (%s, %s) RETURNING *",
(memo.title, memo.content),
```

SQL 문장과 값이 **따로** 전달된다. `%s` 자리에 값이 들어가는 일은 드라이버(psycopg)가 안전하게 한다.

하면 안 되는 방식:

```
# 나쁜 예 – 절대 이렇게 쓰지 않는다
conn.execute(f"INSERT INTO memos (title) VALUES ('{memo.title}')")
```

누군가 제목에 `'); DELETE FROM memos; --` 를 넣으면, 이 글자들이 SQL 문장의 일부가 되어 버린다. 사용자가 보낸 글이 명령으로 실행되는 것이다. 이것이 **SQL 인젝션**이다(7교시 9단계에서 AI 에게 물어본 것).

내 코드를 직접 검사한다.

■ [dev] dev 컨테이너

```
grep -n "execute" app/main.py app/db.py
```

`execute(` 가 나오는 줄을 모두 본다. SQL 문자열 앞에 `f` 가 붙어 있거나, `+` 나 `.format(` 으로 값을 이어 붙인 곳이 있으면 규칙(N2) 위반이다.

(4) `app/main.py` — 한 건 보기와 404

```
row = conn.execute("SELECT * FROM memos WHERE id = %s", (memo_id,)).fetchone()
if row is None:  # 그런 번호가 없으면
    raise not_found()
```

7교시 4단계를 떠올린다. DB 는 없는 번호를 찾아도 오류를 내지 않고 0행을 돌려준다. `fetchone()` 은 그때 `None` 을 준다. 그래서 **코드가 직접** `None` 인지 보고 404 를 만든다.

`(memo_id,)` 의 쉼표에 주목한다. 값이 하나뿐이어도 "묶음"으로 전달해야 해서 쉼표를 붙인다. 쉼표를 빼면 오류가 난다. 초보자가 가장 자주 하는 실수다.

(5) `app/main.py` — 서버가 켜질 때

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    """서버가 켜질 때 한 번 실행된다: 테이블이 없으면 만든다."""
    init_db()
    yield  # 여기서부터 서버가 요청을 받는다


app = FastAPI(title="메모 CRUD", lifespan=lifespan)
```

`yield` 앞은 서버가 켜질 때, 뒤는 꺼질 때 실행된다. 문법이 낯설면 "서버가 켜질 때 `init_db()` 를 한 번 부른다"만 이해하고 넘어간다.

(6) `tests/conftest.py` — 테스트 전용 DB

```
MAIN_URL = os.environ["DATABASE_URL"]  # 평소에 쓰는 DB (memo_db)
TEST_URL = os.environ["TEST_DATABASE_URL"]  # 테스트 전용 DB (memo_test)

# 앱이 테스트 전용 DB 를 보게 주소를 바꿔 끼운다. (app 을 불러오기 전에 해야 한다)
os.environ["DATABASE_URL"] = TEST_URL
```

테스트를 시작할 때 접속 주소를 `memo_test` 로 바꿔치기한다. 앱 코드는 여전히 `DATABASE_URL` 을 읽을 뿐인데, 그 값이 테스트 DB 를 가리키게 된 것이다. **앱 코드에는 "테스트 중이면..." 같은 분기가 없다.**

```
@pytest.fixture
def client():
    with TestClient(app) as test_client:
        with get_conn() as conn:
            # 메모를 모두 지우고 번호도 1번부터 다시 시작하게 한다.
            conn.execute("TRUNCATE TABLE memos RESTART IDENTITY")
        yield test_client
```

6교시의 `reset_store()` 가 `TRUNCATE` 로 바뀌었다. 하는 일은 같다. 테스트마다 깨끗하게. `TRUNCATE` 는 테이블을 통째로 비우는 명령이다. **이 명령이 `memo_test` 에서만 실행된다는 것**을 4단계 (7)에서 확인했다.

(7) AI 에게 묻기

■ [AI] OpenCode 입력 칸

```
파일은 고치지 말고 설명만 해 줘. tests/conftest.py 에서 os.environ["DATABASE_URL"] = TEST_URL 줄을 from app.main import app 아래로 옮기면 어떤 일이 생길 수 있어?
```

■ [AI] OpenCode 입력 칸

```
파일은 고치지 말고 설명만 해 줘. app/main.py 의 with get_conn() as conn: 블록 안에서 오류가 나면 INSERT 한 내용은 어떻게 돼?
```

6단계. ⑤ 기록하고 저장하기 (10분)

`harness/WORKLOG.md` 에 8교시 기록을 쓴다. 이번에는 "3. 하네스가 막거나 물어본 것"에 **AI 가 테스트 파일을 건드렸는지**를 꼭 적는다.

■ [dev] dev 컨테이너

```
git add -A
git status
git commit -m "8교시: 저장소를 PostgreSQL 로 교체 (N1, N2, N4)"
```

OpenCode 를 `/exit` 로 닫는다.

하네스 관점 — 이번 교시에서 본 것

- **테스트는 계약서다.** 속을 통째로 바꿨는데 테스트 12개가 그대로 통과했다. 겉이 바뀌지 않았다는 증거다. 테스트가 없었다면 응답을 하나하나 눈으로 대조해야 했을 것이다.
- **범위 제한이 경보기를 지킨다.** "테스트 파일을 고치지 않는다"는 조건이 없으면, AI 는 실패하는 테스트를 고쳐서 통과시킬 수 있다. 그 조건을 `git diff --stat` 한 줄로 확인했다.
- **확인은 한 단계 더 깊이.** `curl` 은 서버가 하는 말이다. `psql` 로 DB 를 직접 들여다보고, 서버를 다시 켜 보고, 테스트를 돌린 뒤 진짜 데이터를 세어 봤다. AI 의 보고 → 내 `verify.sh` → `curl` → `psql`. 한 단계씩 더 직접적인 증거로 내려간다.

자주 만나는 오류

증상	원인과 해결
`psycopg.OperationalError: connection failed … Name or service not known`	접속 주소의 호스트가 틀렸다(`localhost` 로 적었을 수 있다). 컨테이너에서는 `db` 다. `echo $DATABASE_URL` 로 확인한다.
`KeyError: 'DATABASE_URL'` 또는 `'TEST_DATABASE_URL'`	환경 변수가 없다. dev 컨테이너 안에서 실행했는지 확인한다. **[PS]** `docker compose up -d` 뒤 다시 들어간다.
`relation "memos" does not exist`	테이블을 만드는 코드(`init_db`)가 서버 시작 때 실행되지 않는다. `lifespan` 이 `FastAPI(...)` 에 연결되어 있는지 본다. 테스트에서는 `with TestClient(app)` 를 써야 `lifespan` 이 실행된다.
`TypeError: … not all arguments converted` 또는 `the query has 1 placeholders but 5 parameters were passed`	`(memo_id,)` 의 쉼표를 빠뜨렸다. 값이 하나여도 쉼표가 필요하다.
서버가 500 을 돌려준다	**[PS]** `docker compose logs app --tail 30` 에서 `Traceback` 의 마지막 줄을 읽는다. 그 출력을 AI 에게 붙여 준다.
테스트를 돌리면 진짜 메모가 사라진다	테스트가 `memo_db` 에서 돈다. `conftest.py` 가 `TEST_DATABASE_URL` 로 주소를 바꿔 끼우는지 확인하게 한다.
`database "memo_test" does not exist`	`conftest.py` 가 테스트 DB 를 만들지 않는다. 지시서 5번 조건을 가리켜 고치게 하거나, 급하면 **[PS]** `docker compose exec db psql -U memo -d memo_db -c "CREATE DATABASE memo_test;"`
시각이 9시간 어긋나 보인다	응답의 끝이 `Z` 면 세계 표준시(UTC)다. 한국 시각은 9시간을 더한 값이다. 10교시에 화면에서 한국 시각으로 바꿔 보여 준다.

확인 문제

1. 이번 교시에 "바꾼 것"과 "바꾸지 않은 것"을 각각 세 가지씩 쓰시오.
2. 저장소를 통째로 바꿨는데 `tests/test_memos.py` 를 고칠 필요가 없었던 이유는 무엇인가?
3. `git diff --stat` 에 `tests/test_memos.py` 가 나타났다. 이것이 왜 문제인가? 어떻게 대응하는가?
4. `conn.execute("SELECT * FROM memos WHERE id = %s", (memo_id,))` 에서 `%s` 와 `(memo_id,)` 는 각각 무엇인가? 쉼표는 왜 있는가?
5. f-문자열로 SQL 을 만들면 왜 위험한가? 한 문장으로 쓰시오.
6. `fetchone()` 이 `None` 을 돌려주는 경우는 언제인가? 그때 코드는 무엇을 해야 하는가?
7. 테스트 전용 DB 가 없으면 어떤 사고가 생기는가? 이번 교시에 그 사고가 나지 않았음을 어떻게 확인했는가?
8. 접속 주소 `postgresql://memo:memo1234@db:5432/memo_db` 의 각 부분이 무엇인지 쓰시오. 이 주소는 코드의 어디에 적혀 있는가?

9교시. 수정·삭제 구현

시간	2시간
실습 결과	CRUD 완성 (만들기·보기·고치기·지우기)
시작 상태	8교시를 마친 상태 (메모가 PostgreSQL 에 저장되고 커밋되어 있다)

8교시 결과가 없으면 **[dev]** `sh scripts/checkpoint.sh 08` 로 가져온다.

수업의 흐름에서 이번 교시의 위치

```
① 요구사항 작성 → ② AI 에게 지시 → ③ 하네스로 제어 → ④ 실행·테스트 → ⑤ 수정·재검증·저장
└──────────── 이번 교시: 세 번째 바퀴. 이번에는 내가 지시서를 먼저 써 본다 ────────────┘
```

학습 목표

1. 작업 지시서를 직접 써 보고, 준비된 지시서와 비교한다.
2. PUT 과 DELETE 의 요청·응답·상태 코드를 설명한다.
3. "테스트는 추가만, 삭제·수정은 금지"라는 조건을 `git diff` 로 확인한다.
4. AI 가 쓴 `UPDATE·DELETE` 문에 `WHERE` 가 있는지 직접 검사한다.
5. CRUD 네 가지가 HTTP·파이썬 함수·SQL 로 어떻게 이어지는지 한 장의 표로 정리한다.

1. 개념

PUT 과 DELETE

	PUT `/memos/1`	**DELETE** `/memos/1`
뜻	1번 메모를 이 내용으로 바꿔라	1번 메모를 지워라
요청 본문	있다 (`{"title": …, "content": …}`)	없다
성공	**200** + 고쳐진 메모	**204**, 본문 없음
번호가 없으면	404	404
입력이 틀리면	422 (R6)	—

- PUT 은 메모 **전체**를 새 내용으로 바꾼다. 제목만 보내고 내용을 빼면 내용은 기본값(`""`)이 된다.
- DELETE 가 204 인 이유: 지워서 없어진 것을 돌려줄 수는 없다. "성공했고 돌려줄 것은 없다"가 204 다.

되돌릴 수 없는 작업

만들기와 보기는 잘못되어도 피해가 작다. 고치기와 지우기는 다르다. **있던 데이터가 바뀌거나 사라진다.** 7교시에 본 대로 `WHERE` 를 빠뜨리면 모든 메모가 한꺼번에 바뀌거나 지워진다.

그래서 이번 교시에는 검토할 것이 하나 더 있다. **AI 가 쓴 `UPDATE·DELETE` 문을 한 줄씩 직접 본다.**

따라 하기

1단계. ① 요구사항 읽기 (5분)

`harness/REQUIREMENTS.md` 의 **R4, R5** 를 읽는다. R4 에는 조건이 세 가지, R5 에도 세 가지 들어 있다. 종이에 번호를 붙여 옮겨 적는다. 이것이 4단계에서 확인할 목록이다.

2단계. ② 작업 지시서를 내가 먼저 쓰기 (20분)

지금까지는 준비된 지시서를 썼다. 이번에는 **보기 전에 먼저 써 본다.**

`harness/TASK_TEMPLATE.md` 의 틀을 복사해 메모장이나 VS Code 의 새 파일(저장하지 않아도 된다)에 붙이고 여섯 칸을 채운다. 아래 질문을 따라 쓴다.

칸	스스로 묻는다
1. 목표	이 작업이 끝나면 무엇이 되어 있는가?
2. 요구사항	어느 번호인가? 고치기에도 입력 규칙이 적용되는가?
3. 범위	어떤 파일을 고쳐야 하는가? `app/db.py` 는 고칠 필요가 있는가? 테스트 파일은 어떻게 다뤄야 하는가? (8교시에는 "고치지 않는다"였다. 이번에는?)
4. 하지 말 것	AI 가 덧붙일 만한 것은 무엇인가? (일부만 고치기, 휴지통, 화면 …)
5. 완료 조건	테스트 개수는 어떻게 되어야 하는가? 어떤 경우의 테스트가 있어야 하는가?
6. 진행 방법	계획 → 승인 → 구현 → 검증 → 보고

다 썼으면 `harness/tasks/09_update_delete.md` 를 열어 비교한다.

- 준비된 지시서에 있는데 내 것에 없는 조건은 무엇인가? 그 조건이 없으면 어떤 일이 생길 수 있는가?
- 3번 범위의 "`tests/test_memos.py`(테스트 **추가**만)"과 6번의 "`git diff tests/` 에서 지워진 줄이 없다"를 찾는다. 8교시의 "한 글자도 바꾸지 않는다"가 이번에는 "**늘리기만 한다**"가 되었다.

이번 실습은 준비된 지시서로 진행한다. 내 지시서가 더 낫다고 생각되는 부분이 있으면 지시할 때 말로 덧붙인다.

3단계. ②③ 지시하고 계획 검토하기 (15분)

■ [dev] dev 컨테이너

```
git status
sh scripts/verify.sh
```

`working tree clean`, "검증 통과", 그리고 **테스트 개수**를 적어 둔다. (완성 예시 기준 12개)

■ [dev] dev 컨테이너

```
opencode
```

■ [AI] OpenCode 입력 칸

```
harness/tasks/09_update_delete.md 를 읽고 그 지시대로 작업해 줘. AGENTS.md 의 작업 절차를 따라, 먼저 계획만 말하고 내 승인을 기다려.
```

계획 검토 체크리스트

- ■ 고치는 파일이 `app/main.py` 와 `tests/test_memos.py` 뿐인가?
- ■ 기존 테스트를 고치거나 지우겠다는 말이 **없는가?**
- ■ `UPDATE·DELETE` 에 `WHERE id = %s` 를 쓰겠다고 했는가?
- ■ 없는 번호일 때 404 를 어떻게 알아낼지 말했는가? (7교시: DB 는 오류를 내지 않는다)
- ■ 고칠 때 `updated_at` 을 바꾸겠다고 했는가?
- ■ PATCH, 휴지통, 화면을 만들겠다는 말이 **없는가?**

맞으면 진행 이라고 답한다.

4단계. ④ 검증 (25분)

다른 dev 셸에서 직접 확인한다.

(1) 기계가 확인하는 것

■ [dev] dev 컨테이너

```
sh scripts/verify.sh
```

완성 예시의 결과:

```
.................                                                [100%]
17 passed in 1.59s
```

■ "검증 통과"로 끝난다.

■ 테스트 개수가 3단계의 숫자보다 **늘었다.**

(2) 기존 테스트가 그대로인가 — 이번 교시의 핵심 확인

■ [dev] dev 컨테이너

```
git diff --stat
git diff tests/ ¦ grep "^-" ¦ grep -v "^---"
```

둘째 명령은 tests/ 의 변경에서 **지워진 줄**만 골라 보여 준다.

■ 첫째 명령: app/main.py 와 tests/test_memos.py 두 파일만 나온다.

■ 둘째 명령: **아무것도 출력되지 않는다.** (지워진 줄이 없다)

출력이 있으면 AI 가 기존 테스트를 건드린 것이다. 무엇을 왜 바꿨는지 묻고 되돌리게 한다.

(3) SQL 을 직접 검사한다

■ [dev] dev 컨테이너

```
grep -n -A3 "UPDATE\¦DELETE FROM" app/main.py
```

UPDATE 와 DELETE FROM 이 나오는 줄과 그 아래 세 줄을 보여 준다.

■ UPDATE 문에 WHERE id = %s 가 있다.

■ DELETE 문에 WHERE id = %s 가 있다.

■ 값을 f-문자열이나 + 로 이어 붙이지 않았다.

테스트가 통과했더라도 **이 세 줄은 반드시 눈으로 본다.** "1번을 고쳤더니 1번이 바뀌었다"는 테스트는 WHERE 가 없어도 통과할 수 있다. 메모가 한 건뿐인 테스트에서는 "전부 바꾸기"와 "1번만 바꾸기"의 결과가 같기 때문이다.

(4) 직접 불러 보기

8교시에 넣어 둔 메모 두 건(1번 장보기, 2번 운동)이 있다. 없으면 curl 로 두 건을 만든다.

고치기:

■ [dev] dev 컨테이너

```
curl -s -X PUT http://app:8000/memos/1 -H "Content-Type: application/json" -d '{"title": "장보기(수정)", "content": "우유, 달걀, 두부"}'
```

```
{"id":1,"title":"장보기(수정)","content":"우유, 달걀, 두부","created_at":"2026-11-10T05:47:07.051297Z","updated_at":"2026-11-10T05:47:11
  .488803Z"}
```

created_at 은 그대로이고 updated_at 만 바뀌었다. (R4)

지우기 — 상태 코드를 본다:

■ [dev] dev 컨테이너

```
curl -s -o /dev/null -w "%{http_code}\n" -X DELETE http://app:8000/memos/2
```

```
204
```

목록:

■ [dev] dev 컨테이너

```
curl -s http://app:8000/memos
```

```
[{"id":1,"title":"장보기(수정)","content":"우유, 달걀, 두부","created_at":"2026-11-10T05:47:07.051297Z","updated_at":"2026-11-10T05:47:1
  1.488803Z"}]
```

2번이 사라지고 1번만 남았다. **1번이 지워지지 않았다는 것**이 `WHERE` 가 제대로 있다는 실제 증거다.

같은 메모를 한 번 더 지운다:

■ [dev] dev 컨테이너

```
curl -s -X DELETE http://app:8000/memos/2
```

```
{"detail":"메모를 찾을 수 없습니다."}
```

없는 번호를 고친다:

■ [dev] dev 컨테이너

```
curl -s -X PUT http://app:8000/memos/999 -H "Content-Type: application/json" -d '{"title": "없는 메모", "content": ""}'
```

```
{"detail":"메모를 찾을 수 없습니다."}
```

(5) DB 에서 직접 확인

■ [PS] 윈도우 PowerShell

```
docker compose exec db psql -U memo -d memo_db -c "SELECT id, title, content FROM memos ORDER BY id;"
```

```
 id ¦    title     ¦     content
----+--------------+------------------
  1 ¦ 장보기(수정) ¦ 우유, 달걀, 두부
(1 row)
```

(6) 1단계에서 적은 R4·R5 의 조건 여섯 가지를 하나씩 짚으며, 각각을 확인하는 테스트가 `tests/test_memos.py` 에 있는지 찾는다. `harness/VERIFY_CHECKLIST.md` 의 대응표를 쓴다.

5단계. ⑤ 코드 읽기 (25분)

완성 예시(`checkpoints/lesson09`)를 기준으로 읽는다.

(1) 고치기

```python
@app.put("/memos/{memo_id}", response_model=MemoOut)
def update_memo(memo_id: int, memo: MemoIn) -> dict:
    with get_conn() as conn:
        # WHERE 가 없으면 모든 메모가 바뀐다. 반드시 id 로 한 건만 고른다.
        row = conn.execute(
            "UPDATE memos SET title = %s, content = %s, updated_at = now() "
            "WHERE id = %s RETURNING *",
            (memo.title, memo.content, memo_id),
        ).fetchone()
    if row is None:
        raise not_found()
    return row
```

- 인자가 둘이다. `memo_id` 는 **주소**에서, `memo` 는 **본문**에서 온다. FastAPI 가 알아서 나눠 준다.
- `memo: MemoIn` — 만들기와 **같은 클래스**를 쓴다. 그래서 입력 규칙(R6)이 고치기에도 저절로 적용된다. 규칙을 한 곳에만 적어 두는 이유다.
- `%s` 가 세 개, 값도 세 개. **순서가 맞아야 한다.** `(memo.title, memo.content, memo_id)` 의 순서를 SQL 의 `%s` 순서와 하나씩 짚어 본다.
- 문자열 두 줄이 나란히 있으면 파이썬이 하나로 이어 붙인다. 첫 줄 끝의 공백(`now() "`)이 없으면 `now()WHERE` 가 되어 오류가 난다.
- 없는 번호면 `UPDATE` 는 0행을 바꾸고 `RETURNING` 은 아무것도 돌려주지 않는다. 그래서 `row` 가 `None` 이 되고 404 가 된다. 7교시 5·7단계에서 본 "DB 는 없다를 오류로 알리지 않는다"를 코드가 처리한 것이다.

(2) 지우기

```
@app.delete("/memos/{memo_id}", status_code=204)
def delete_memo(memo_id: int) -> None:
    with get_conn() as conn:
        row = conn.execute(
            "DELETE FROM memos WHERE id = %s RETURNING id", (memo_id,)
        ).fetchone()
    if row is None:
        raise not_found()
    # 204 는 "성공했고, 돌려줄 내용은 없음"이라는 뜻이라 아무것도 돌려주지 않는다.
```

- `-> None` 과 `return` 없음 — 돌려줄 것이 없다. 204 와 짝이다.
- `RETURNING id` — 지운 행의 번호를 돌려받는다. 지운 것이 없으면 `None` 이다. 이것으로 "지웠는가, 없었는가"를 구분한다.

(3) 404 를 한 곳으로

```
def not_found() -> HTTPException:
    """'그 번호의 메모가 없다'는 404 오류를 한 곳에서 만든다."""
    return HTTPException(status_code=404, detail="메모를 찾을 수 없습니다.")
```

보기·고치기·지우기 세 곳에서 같은 404 를 쓴다. 문구를 세 번 적으면 언젠가 한 곳만 다르게 고쳐지는 일이 생긴다. 한 곳에 두면 그럴 수 없다.

(4) 테스트

```
def test_update_memo(client):
    created = make_memo(client)
    response = client.put(
        f"/memos/{created['id']}", json={"title": "장보기(수정)", "content": "우유"}
    )
    assert response.status_code == 200
    assert response.json()["title"] == "장보기(수정)"
    # 다시 조회해도 바뀐 내용이 남아 있어야 한다(진짜로 저장되었는지 확인).
    assert client.get(f"/memos/{created['id']}").json()["content"] == "우유"
```

마지막 줄에 주목한다. PUT 의 응답만 보지 않고 **다시 조회해서** 확인한다. 응답만 그럴듯하게 돌려주고 저장은 하지 않는 버그를 잡는다.

(5) 테스트가 못 잡는 것을 잡아 보기 — 직접 실험

4단계 (3)에서 "`WHERE` 가 없어도 테스트가 통과할 수 있다"고 했다. 정말 그런지 본다.

VS Code 에서 `app/main.py` 의 `UPDATE` 문에서 `WHERE id = %s` 를 지우고, 값 묶음에서도 `, memo_id` 를 지운다. 저장하고 실행한다.

■ [dev] dev 컨테이너

```
pytest -q
```

실패하는 테스트가 있는가? 어떤 테스트가 잡았는가, 혹은 하나도 잡지 못했는가? 결과를 적는다.

Ctrl+Z 로 반드시 되돌리고 `sh scripts/verify.sh` 로 통과를 확인한다.

하나도 잡지 못했다면, 테스트에 빈틈이 있는 것이다. AI 에게 메우게 한다.

■ [AI] OpenCode 입력 칸

```
UPDATE 문에서 WHERE 를 빼도 테스트가 모두 통과했어. "메모 두 건을 만들고 한 건만 고쳤을 때 다른 한 건은 바뀌지 않는다"를 확인하는 테스트
  를 tests/test_memos.py 에 추가해 줘. 삭제에 대해서도 같은 테스트를 추가해 줘. 기존 테스트는 건드리지 마. 먼저 계획을 말해 줘.
```

추가된 뒤, 같은 실험(WHERE 지우기)을 다시 해서 이번에는 테스트가 실패하는지 확인한다. 확인했으면 되돌린다.

> 방금 한 일에 이름이 있다. **코드를 일부러 망가뜨려서 테스트가 알아채는지 보는 것.** 테스트를 믿어도 되는지 확인하는 가장 확실한 방법이다.

6단계. CRUD 한 장으로 정리하기 (10분)

완성된 CRUD 를 표로 정리한다. 내 `app/main.py` 를 보며 빈칸을 채운다.

하는 일	HTTP	파이썬 함수	SQL	성공	실패
만들기	`POST /memos`		`INSERT … RETURNING *`	201	422
목록	`GET /memos`		`SELECT … ORDER BY id DESC`	200	—
한 건	`GET /memos/{id}`		`SELECT … WHERE id = %s`	200	404
고치기	`PUT /memos/{id}`			200	404, 422
지우기	`DELETE /memos/{id}`			204	404

이 표를 `harness/WORKLOG.md` 의 9교시 기록에 붙인다. 12교시 발표에 쓴다.

7단계. ⑤ 기록하고 저장하기 (10분)

`harness/WORKLOG.md` 에 9교시 기록을 쓴다. 5단계 (5)의 실험 결과(테스트가 잡았는가)를 꼭 적는다.

■ [dev] dev 컨테이너

```
git add -A
git status
git commit -m "9교시: 메모 수정·삭제 구현, CRUD 완성 (R4, R5)"
```

OpenCode 를 `/exit` 로 닫는다.

하네스 관점 — 이번 교시에서 본 것

- **테스트 조건의 세 가지 모양.** 6교시 "만들어라", 8교시 "한 글자도 바꾸지 마라", 9교시 "늘리기만 해라". 작업의 성격에 따라 테스트를 묶는 방식이 달라진다. 그리고 매번 `git diff` 한 줄로 확인했다.
- **위험한 작업일수록 사람이 직접 본다.** 되돌릴 수 없는 `UPDATE·DELETE` 는 테스트 통과만 믿지 않고 SQL 세 줄을 눈으로 봤다.
- **테스트도 검증 대상이다.** 코드를 일부러 망가뜨려 테스트가 알아채는지 봤다. 못 알아챘다면 그 빈틈을 메우는 것이 하네스를 고치는 일이다.
- **지시서를 쓰는 것이 설계다.** 2단계에서 범위와 "하지 말 것"을 정할 때, 이미 무엇을 어떻게 만들지 절반은 결정했다.

자주 만나는 오류

증상	원인과 해결
PUT 이 405 `Method Not Allowed`	`@app.put("/memos/{memo_id}")` 가 없거나 주소가 다르다. `/docs` 에서 PUT 이 보이는지 확인한다.
PUT 에 제목만 보냈더니 내용이 지워졌다	정상이다. PUT 은 전체를 바꾼다. 안 보낸 `content` 는 기본값 `""` 가 된다.
`syntax error at or near "WHERE"`	두 줄로 나눈 SQL 문자열 사이에 공백이 없다(`now()WHERE`). 첫 줄 끝에 공백을 넣는다.
`the query has 3 placeholders but 2 parameters were passed`	`%s` 의 개수와 값의 개수가 다르다. 하나씩 짚어 센다.
고쳤는데 다른 메모까지 바뀌었다	`WHERE` 가 없다. 4단계 (3)의 검사를 한다. DB 의 데이터는 되돌릴 수 없으니 메모를 다시 만든다.
DELETE 가 200 과 `null` 을 돌려준다	`status_code=204` 를 빠뜨렸다. 요구사항 R5 를 가리켜 고치게 한다.
`grep "^-"` 에 출력이 있다	기존 테스트의 줄이 지워졌거나 바뀌었다. `git diff tests/` 전체를 읽고, 이유를 AI 에게 묻는다.

확인 문제

1. PUT 과 DELETE 의 성공 상태 코드가 다른 이유를 설명하시오.
2. `update_memo(memo_id: int, memo: MemoIn)` 에서 두 인자는 요청의 어느 부분에서 오는가?
3. 고치기 함수가 만들기와 같은 `MemoIn` 을 쓰면 무엇이 좋은가?
4. 없는 번호를 고치려 할 때, DB 는 오류를 내지 않는데 서버는 어떻게 404 를 돌려주는가? 코드의 어느 줄이 그 일을 하는가?
5. 메모가 한 건뿐인 테스트로는 `WHERE` 가 빠진 `UPDATE` 를 잡지 못하는 이유를 설명하시오.
6. `test_update_memo` 가 PUT 응답만 확인하지 않고 다시 GET 으로 조회하는 이유는 무엇인가?
7. 8교시와 9교시의 테스트 파일 조건은 어떻게 다른가? 각각 어떤 명령으로 확인했는가?
8. 내가 쓴 지시서와 준비된 지시서를 비교했을 때, 내 것에 빠져 있던 조건 하나와 그것이 없으면 생길 수 있는 일을 쓰시오.

10교시. HTML·JS 화면 연결

시간	2시간
실습 결과	웹에서 CRUD 실행 (브라우저 화면에서 메모를 쓰고, 보고, 고치고, 지운다)
시작 상태	9교시를 마친 상태 (CRUD API 가 완성되어 커밋되어 있다)

9교시 결과가 없으면 **[dev]** `sh scripts/checkpoint.sh 09` 로 가져온다.

수업의 흐름에서 이번 교시의 위치

```
① 요구사항 작성 → ② AI 에게 지시 → ③ 하네스로 제어 → ④ 실행·테스트 → ⑤ 수정·재검증·저장
└──────── 이번 교시: 네 번째 바퀴. 테스트가 보지 못하는 것을 사람이 확인한다 ────────┘
```

학습 목표

1. HTML·CSS·자바스크립트가 각각 맡는 일을 구분한다.
2. 화면의 버튼이 `fetch` 를 거쳐 API 로 이어지는 흐름을 설명한다.
3. 브라우저 개발자 도구로 화면이 보내는 요청을 직접 본다.
4. 자동 테스트가 확인하지 못하는 것을 수동 점검표로 확인한다.
5. `innerHTML` 과 `textContent` 의 차이를 실험으로 확인한다.

1. 개념

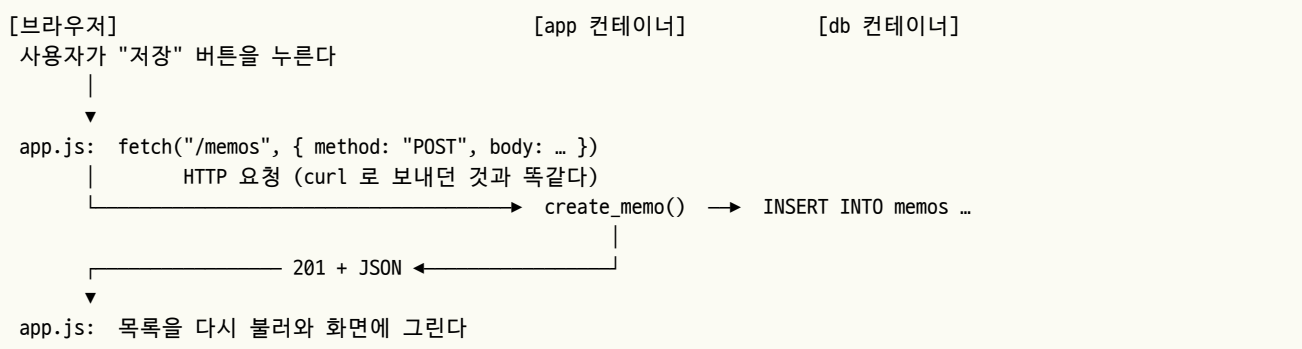
세 가지 파일

파일	맡는 일	비유
`index.html`	화면의 뼈대. 입력 칸, 버튼, 목록 자리	집의 골조
`style.css`	꾸미기. 색, 크기, 간격	도배와 페인트
`app.js`	동작. 버튼을 누르면 무슨 일이 일어나는가	전기와 배관

`style.css` 가 없어도 메모장은 동작한다. `app.js` 가 없으면 버튼을 눌러도 아무 일도 일어나지 않는다.

화면과 API 는 어떻게 이어지는가

지금까지 `/docs` 화면이나 `curl` 로 API 를 불렀다. 이제 그 일을 **자바스크립트가 대신** 한다.



`fetch` 는 자바스크립트에서 HTTP 요청을 보내는 함수다. `curl` 과 짝지어 보면 읽힌다.

	curl (4교시)	fetch (이번 교시)
주소	`http://app:8000/memos`	`"/memos"`
방식	`-X POST`	`method: "POST"`
머리말	`-H "Content-Type: application/json"`	`headers: { "Content-Type": "application/json" }`
본문	`-d '{"title": "장보기"}'`	`body: JSON.stringify({ title: "장보기" })`

API 는 하나도 바뀌지 않는다. 9교시까지 만든 것을 그대로 쓴다. 화면은 API 를 부르는 또 하나의 손님일 뿐이다.

자동 테스트가 보지 못하는 것

`pytest` 는 서버에 요청을 보내고 응답을 확인한다. **브라우저를 열지 않는다.** 그래서 다음은 확인하지 못한다.

- 버튼을 눌렀을 때 정말 요청이 나가는가
- 저장한 뒤 목록이 새로 그려지는가
- 오류 문구가 화면에 보이는가
- 글자가 상자 밖으로 삐져나오지 않는가

이번 교시의 검증은 **자동 테스트 + 사람의 수동 점검표** 두 겹이다. 하네스의 "검증"에 사람의 눈이 들어가야 하는 자리가 있다.

따라 하기

1단계. ①② 요구사항과 지시서 읽기 (10분)

`harness/REQUIREMENTS.md` 의 **R7** 과 **N6** 을 읽는다.

`harness/tasks/10_web_page.md` 를 읽는다. 이번 지시서에서 눈여겨볼 곳:

- **4. 하지 말 것** — 프레임워크·외부 라이브러리·CDN 금지. AI 는 화면을 만들라고 하면 React 나 Bootstrap 을 가져오려는 경향이 있다. 그러면 내가 읽어야 할 것이 몇 배로 늘어난다.
- **5. 구현 조건** — 화면의 구성과 동작이 문장으로 적혀 있다. 화면은 "어떻게 생겨야 하는가"를 말로 정해 주지 않으면 매번 전혀 다른 것이 나온다.
- **6. 완료 조건** — 다섯 개 중 두 개에 "(사람이 확인한다)"가 붙어 있다. AI 가 확인할 수 없는 조건이다.

2단계. ②③ 지시하고 계획 검토하기 (15분)

■ [dev] dev 컨테이너

```
git status
sh scripts/verify.sh
opencode
```

■ [AI] OpenCode 입력 칸

```
harness/tasks/10_web_page.md 를 읽고 그 지시대로 작업해 줘. AGENTS.md 의 작업 절차를 따라, 먼저 계획만 말하고 내 승인을 기다려.
```

계획 검토 체크리스트

- ■ 새 파일이 `app/static/` 아래 세 개(`index.html`, `app.js`, `style.css`)인가?
- ■ React, Vue, jQuery, Bootstrap, Tailwind, CDN 이라는 낱말이 **없는가?**
- ■ `/memos` API 를 고치거나 새 API 를 만들겠다는 말이 **없는가?**
- ■ 사용자 글을 `textContent` 로 넣겠다고 했는가?
- ■ `GET /` 의 테스트를 추가하겠다고 했는가?
- ■ `npm`, `node` 를 설치하거나 실행하겠다는 말이 **없는가?**

맞으면 진행.

AI 가 `curl http://app:8000/` 로 화면이 내려오는지 확인할 수 있다. 하지만 **AI 는 브라우저를 열어 볼 수 없다.** AI 의 보고에 "화면에서 정상 동작을 확인했습니다"라는 말이 있다면, 무엇으로 확인했는지 되묻는다.

3단계. ④ 검증 1 — 기계가 확인하는 것 (10분)

■ **[dev] dev 컨테이너** (다른 셸)

```
sh scripts/verify.sh
```

완성 예시의 결과:

```
..................                                       [100%]
18 passed in 1.77s
```

■ **[dev] dev 컨테이너**

```
git status
git diff --stat
git diff tests/ ¦ grep "^-" ¦ grep -v "^---"
```

- 새 파일은 `app/static/` 아래 세 개다.
- 바뀐 파일은 `app/main.py`, `tests/test_memos.py` 다.
- 셋째 명령은 아무것도 출력하지 않는다. (기존 테스트가 지워지지 않았다)

금지한 것이 들어오지 않았는지 검사한다.

■ **[dev] dev 컨테이너**

```
grep -n "innerHTML" app/static/app.js
grep -n -i "cdn\¦https://\¦http://" app/static/index.html app/static/app.js
```

- 첫째 명령: 출력이 없다. (`innerHTML` 을 쓰지 않았다)
- 둘째 명령: 출력이 없다. (외부에서 가져오는 것이 없다)

화면 파일이 내려오는지 확인한다.

■ **[dev] dev 컨테이너**

```
curl -s -o /dev/null -w "%{http_code} %{content_type}\n" http://app:8000/
curl -s -o /dev/null -w "%{http_code} %{content_type}\n" http://app:8000/static/app.js
curl -s -o /dev/null -w "%{http_code} %{content_type}\n" http://app:8000/static/style.css
```

```
200 text/html; charset=utf-8
200 text/javascript; charset=utf-8
200 text/css; charset=utf-8
```

4단계. ④ 검증 2 — 사람이 확인하는 것 (25분)

브라우저에서 http://localhost:8000 을 연다. 아래 점검표를 **순서대로** 한다. 하나라도 안 되면 5단계 뒤의 "화면이 이상할 때"로 간다.

수동 점검표

번호	할 일	기대하는 결과	확인
1	화면을 연다	"메모장" 제목, 입력 칸 둘, 저장 버튼, 9교시에 남겨 둔 메모가 목록에 보인다	[ ]
2	제목 "회의", 내용 "오후 3시" 를 넣고 저장	새로 고침 없이 목록 **맨 위**에 나타난다. 입력 칸이 비워진다	[ ]
3	제목을 비운 채 저장	저장되지 않는다. 안내가 나온다	[ ]

번호	할 일	기대하는 결과	확인
4	"회의" 메모의 수정 버튼	위 입력 칸에 "회의", "오후 3시" 가 채워진다. 버튼이 "수정 저장"으로 바뀐다	[ ]
5	내용을 "오후 4시" 로 바꾸고 수정 저장	목록의 내용이 바뀐다. 메모가 하나 더 생기지 **않는다**	[ ]
6	수정 버튼을 누른 뒤 수정 취소	입력 칸이 비워지고 버튼이 "저장"으로 돌아온다	[ ]
7	삭제 버튼	정말 지울지 묻는다. 취소하면 그대로 있다	[ ]
8	삭제 버튼 → 확인	목록에서 사라진다	[ ]
9	브라우저를 새로 고친다(F5)	2~8 의 결과가 그대로다 (DB 에 저장되었다)	[ ]
10	내용에 여러 줄을 넣고 저장	줄바꿈이 그대로 보인다	[ ]
11	제목에 띄어쓰기 없는 아주 긴 글자(100자)를 넣고 저장	글자가 상자 밖으로 나가지 않는다	[ ]

화면이 보내는 요청을 직접 본다.

브라우저에서 **F12** 를 눌러 개발자 도구를 열고 **Network(네트워크)** 탭을 고른다. 그 상태에서 메모를 하나 저장한다.

목록에 요청 두 줄이 나타난다.

이름	방식	상태	뜻
`memos`	POST	201	저장 버튼이 보낸 요청
`memos`	GET	200	저장 뒤 목록을 다시 불러온 요청

POST 줄을 눌러 **Payload(페이로드)** 를 본다. 4교시에 `curl -d` 로 보내던 JSON 과 같은 모양이다. **Response(응답)** 에는 `curl` 로 보던 응답이 있다.

수정과 삭제도 해 보고, 각각 어떤 방식·주소·상태 코드가 나가는지 6단계의 표에 적는다.

사용자 글이 글자로만 다뤄지는가 — 직접 실험

제목 칸에 아래를 그대로 넣고 저장한다.

```
<b>굵게</b><img src=x onerror="alert('XSS')">
```

결과	뜻
목록에 `<b>굵게</b><img src=x …>` 가 **글자 그대로** 보인다	안전하다. `textContent` 로 넣었다
"굵게"가 굵은 글씨로 보이거나, 알림 창이 뜬다	**위험하다.** 사용자가 넣은 글이 HTML 로 실행되었다. `innerHTML` 을 썼다

둘째 결과가 나왔다면 AI 에게 지시서 5번의 해당 줄을 가리켜 고치게 한다. 확인이 끝나면 이 메모는 지운다.

왜 위험한가: 메모장을 여러 사람이 함께 쓴다면, 한 사람이 넣은 글이 다른 사람의 브라우저에서 **코드로 실행**된다. 이것을 XSS 라고 한다. 8교시의 SQL 인젝션과 같은 종류의 문제다. **사용자가 보낸 글은 언제나 글자로만 다룬다.**

5단계. ⑤ 코드 읽기 (25분)

완성 예시(`checkpoints/lesson10`)를 기준으로 읽는다.

(1) `app/main.py` — 화면 파일 내려 주기

```
STATIC_DIR = Path(__file__).parent / "static"


@app.get("/", include_in_schema=False)
```

```
def index() -> FileResponse:
    """브라우저로 http://localhost:8000 에 들어오면 화면 파일을 보낸다."""
    return FileResponse(STATIC_DIR / "index.html")


# /static/app.js, /static/style.css 같은 파일을 그대로 내려 준다.
app.mount("/static", StaticFiles(directory=STATIC_DIR), name="static")
```

- `Path(__file__).parent / "static"` — "이 파일(`main.py`)이 있는 폴더 아래의 `static` 폴더".
- 지금까지의 함수는 딕셔너리(→ JSON)를 돌려줬다. 이 함수는 **파일**을 돌려준다.
- `app.mount("/static", …)` — `/static/…` 으로 온 요청은 `static` 폴더에서 파일을 찾아 준다.
- `include_in_schema=False` — `/docs` 목록에 보이지 않게 한다. API 가 아니라 화면이기 때문이다.

(2) `index.html` — 뼈대

```
<form id="memo-form">
  <input id="title" type="text" placeholder="제목 (필수, 100자까지)" maxlength="100" required>
  <textarea id="content" placeholder="내용 (1000자까지)" maxlength="1000" rows="3"></textarea>
  <button id="save-button" type="submit">저장</button>
</form>

<p id="message" role="status"></p>
<ul id="memo-list"></ul>

<script src="/static/app.js"></script>
```

- `id="…"` — 자바스크립트가 이 요소를 찾을 때 쓰는 이름표다.
- `maxlength="100" required` — 브라우저가 먼저 막아 준다. 수동 점검표 3번에서 본 안내가 이것이다.
- `<ul id="memo-list"></ul>` — **비어 있다.** 목록은 `app.js` 가 채운다.

입력 규칙이 이제 세 겹이다: 브라우저(`maxlength`, `required`) → 서버(`MemoIn`) → DB(`NOT NULL`, `VARCHAR(100)`). 브라우저의 검사는 편의일 뿐이다. `curl` 로 직접 보내면 건너뛸 수 있으므로 서버의 검사가 반드시 있어야 한다.

(3) `app.js` — 목록 불러오기

```
async function loadMemos() {
  const response = await fetch("/memos"); // GET 은 기본값이라 따로 적지 않는다
  const memos = await response.json();

  memoList.replaceChildren(); // 이전 목록을 비운다
  for (const memo of memos) {
    memoList.appendChild(makeMemoItem(memo));
  }
}
```

- `fetch("/memos")` — `curl -s http://app:8000/memos` 와 같은 일이다.
- `await` — "답이 올 때까지 기다린다". 요청은 시간이 걸리기 때문이다. `await` 를 쓰는 함수 앞에는 `async` 를 붙인다.
- `response.json()` — JSON 글자를 자바스크립트 객체로 바꾼다.
- `for (const memo of memos)` — 메모마다 한 번씩 되풀이한다.

(4) `app.js` — 메모 한 건 그리기

```
const title = document.createElement("strong");
title.textContent = memo.title; // textContent: 글자를 글자 그대로 넣는다
```

4단계의 실험에서 `<b>굵게</b>` 가 글자 그대로 보인 이유가 이 줄이다.

(5) `app.js` — 저장 버튼

```
form.addEventListener("submit", async (event) => {
```

```
  event.preventDefault(); // 폼이 페이지를 새로 고치지 않게 막는다

  const body = { title: titleInput.value, content: contentInput.value };
  // 수정 중이면 PUT, 아니면 POST
  const url = editingId === null ? "/memos" : `/memos/${editingId}`;
  const method = editingId === null ? "POST" : "PUT";

  const response = await fetch(url, {
    method,
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(body),
  });

  if (!response.ok) {
    showMessage(await readError(response));
    return;
  }
  stopEdit();
  await loadMemos(); // 목록을 다시 불러와 방금 바뀐 내용을 보여 준다
});
```

- `addEventListener("submit", …)` — "폼이 제출되면 이 함수를 실행해라".
- `editingId` — 지금 수정 중인 메모의 번호. `null` 이면 새 메모다. **저장 버튼 하나가 POST 와 PUT 을 겸한다.** 수동 점검표 5번("메모가 하나 더 생기지 않는다")이 이 줄을 확인한 것이다.
- `조건 ? A : B` — 조건이 참이면 A, 아니면 B.
- `response.ok` — 상태 코드가 2xx 이면 참이다. 422 나 404 면 거짓이다.

(6) AI 에게 묻기

■ [AI] OpenCode 입력 칸

```
파일은 고치지 말고 설명만 해 줘. app/static/app.js 에서 "수정" 버튼을 누른 순간부터 "수정 저장"을 눌러 목록이 다시 그려질 때까지, 실행되는 함수와 보내지는 HTTP 요청을 순서대로 번호를 붙여 알려 줘.
```

■ [AI] OpenCode 입력 칸

```
파일은 고치지 말고 설명만 해 줘. event.preventDefault() 를 지우면 저장 버튼을 눌렀을 때 화면에서 어떤 일이 벌어져?
```

둘째 질문의 답은 실험으로 확인한다. 그 줄을 지우고(주석으로 바꾸고) 브라우저를 새로 고친 뒤 저장을 눌러 본다. 확인했으면 Ctrl+Z 로 되돌린다.

6단계. 화면 ↔ API ↔ SQL 한 장으로 (5분)

개발자 도구의 Network 탭에서 본 것으로 채운다.

화면에서 하는 일	나가는 요청	상태 코드	실행되는 SQL
화면을 연다	`GET /memos`		`SELECT …`
저장			
수정 저장			
삭제			

`harness/WORKLOG.md` 의 10교시 기록에 붙인다.

7단계. ⑤ 기록하고 저장하기 (5분)

■ [dev] dev 컨테이너

```
git add -A
git status
git commit -m "10교시: 웹 화면 연결 (R7, N6)"
```

OpenCode 를 `/exit` 로 닫는다.

화면이 이상할 때 — 오류를 AI 에게 전달하는 법

서버 쪽 오류는 `verify.sh` 의 출력을 붙여 주면 됐다. 화면 쪽 오류는 **브라우저가 알려 주는 것**을 붙여 준다.

1. F12 → **Console(콘솔)** 탭. 빨간 글씨가 있으면 그것이 오류다. 전체를 복사한다.
2. F12 → **Network** 탭. 빨간 줄(실패한 요청)의 방식·주소·상태 코드를 적는다.
3. **무엇을 했더니, 무엇을 기대했는데, 무엇이 일어났는지**를 함께 적는다.

■ [AI] OpenCode 입력 칸

```
화면에서 문제가 있어.
- 한 일: 메모의 "수정" 버튼을 누르고 내용을 바꾼 뒤 "수정 저장"을 눌렀다.
- 기대한 것: 그 메모의 내용이 바뀐다.
- 일어난 일: 같은 제목의 메모가 하나 더 생겼다.
- Network 탭: PUT 이 아니라 POST /memos 201 이 나갔다.
- Console 탭: 오류 없음.
원인을 먼저 설명하고, 내가 "진행"이라고 하면 고쳐 줘.
```

"수정이 안 돼요"와 비교해 본다. AI 가 추측할 필요가 없다.

하네스 관점 — 이번 교시에서 본 것

- **검증에는 사람의 자리가 있다.** `pytest` 18개가 통과해도 버튼이 동작하는지는 알 수 없다. 자동 테스트가 못 보는 것은 점검표로 사람이 본다. "자동으로 확인되는 것"과 "사람이 확인해야 하는 것"의 경계를 아는 것이 중요하다.
- **AI 가 확인할 수 없는 것을 확인했다고 말하게 두지 않는다.** AI 는 브라우저를 열 수 없다. 지시서의 완료 조건에 "(사람이 확인한다)"를 따로 적어 둔 이유다.
- **금지는 검사할 수 있게 쓴다.** "`innerHTML` 을 쓰지 않는다"는 `grep` 한 줄로 확인된다. "안전하게 만든다"는 확인할 수 없다. 규칙은 확인할 수 있는 문장으로 쓴다.
- **좋은 오류 보고가 좋은 수정을 만든다.** 한 일·기대·실제·증거. 사람 개발자에게 버그를 알릴 때와 똑같다.

자주 만나는 오류

증상	원인과 해결
`http://localhost:8000` 이 `{"detail":"Not Found"}`	`GET /` 가 연결되지 않았다. `/docs` 가 아니라 `curl -s -o /dev/null -w "%{http_code}\n" http://app:8000/` 로 확인한다.
화면은 나오는데 꾸밈이 없고 버튼이 동작하지 않는다	`app.js`, `style.css` 를 못 가져왔다. Network 탭에서 두 파일이 404 인지 본다. `index.html` 의 경로(`/static/…`)와 `app.mount` 를 확인한다.
고쳤는데 화면이 그대로다	브라우저가 옛 파일을 기억하고 있다. **Ctrl+F5**(강력 새로 고침)를 누른다.
저장을 누르면 화면이 깜빡이고 주소 끝에 `?` 가 붙는다	`event.preventDefault()` 가 없다.
Console 에 `Cannot read properties of null`	자바스크립트가 찾는 `id` 가 HTML 에 없다. 이름이 서로 같은지 본다.
목록에 `[object Object]` 가 보인다	객체를 통째로 글자로 넣었다. `memo.title` 처럼 항목을 꺼내야 한다.
시각이 9시간 어긋난다	서버는 세계 표준시(UTC)로 준다. 화면에서 `new Date(…).toLocaleString("ko-KR")` 로 바꿔 보여 준다.
`RuntimeError: Directory '…/static' does not exist`	`app/static` 폴더가 없거나 경로가 틀렸다. `ls app/static` 으로 확인한다.

확인 문제

1. HTML, CSS, 자바스크립트 중 하나를 지웠을 때 각각 어떤 일이 생기는가?
2. `fetch("/memos", { method: "POST", … })` 를 같은 일을 하는 `curl` 명령으로 바꿔 쓰시오.
3. 화면을 만들면서 `/memos` API 를 고칠 필요가 없었던 이유는 무엇인가?
4. `pytest` 가 모두 통과했는데 화면의 저장 버튼이 동작하지 않을 수 있는가? 이유를 쓰시오.
5. 제목에 `<b>굵게</b>` 를 넣었을 때 `textContent` 와 `innerHTML` 은 각각 어떻게 보여 주는가? 왜 한쪽이 위험한가?
6. 입력 규칙이 브라우저·서버·DB 세 곳에 있다. 브라우저의 검사만 있으면 안 되는 이유를 `curl` 을 예로 들어 설명하시오.
7. 저장 버튼 하나가 새 메모 저장과 수정 저장을 겸한다. 코드는 둘을 무엇으로 구분하는가?
8. "수정이 안 돼요"라는 보고를 AI 가 바로 고칠 수 있는 보고로 고쳐 쓰시오. (한 일·기대·실제·증거)

11교시. 자동 테스트와 오류 수정

시간	2시간
실습 결과	테스트 통과 (버그를 테스트로 붙잡고, AI 가 고치고, 다시 검증한다)
시작 상태	10교시를 마친 상태 (화면까지 완성되어 커밋되어 있다)

10교시 결과가 없으면 **[dev]** `sh scripts/checkpoint.sh 10` 으로 가져온다.

수업의 흐름에서 이번 교시의 위치

```
① 요구사항 작성 → ② AI 에게 지시 → ③ 하네스로 제어 → ④ 실행·테스트 → ⑤ 수정·재검증·저장
                                                                  └ 이번 교시의 주인공 ┘
```

지금까지는 "새 기능을 만든다 → 검증한다"였다. 이번에는 순서가 다르다. **이미 있는 버그를 찾는다 → 테스트로 붙잡는다 → AI 가 고친다 → 다시 검증한다.**

학습 목표

1. `pytest` 의 실패 출력을 읽고 "무엇을 기대했는데 무엇이 왔는지" 찾는다.
2. 버그를 재현하고, 그 버그를 잡는 테스트를 **수정보다 먼저** 쓴다.
3. 실패하는 테스트를 근거로 AI 에게 수정을 지시한다.
4. AI 가 테스트를 고치지 않고 코드를 고쳤는지 확인한다.
5. 요구사항과 테스트의 대응표를 완성한다.

1. 개념

테스트를 먼저 쓴다 — 빨강, 초록

버그를 고치는 순서가 중요하다.

```
나쁜 순서                          좋은 순서
──────────────────────────         ──────────────────────────────────
1. AI 에게 "고쳐 줘"               1. 버그를 직접 재현한다
2. AI 가 "고쳤습니다"              2. 버그를 잡는 테스트를 쓴다
3. 눈으로 한 번 확인               3. 테스트가 실패하는 것을 확인한다   ← 빨강
                                   4. AI 에게 고치게 한다
                                   5. 테스트가 통과하는 것을 확인한다   ← 초록
```

테스트를 먼저 쓰면 세 가지를 얻는다.

- **고쳐졌다는 증거.** 실패하던 테스트가 통과로 바뀐다. "고쳤습니다"라는 말이 아니라 결과다.
- **다시 생기지 않는다는 보장.** 나중에 누가(사람이든 AI 든) 이 버그를 되살리면 테스트가 실패한다.
- **AI 에게 주는 정확한 목표.** "이 테스트가 통과하게 해 줘"는 오해할 수 없는 지시다.

3단계 "빨강 확인"을 건너뛰면 안 된다. 처음부터 통과하는 테스트는 버그를 잡지 못하는 테스트다.

pytest 실패 출력 읽는 법

```
    def test_create_memo_blank_title(client):
        response = client.post("/memos", json={"title": "   ", "content": "공백 제목"})
>       assert response.status_code == 422
E       assert 201 == 422
E        +  where 201 = <Response [201 Created]>.status_code

tests/test_memos.py:120: AssertionError
========================== short test summary info ==========================
FAILED tests/test_memos.py::test_create_memo_blank_title - assert 201 == 422
```

표시	뜻
`>` 로 시작하는 줄	실패한 줄
`E` 로 시작하는 줄	이유. `assert 201 == 422` → **실제로 온 값 == 기대한 값**
`tests/test_memos.py:120`	파일과 줄 번호
`FAILED …::test_create_memo_blank_title`	실패한 테스트의 이름

읽는 순서: **맨 아래 요약부터** 본다. 어떤 테스트가 몇 개 실패했는지 알고, 위로 올라가 `E` 줄을 읽는다.

따라 하기

1단계. 버그 찾기 — 요구사항과 대조 (15분)

`harness/REQUIREMENTS.md` 의 R6 을 다시 읽는다. "11교시에 추가"라고 표시된 두 줄이 있다.

- `title` 이 공백뿐(`"  "`)이면 422
- `title` 의 앞뒤 공백은 잘라 내고 저장한다

지금 코드는 이 두 줄을 지키고 있는가? **직접 보낸다.**

■ [dev] dev 컨테이너

```
curl -s -X POST http://app:8000/memos -H "Content-Type: application/json" -d '{"title": "   ", "content": "공백 제목"}'
```

```
{"id":6,"title":"   ","content":"공백 제목","created_at":"2026-11-10T05:50:36.445321Z","updated_at":"2026-11-10T05:50:36.445321Z"}
```

(`id` 와 시각은 사람마다 다르다.)

저장되었다. 요구사항은 422 를 말하는데 201 이 왔다. 버그다.

브라우저에서 http://localhost:8000 을 열어 본다. 제목이 보이지 않는 메모가 목록에 있다. 화면에서도 제목 칸에 띄어쓰기만 넣고 저장해 본다. 브라우저의 `required` 는 띄어쓰기를 "입력한 것"으로 보고 통과시킨다.

왜 이런 버그가 있었는가. `Field(min_length=1)` 은 글자 **수**만 센다. `"  "` 는 세 글자다. 6교시의 테스트는 빈 문자열(`""`)만 확인했다. **테스트에 없는 경우는 확인되지 않은 채 남는다.** 테스트 18개가 모두 통과한다는 것은 "버그가 없다"가 아니라 "테스트에 적은 18가지는 맞다"는 뜻이다.

방금 만든 공백 메모는 화면에서 지운다.

2단계. 버그를 테스트로 붙잡기 — 빨강 (20분)

이번 테스트는 **AI 에게 시키지 않고 내가 직접 쓴다.** 무엇이 맞는 동작인지는 사람이 정하는 것이기 때문이다.

먼저 깨끗한 상태인지 확인한다.

■ [dev] dev 컨테이너

```
git status
sh scripts/verify.sh
```

VS Code 에서 `tests/test_memos.py` 를 연다. 입력 규칙(R6) 테스트들이 모여 있는 곳 아래에 다음 두 테스트를 추가하고 저장한다. 위아래 함수와 두 줄씩 띄운다.

```
def test_create_memo_blank_title(client):
    # 11교시에 추가한 테스트: 공백만 있는 제목은 받지 않는다.
    response = client.post("/memos", json={"title": "   ", "content": "공백 제목"})
    assert response.status_code == 422


def test_title_is_trimmed(client):
    # 11교시에 추가한 테스트: 제목 앞뒤 공백은 잘라서 저장한다.
    response = client.post("/memos", json={"title": "  장보기  ", "content": ""})
    assert response.status_code == 201
    assert response.json()["title"] == "장보기"
```

요구사항의 두 줄과 나란히 놓고 읽는다. **문장 하나가 테스트 하나다.**

실행한다.

■ [dev] dev 컨테이너

```
pytest -q
```

완성 예시 기준의 결과(줄 번호와 개수는 내 코드에 따라 다르다):

```
.................FF.                                                [100%]
================================== FAILURES ===================================
_________________________ test_create_memo_blank_title _________________________
(가운데 생략)
>       assert response.status_code == 422
E       assert 201 == 422
E        +  where 201 = <Response [201 Created]>.status_code
(가운데 생략)
____________________________ test_title_is_trimmed _____________________________
(가운데 생략)
E       AssertionError: assert '  장보기  ' == '장보기'
(가운데 생략)
=========================== short test summary info ===========================
FAILED tests/test_memos.py::test_create_memo_blank_title - assert 201 == 422
FAILED tests/test_memos.py::test_title_is_trimmed - AssertionError: assert ' ...
2 failed, 18 passed in 2.27s
```

실패가 정상이다. 확인할 것:

■ 실패한 테스트가 **방금 추가한 두 개뿐**이다. (`2 failed`)

■ 나머지는 그대로 통과한다. (`18 passed`)

■ 실패 이유가 1단계에서 본 버그와 같다. (`assert 201 == 422`)

만약 방금 추가한 테스트가 **통과**한다면, 버그가 없거나 테스트가 버그를 잡지 못하는 것이다. 테스트를 다시 본다.

이 상태로 커밋할 수 있을까? 해 본다.

■ [dev] dev 컨테이너

```
git add tests/test_memos.py
git commit -m "실험: 실패하는 테스트만 커밋"
```

```
[pre-commit] 저장하기 전에 검증을 실행합니다...
(가운데 생략)
2 failed, 18 passed in 2.27s
```

```
[pre-commit] 검증 실패: 커밋을 취소했습니다.
[pre-commit] 위의 오류를 고친 뒤 다시 커밋하세요.
```

5교시에 세운 장치가 막는다. 버그를 고치기 전에는 저장할 수 없다.

3단계. ② AI 에게 수정 지시 (15분)

harness/tasks/11_blank_title_bug.md 를 읽는다. 지금까지의 지시서와 다른 점:

- **2. 현재 상황** — 사람이 직접 확인한 사실이 적혀 있다. 무엇을 보냈더니 무엇이 왔는지, 어떤 테스트가 실패하는지. AI 가 추측할 것이 없다.
- **4. 범위** — 고칠 파일이 app/schemas.py **하나**다. tests/ 는 폴더 전체가 금지다.
- **5. 하지 말 것** — "app/main.py 의 함수마다 검사 코드를 넣지 않는다." 만들기와 고치기 두 군데에 같은 검사를 복사하는 방법을 막는다.
- **7. 진행 방법** — 계획 전에 **원인 설명**을 먼저 요구한다.

■ **[dev] dev 컨테이너**

```
opencode
```

■ **[AI] OpenCode 입력 칸**

```
harness/tasks/11_blank_title_bug.md 를 읽고 그 지시대로 작업해 줘. 먼저 원인을 설명하고, 고칠 방법을 말한 뒤 내 승인을 기다려.
```

원인 설명 검토. AI 의 설명이 1단계에서 내가 이해한 것과 같은가?

■ min_length=1 이 글자 수만 센다는 점을 짚었는가?

■ " " 가 세 글자라서 통과했다고 설명했는가?

계획 검토 체크리스트

■ 고치는 파일이 app/schemas.py 하나인가?

■ tests/ 를 고치겠다는 말이 **없는가?**

■ 만들기와 고치기에 **똑같이** 적용되는 방법인가? (MemoIn 한 곳을 고친다)

■ 앞뒤 공백을 **잘라서 저장**하는 것까지 포함되어 있는가? (요구사항의 둘째 줄)

맞으면 진행.

4단계. ④⑤ 재검증 — 초록 (15분)

■ **[dev] dev 컨테이너** (다른 셸)

```
sh scripts/verify.sh
```

완성 예시의 결과:

```
....................                                                [100%]
20 passed in 2.20s
```

■ 2 failed 가 사라졌다. 실패하던 두 테스트가 통과한다.

■ 원래 통과하던 18개도 여전히 통과한다.

AI 가 무엇을 고쳐서 통과시켰는가 — 가장 중요한 확인

■ **[dev] dev 컨테이너**

```
git diff --stat
```

■ app/schemas.py 가 있다.

■ tests/test_memos.py 의 변경은 **2단계에서 내가 추가한 것뿐**이다.

둘째 항목은 직접 본다.

■ [dev] dev 컨테이너

```
git diff tests/
```

초록색(+) 줄이 내가 2단계에서 붙여 넣은 그대로인가? 줄이 지워졌거나(-), `assert` 의 숫자가 바뀌었거나, `skip` 이라는 낱말이 생겼다면 **AI 가 테스트를 고쳐서 통과시킨 것**이다. 이 경우 결과를 받아들이지 않는다.

■ [AI] OpenCode 입력 칸

```
git diff tests/ 를 보니 내가 쓴 테스트가 바뀌었어. 지시서 4번에서 tests/ 는 고치면 안 된다고 했어. tests/ 를 내가 쓴 그대로 되돌리고, ap
  p/schemas.py 만 고쳐서 통과시켜 줘.
```

직접 불러 보기

■ [dev] dev 컨테이너

```
curl -s -X POST http://app:8000/memos -H "Content-Type: application/json" -d '{"title": "   ", "content": "공백 제목"}'
```

```
{"detail":[{"type":"value_error","loc":["body","title"],"msg":"Value error, 제목은 공백만으로 채울 수 없습니다.","input":"   ","ctx":{"e
  rror":{}}}]}
```

(`msg` 의 문구는 AI 가 쓴 것에 따라 다르다.)

■ [dev] dev 컨테이너

```
curl -s -o /dev/null -w "%{http_code}\n" -X POST http://app:8000/memos -H "Content-Type: application/json" -d '{"title": "   ", "content
  ": "공백 제목"}'
```

```
422
```

1단계에서 201 이던 것이 422 가 되었다.

고치기에도 적용되는가? 지시서가 "입력 규칙은 한 곳에 둔다"고 한 이유를 확인한다. 있는 메모 번호로 해 본다.

■ [dev] dev 컨테이너

```
curl -s -o /dev/null -w "%{http_code}\n" -X PUT http://app:8000/memos/1 -H "Content-Type: application/json" -d '{"title": "   ", "conten
  t": ""}'
```

`422` 가 나와야 한다. `200` 이 나오면 만들기에만 검사를 넣은 것이다.

브라우저 화면에서도 제목에 띄어쓰기만 넣고 저장해 본다. 저장되지 않고 안내 문구가 나오는가?

5단계. ⑤ 코드 읽기 (15분)

완성 예시(`checkpoints/lesson11/app/schemas.py`)를 기준으로 읽는다. 바뀐 것은 이 부분뿐이다.

```
class MemoIn(BaseModel):
    title: str = Field(min_length=1, max_length=100)
    content: str = Field(default="", max_length=1000)

    @field_validator("title")
    @classmethod
    def title_must_not_be_blank(cls, value: str) -> str:
        """11교시에 추가: 공백만 있는 제목("   ")은 받지 않는다."""
        stripped = value.strip()  # 앞뒤 공백을 잘라 낸다
        if not stripped:
            raise ValueError("제목은 공백만으로 채울 수 없습니다.")
        return stripped  # 잘라 낸 값으로 저장한다
```

- `@field_validator("title")` — "`title` 값이 들어올 때 이 함수를 거치게 해라".
- `value.strip()` — 글자의 앞뒤 공백을 잘라 낸다. `" 장보기 "` → `"장보기"`, `"  "` → `""`.
- `if not stripped:` — 빈 문자열이면 참이다.

- `raise ValueError(…)` — 여기서 오류를 일으키면 FastAPI 가 422 로 바꿔 돌려준다.
- `return stripped` — **돌려준 값이 저장된다.** 그래서 앞뒤 공백이 잘린 제목이 저장된다. 요구사항의 둘째 줄이 이 한 줄이다.

`MemoIn` 은 만들기(`create_memo`)와 고치기(`update_memo`)가 함께 쓴다. **한 곳을 고쳤더니 두 API 가 함께 고쳐졌다.** 9교시에 "같은 클래스를 쓰면 무엇이 좋은가"를 물었던 답이다.

■ [AI] OpenCode 입력 칸

```
파일은 고치지 말고 설명만 해 줘. app/schemas.py 의 validator 에서 return stripped 를 return value 로 바꾸면 어떤 테스트가 실패해? 이유도 알려 줘.
```

답을 들은 뒤 **직접 바꿔서 실행해 본다.** AI 가 말한 테스트가 실패하는가? 확인하고 Ctrl+Z 로 되돌린다.

6단계. 요구사항 ↔ 테스트 대응표 완성하기 (15분)

모든 요구사항에 테스트가 있는지 마지막으로 점검한다. 테스트 이름을 자세히 출력한다.

■ [dev] dev 컨테이너

```
pytest -v
```

완성 예시의 결과:

```
tests/test_health.py::test_health PASSED                              [  5%]
tests/test_health.py::test_hello_default PASSED                       [ 10%]
tests/test_health.py::test_hello_with_name PASSED                     [ 15%]
tests/test_memos.py::test_create_memo PASSED                          [ 20%]
tests/test_memos.py::test_create_memo_without_content PASSED          [ 25%]
tests/test_memos.py::test_list_memos_empty PASSED                     [ 30%]
tests/test_memos.py::test_list_memos_newest_first PASSED              [ 35%]
tests/test_memos.py::test_get_memo PASSED                             [ 40%]
tests/test_memos.py::test_get_memo_not_found PASSED                   [ 45%]
tests/test_memos.py::test_update_memo PASSED                          [ 50%]
tests/test_memos.py::test_update_memo_not_found PASSED                [ 55%]
tests/test_memos.py::test_update_memo_empty_title PASSED              [ 60%]
tests/test_memos.py::test_delete_memo PASSED                          [ 65%]
tests/test_memos.py::test_delete_memo_not_found PASSED                [ 70%]
tests/test_memos.py::test_create_memo_empty_title PASSED              [ 75%]
tests/test_memos.py::test_create_memo_title_too_long PASSED           [ 80%]
tests/test_memos.py::test_create_memo_content_too_long PASSED         [ 85%]
tests/test_memos.py::test_create_memo_blank_title PASSED              [ 90%]
tests/test_memos.py::test_title_is_trimmed PASSED                     [ 95%]
tests/test_memos.py::test_index_page PASSED                           [100%]

============================== 20 passed in 2.02s ==============================
```

`harness/VERIFY_CHECKLIST.md` 3단계의 대응표를 펴고, **내** 테스트 이름으로 채운다. `harness/REQUIREMENTS.md` 의 R1~R7 을 한 문장씩 짚어 가며, 그 문장을 확인하는 테스트가 있는지 본다.

빈칸이 있으면 그것이 다음 버그가 숨어 있을 자리다. 예를 들어:

- R6 "content 가 1000자를 넘으면 422" — 테스트가 있는가?
- R4 "고칠 때도 입력 규칙이 적용된다" — 테스트가 있는가?
- 9교시 5단계 (5)에서 추가한 "한 건만 고쳐진다" 테스트가 있는가?

빠진 것은 2~4단계와 같은 방법으로 채운다. 테스트를 먼저 쓰고(이번에는 처음부터 통과할 수도 있다. 그러면 버그가 없다는 뜻이다), 실패하면 AI 에게 고치게 한다.

7단계. ⑤ 기록하고 저장하기 (5분)

harness/WORKLOG.md 에 11교시 기록을 쓴다. 이번에는 "빨강 → 초록"의 과정을 적는다. 어떤 테스트가, 무엇 때문에 실패했고, 무엇을 고쳐서 통과했는가.

■ [dev] dev 컨테이너

```
git add -A
git status
git commit -m "11교시: 공백 제목 버그 수정 (R6)"
```

이번에는 pre-commit 이 통과시킨다. 2단계에서 막혔던 커밋이 이제 된다.

OpenCode 를 /exit 로 닫는다.

하네스 관점 — 이번 교시에서 본 것

```
버그 발견 ──▶ 테스트 작성(사람) ──▶ 빨강 확인 ──▶ AI 수정 ──▶ 초록 확인 ──▶ 커밋
                  │                    │             │            │
                  │                    │             │            └ pre-commit 이 통과시킨다
                  │                    │             └ tests/ 는 건드리지 못한다 (지시서 + git diff 로 확인)
                  │                    └ 이 상태로는 커밋이 막힌다 (pre-commit)
                  └ "무엇이 맞는 동작인가"는 사람이 정한다
```

- **기준은 사람이, 구현은 AI 가.** 테스트(기준)를 사람이 쓰고 AI 는 그 기준을 만족시키는 코드만 썼다. 기준과 구현을 같은 쪽이 다 쓰면, 기준을 구현에 맞춰 버릴 수 있다.
- **"테스트 통과"의 뜻을 정확히 안다.** 테스트 18개가 통과하던 때에도 버그는 있었다. 통과는 "적은 것이 맞다"일 뿐이다. 그래서 요구사항과 테스트를 한 문장씩 대조한다.
- **하네스는 자란다.** 버그 하나를 고칠 때마다 테스트가 늘고, 다음에는 같은 버그가 통과할 수 없게 된다. 프로젝트가 진행될수록 검증이 촘촘해진다.

자주 만나는 오류

증상	원인과 해결
2단계에서 추가한 테스트가 처음부터 통과한다	AI 가 앞 교시에서 이미 공백 검사를 넣었을 수 있다. curl 로 1단계를 다시 해 본다. 422 가 온다면 버그가 없는 것이다. 테스트는 그대로 두고 6단계로 간다.
IndentationError 또는 테스트가 수집되지 않는다	붙여 넣을 때 들여쓰기가 틀어졌다. def 는 줄 맨 앞, 그 아래는 4칸이다.
고친 뒤 다른 테스트가 실패한다	수정이 다른 동작을 깨뜨렸다. 테스트가 제 일을 한 것이다. 실패 출력을 AI 에게 붙여 주고, 원인부터 설명하게 한다.
AI 가 app/main.py 에 if not memo.title.strip(): 을 두 군데 넣었다	지시서 5번 위반이다. 동작은 하지만 규칙이 두 곳으로 흩어진다. app/schemas.py 한 곳으로 옮기게 한다.
커밋이 막힌다	[pre-commit] 아래의 출력을 읽는다. 아직 실패하는 테스트가 있다.
화면에서는 여전히 공백 제목이 저장되는 것 같다	Ctrl+F5 로 새로 고친다. Network 탭에서 POST 의 상태 코드가 422 인지 본다.

확인 문제

1. `assert 201 == 422` 에서 어느 쪽이 실제로 온 값이고 어느 쪽이 기대한 값인가?
2. 버그를 고치기 **전에** 테스트를 쓰고 실패를 확인하는 이유를 두 가지 쓰시오.
3. 2단계에서 추가한 테스트가 처음부터 통과한다면 어떤 두 가지 가능성이 있는가?
4. 테스트 18개가 모두 통과하는데도 공백 제목 버그가 있었던 이유를 설명하시오.
5. 이번 교시에 테스트를 AI 가 아니라 사람이 쓴 이유는 무엇인가?
6. AI 가 테스트를 고쳐서 통과시켰는지 어떤 명령으로, 무엇을 보고 판단하는가?
7. 공백 검사를 `app/main.py` 의 두 함수에 넣는 것과 `app/schemas.py` 의 `MemoIn` 에 넣는 것의 차이를 설명하시오.
8. `return stripped` 한 줄이 요구사항의 어느 문장을 구현하는가?

12교시. GitHub 저장 및 발표

시간	2시간
실습 결과	최종 프로젝트 (내 GitHub 저장소 + 발표)
시작 상태	11교시를 마친 상태 (모든 테스트가 통과하고 커밋되어 있다)

11교시 결과가 없으면 **[dev]** `sh scripts/checkpoint.sh 11` 로 가져온다.

수업의 흐름에서 이번 교시의 위치

```
① 요구사항 작성 → ② AI 에게 지시 → ③ 하네스로 제어 → ④ 실행·테스트 → ⑤ 수정·재검증·저장
                                                                    └ Git 에 저장 → GitHub 로
```

학습 목표

1. 커밋 기록을 읽고, 프로젝트가 만들어진 과정을 설명한다.
2. 올리면 안 되는 것(`.env`, DB 데이터)이 빠져 있는지 확인한다.
3. 내 GitHub 저장소를 만들어 결과물을 올린다(push).
4. 저장소를 새로 내려받아, 다른 사람이 따라 할 수 있는지 확인한다.
5. 작업 기록을 바탕으로 "AI 에게 맡기고 내가 통제한 과정"을 발표한다.

1. 개념

커밋과 푸시

	하는 일	어디에 저장되는가
커밋(commit)	지금 상태를 기록으로 남긴다	내 PC (`C:\harness-crud-lab\.git`)
푸시(push)	내 PC 의 커밋을 인터넷의 저장소로 보낸다	GitHub

3교시부터 해 온 커밋은 모두 내 PC 에만 있다. PC 가 고장 나면 함께 사라진다. 푸시하면 GitHub 에 사본이 생긴다.

원격 저장소

`git clone` 으로 내려받은 저장소는 원래 주소를 `origin` 이라는 이름으로 기억한다. 지금 `origin` 은 강사의 저장소(`sinaboro/harness-crud-lab`)다. 거기에는 올릴 권한이 없다.

그래서 **내 저장소를 새로 만들고**, `origin` 이 그곳을 가리키게 바꾼다.

```
지금:    origin → github.com/sinaboro/harness-crud-lab      (강사 저장소. 올릴 수 없다)

바꾼 뒤: upstream → github.com/sinaboro/harness-crud-lab    (강사 저장소. 이름만 바꿔 남겨 둔다)
         origin   → github.com/내아이디/my-memo-crud        (내 저장소. 여기에 올린다)
```

푸시는 어디에서 하는가

커밋은 dev 컨테이너에서 했다. 푸시는 **윈도우 PowerShell** 에서 한다. GitHub 로그인 창이 윈도우에서 뜨고, 로그인 정보도 윈도우에 저장되기 때문이다. 컨테이너에는 내 GitHub 로그인 정보가 없다. AI 에이전트가 그 정보를 쓸 수도 없다.

따라 하기

1단계. 마지막 검증 (10분)

올리기 전에 처음부터 끝까지 다시 확인한다.

■ [dev] dev 컨테이너

```
git status
sh scripts/verify.sh
```

■ `nothing to commit, working tree clean`

■ "검증 통과"

`harness/VERIFY_CHECKLIST.md` 의 5단계(직접 실행해 본다)를 한 번 더 한다. 브라우저에서 http://localhost:8000 을 열어 쓰기·수정·삭제를 한다.

2단계. 커밋 기록 읽기 (15분)

■ [dev] dev 컨테이너

```
git log --oneline
```

맨 위가 가장 최근이다. 아래에서 위로 읽으면 프로젝트가 자란 순서다. 내 기록은 대략 이런 모양일 것이다. (맨 아래 몇 줄은 강사가 만든 출발점이다.)

```
(번호) 11교시: 공백 제목 버그 수정 (R6)
(번호) 10교시: 웹 화면 연결 (R7, N6)
(번호) 9교시: 메모 수정·삭제 구현, CRUD 완성 (R4, R5)
(번호) 8교시: 저장소를 PostgreSQL 로 교체 (N1, N2, N4)
(번호) 6교시: 메모 만들기·보기 초안 (R1, R2, R3, R6)
(번호) 5교시: 하네스 세우기 (AGENTS.md, 권한 규칙)
(번호) 4교시: 메모 만들기·목록 보기(메모리 저장) 추가
(번호) 3교시: /hello 주소와 테스트 추가
```

커밋 하나는 검증을 통과한 지점 하나다. 5교시 이후의 모든 커밋은 `pre-commit` 을 통과했다.

교시마다 무엇이 바뀌었는지 본다. 8교시 커밋을 예로 든다.

■ [dev] dev 컨테이너

```
git show --stat (8교시 커밋의 번호)
```

그 커밋에서 바뀐 파일 목록이 나온다. `tests/test_memos.py` 가 **없는지** 확인한다. 8교시의 조건("테스트는 한 글자도 바꾸지 않는다")이 기록으로 남아 있는 것이다.

프로젝트가 얼마나 자랐는지도 본다.

■ [dev] dev 컨테이너

```
git diff --stat (3교시 커밋의 번호) HEAD -- app tests
```

3교시부터 지금까지 `app` 과 `tests` 에서 바뀐 양이 나온다. 이 중 내가 손으로 친 줄은 얼마나 되는가? 나머지는 AI 가 썼고, **나는 그것을 읽고 검증했다.**

3단계. 올리면 안 되는 것 확인하기 (10분)

GitHub 에 올린 것은 누구나 볼 수 있다. 올리고 나서 지워도 기록에 남는다. **올리기 전에** 확인한다.

(1) Git 이 관리하는 파일 목록에 비밀이 없는가

■ [dev] dev 컨테이너

```
git ls-files ¦ grep -i "env"
```

```
.env.example
```

`.env.example` **한 줄만** 나와야 한다. `.env` 가 보이면 멈추고 강사에게 알린다.

■ [dev] dev 컨테이너

```
git ls-files ¦ grep "^data/"
```

아무것도 출력되지 않아야 한다. DB 데이터 폴더는 올리지 않는다.

(2) 커밋 기록 전체에 키가 들어간 적이 없는가

■ [dev] dev 컨테이너

```
git log -p --all ¦ grep -c "sk-or-v1-[A-Za-z0-9]\{20,\}"
```

`0` 이 나와야 한다. API 키처럼 생긴 글자가 지금까지의 어떤 커밋에도 없다는 뜻이다.

`0` 이 아니면 **푸시하지 않는다.** 그 키는 이미 기록에 남았으므로 OpenRouter 의 Keys 화면에서 그 키를 지우고 새로 만든다. 그리고 강사와 함께 기록을 정리한다.

(3) 코드에 비밀번호가 없는가

■ [dev] dev 컨테이너

```
grep -rn "memo1234" app/ tests/
```

아무것도 출력되지 않아야 한다. (`docker-compose.yml` 과 `.env.example` 의 `memo1234` 는 실습용 기본값이다. 실제 서비스에서는 이런 값도 올리지 않는다.)

4단계. 내 프로젝트의 README 쓰기 (20분)

지금 `README.md` 는 이 **과정**의 안내문이다. 내 저장소의 첫 화면에는 내 **프로젝트**의 소개가 있어야 한다.

과정 안내문은 이름을 바꿔 남겨 둔다.

■ [dev] dev 컨테이너

```
git mv README.md COURSE.md
```

VS Code 에서 새 `README.md` 를 만든다. 아래 틀을 채운다. **이 문서는 AI 에게 시키지 않고 내가 쓴다.** 3일 동안 한 일을 내 말로 정리하는 것이 이 과정의 마지막 학습이다.

```
# 메모 CRUD – AI 에이전트와 함께 만든 첫 웹 서비스

(한두 문장 소개: 무엇을 하는 서비스인가)

## 기능
- (R1~R7 을 내 말로)

## 실행 방법
1. Docker Desktop 을 설치한다.
2. 이 저장소를 내려받는다.
3. `.env.example` 을 복사해 `.env` 를 만든다.
4. `docker compose up -d --build`
5. 브라우저에서 http://localhost:8000

## 만든 방법 – 하네스 엔지니어링
코드의 대부분은 AI 에이전트(OpenCode + OpenRouter)가 썼고, 나는 아래 장치로 그 작업을 통제했다.

¦ 요소 ¦ 파일 ¦ 한 일 ¦
¦---¦---¦---¦
```

```
¦ 규칙 ¦ AGENTS.md ¦ (내 말로) ¦
¦ 권한 ¦ opencode.jsonc ¦ (내 말로) ¦
¦ 검증 ¦ scripts/verify.sh, tests/, .githooks/pre-commit ¦ (내 말로) ¦

## API
¦ 하는 일 ¦ 방식·주소 ¦ 성공 ¦ 실패 ¦
¦---¦---¦---¦---¦
(9교시 6단계에서 만든 표)

## 테스트
(테스트 개수, 실행 방법, 요구사항과 테스트의 대응)

## 배운 것
(harness/WORKLOG.md 에서 세 가지를 골라)

## 과정 안내
이 프로젝트는 harness-crud-lab 과정의 결과물이다. 과정 안내는 COURSE.md 에 있다.
```

초안을 다 쓴 뒤 AI 에게 **검토만** 시킨다.

■ [dev] dev 컨테이너

```
python scripts/ask_ai.py --file README.md "이 README 를 처음 보는 사람 입장에서 읽고, 실행 방법에서 빠진 단계나 모호한 문장이 있으면 지
  적만 해 줘. 고쳐 쓰지는 마."
```

지적 중 맞는 것만 내가 고친다.

커밋한다.

■ [dev] dev 컨테이너

```
sh scripts/verify.sh
git add -A
git status
git commit -m "12교시: 프로젝트 README 작성"
```

5단계. GitHub 에 올리기 (20분)

(1) 내 저장소 만들기

1. 브라우저에서 https://github.com 에 로그인한다.
2. 오른쪽 위 **+** → **New repository**.
3. Repository name: `my-memo-crud` (원하는 이름)
4. **Public** 을 고른다.
5. "Add a README file", ".gitignore", "license" 는 **모두 선택하지 않는다.** (빈 저장소여야 한다)
6. **Create repository** 를 누른다.
7. 다음 화면에 나오는 주소(`https://github.com/내아이디/my-memo-crud.git`)를 복사한다.

(2) 원격 저장소 바꾸기

여기부터는 **윈도우 PowerShell** 에서 한다.

■ [PS] 윈도우 PowerShell

```
cd C:\harness-crud-lab
git remote -v
```

`origin` 이 `sinaboro/harness-crud-lab` 을 가리키고 있다.

■ [PS] 윈도우 PowerShell

```
git remote rename origin upstream
git remote add origin https://github.com/내아이디/my-memo-crud.git
git remote -v
```

내아이디 자리에 내 GitHub 아이디를 넣는다. 마지막 명령의 결과에 origin 이 내 저장소로, upstream 이 강사 저장소로 나오면 된다.

(3) 올리기

■ [PS] 윈도우 PowerShell

```
git push -u origin main
```

처음 올릴 때 GitHub 로그인 창이 뜬다. 브라우저로 로그인한다. 올라간 커밋 수와 함께 main -> main 이 나오면 성공이다.

- -u origin main — "앞으로 이 브랜치는 origin 의 main 과 짝이다"라고 기억시킨다. 다음부터는 git push 만 쳐도 된다.

(4) 확인

브라우저에서 내 저장소(https://github.com/내아이디/my-memo-crud)를 연다.

■ 4단계에서 쓴 README 가 첫 화면에 보인다.

■ 파일 목록에 .env 가 **없다.** .env.example 은 있다.

■ 파일 목록에 data 폴더가 **없다.**

■ 커밋 수를 누르면 교시별 커밋이 보인다.

6단계. 다른 사람이 따라 할 수 있는가 (15분)

내 PC 에서 되는 것과 **다른 사람의 PC 에서 되는 것**은 다르다. 내 저장소를 다른 폴더에 새로 내려받아 README 대로 해 본다.

지금 켜져 있는 컨테이너를 먼저 내린다. (같은 8000 번 포트를 쓰기 때문이다)

■ [PS] 윈도우 PowerShell

```
cd C:\harness-crud-lab
docker compose down
```

새로 내려받아 켠다.

■ [PS] 윈도우 PowerShell

```
git clone https://github.com/내아이디/my-memo-crud.git C:\memo-check
cd C:\memo-check
Copy-Item .env.example .env
docker compose up -d --build
docker compose ps
```

.env 의 OPENROUTER_API_KEY 는 비워 둔다. 메모장을 실행하는 데는 AI 키가 필요 없다.

■ 브라우저에서 http://localhost:8000 이 열린다.

■ 메모를 쓰고, 고치고, 지울 수 있다.

■ 메모 목록이 **비어 있는 상태**에서 시작한다. (DB 데이터는 올리지 않았으므로 새 PC 에는 새 DB 가 만들어진다)

테스트도 돌려 본다.

■ [PS] 윈도우 PowerShell

```
docker compose exec dev sh scripts/verify.sh
```

■ "검증 통과"

API 키 없이도, 내 PC 의 다른 폴더에서, 저장소에 올라간 것만으로 서비스가 뜨고 테스트가 통과한다. 이것이 "재현할 수 있다"는 뜻이다.

README 의 실행 방법대로 했는데 빠진 단계가 있었다면, 돌아가서 README 를 고치고 다시 올린다.

뒷정리를 하고 원래 폴더로 돌아온다.

■ [PS] 윈도우 PowerShell

```
docker compose down
cd C:\harness-crud-lab
Remove-Item -Recurse -Force C:\memo-check
docker compose up -d
```

> `C:\memo-check` 에서는 `pre-commit` 훅이 꺼져 있다. `core.hooksPath` 설정은 Git 에 올라가지 않기 때문이다(5교시 5단계의 참고). 새로 내려받은 저장소에서 작업을 이어 가려면 `git config core.hooksPath .githooks` 를 다시 실행한다.

7단계. 발표 준비와 발표 (30분)

발표는 **5분**이다. "무엇을 만들었는가"보다 **"어떻게 통제했는가"** 를 말한다. `harness/WORKLOG.md` 를 펴 놓고 준비한다.

발표 순서

순서	시간	내용	보여 줄 것
1	30초	무엇을 만들었는가	브라우저에서 쓰기·수정·삭제 시연
2	1분	AI 에게 어떻게 시켰는가	작업 지시서 하나(`harness/tasks/…`)의 "범위"와 "하지 말 것"
3	1분 30초	하네스가 실제로 막거나 잡아낸 일 **하나**	`WORKLOG.md` 의 기록. 권한이 막은 명령, 계획에서 빼게 한 것, 테스트가 잡은 실수 중에서
4	1분	내가 이해한 코드 **한 군데**	코드 대여섯 줄을 화면에 띄우고 한 줄씩 설명
5	30초	검증	`sh scripts/verify.sh` 실행, `git log --oneline`
6	30초	배운 것 한 가지, 아직 모르는 것 한 가지	—

3번이 발표의 중심이다. 좋은 예:

- "AI 의 계획에 삭제 API 가 들어 있어서 지시서 4번을 가리켜 빼게 했습니다."
- "8교시에 `git diff --stat` 을 보니 테스트 파일이 바뀌어 있었습니다. 되돌리게 하고 `app/` 만 고치게 했습니다."
- "9교시에 `WHERE` 를 일부러 지워 봤는데 테스트가 잡지 못했습니다. 메모 두 건으로 확인하는 테스트를 추가했습니다."
- "AI 가 `pip install` 을 실행하려 해서 거절했습니다."

4번에서 고를 만한 코드

- `conn.execute("… WHERE id = %s", (memo_id,))` — 자리표시자와 그 이유
- `if row is None: raise not_found()` — DB 는 "없다"를 오류로 알리지 않는다
- `@field_validator("title")` — 요구사항 한 줄이 코드가 된 곳
- `title.textContent = memo.title` — 사용자 글을 글자로만 다루는 이유

듣는 사람이 할 질문 — 발표마다 하나씩 묻는다.

- "그 줄을 지우면 어떻게 되나요?"
- "AI 가 그렇게 하지 않았다는 것을 어떻게 확인했나요?"
- "그 테스트는 어떤 요구사항을 확인하나요?"

8단계. 돌아보기 (10분)

`harness/WORKLOG.md` 맨 아래에 적는다.

```
## 과정을 마치며

### 1. 1교시의 나에게 "하네스 엔지니어링"을 한 문장으로 설명한다면

### 2. AI 가 가장 잘한 일과, 내가 반드시 확인해야 했던 일

### 3. 하네스의 다섯 요소 중 가장 효과가 컸던 것과 그 이유

### 4. 내 AGENTS.md 에 새로 넣고 싶은 규칙 한 줄 (3일 동안의 경험에서)

### 5. 다음에 혼자 만들어 보고 싶은 것
```

4번에 적은 규칙을 실제로 `AGENTS.md` 에 넣고 커밋·푸시한다. **하네스는 한 번 세우고 끝나는 것이 아니라, 겪은 일로부터 계속 고쳐 가는 것이다.**

■ [dev] dev 컨테이너

```
git add -A
git commit -m "12교시: 회고와 AGENTS.md 보완"
```

■ [PS] 윈도우 PowerShell

```
git push
```

이 다음에 — 스스로 해 볼 것

같은 방법으로 기능을 하나씩 더해 본다. **순서는 언제나 같다.** 요구사항을 쓰고 → 지시서를 쓰고 → 계획을 검토하고 → 검증하고 → 읽고 → 커밋한다.

해 볼 것	먼저 써야 하는 요구사항	조심할 점
메모 검색 (`GET /memos?q=우유`)	무엇을 검색하는가(제목? 내용?), 결과가 없으면?	검색어도 `%s` 자리표시자로 넣는다
완료 표시 (할 일 목록으로)	새 열의 이름·타입·기본값	이미 있는 테이블을 바꾸는 방법이 필요하다
메모 개수 제한 (목록을 20개씩)	몇 개씩, 다음 묶음은 어떻게 부르는가	기존 테스트가 깨지지 않아야 한다
다른 주제의 CRUD (도서 목록, 가계부)	R1~R7 을 새 주제로 다시 쓴다	새 폴더에서 하네스부터 세운다

마지막 줄을 권한다. `AGENTS.md`, `opencode.jsonc`, `scripts/verify.sh`, `.githooks/pre-commit`, `harness/TASK_TEMPLATE.md`, `harness/VERIFY_CHECKLIST.md` 는 주제가 바뀌어도 거의 그대로 쓸 수 있다. **3일 동안 만든 것 중 가장 오래 쓸 것은 메모장이 아니라 이 하네스다.**

하네스 관점 — 과정 전체를 돌아보며

교시	사람이 한 일	AI 가 한 일	통제한 장치
1~2	개념, 환경	질문에 답	컨테이너, `.env`
3~4	`/hello`, GET·POST 를 손으로	코드 설명	(없음 — 불편을 겪어 봄)
5	규칙·권한·검증을 세움	—	`AGENTS.md`, `opencode.jsonc`, `pre-commit`
6	계획 승인, 검토	메모 초안 + 테스트	지시서, 권한, 검증
7	SQL 을 손으로	질문에 답	—
8	테스트가 안 바뀌었는지 확인	저장소 교체	"테스트는 그대로"
9	지시서 작성, SQL 검사	수정·삭제	"테스트는 추가만"
10	수동 점검표	화면	"사람이 확인한다"
11	테스트를 먼저 씀	버그 수정	"tests/ 는 금지"
12	README, 발표	검토 의견	올리기 전 확인

코드는 AI 가 더 많이 썼다. 그러나 **무엇을 만들지, 어디까지 허락할지, 결과를 받아들일지**는 처음부터 끝까지 사람이 정했다.

자주 만나는 오류

증상	원인과 해결
`git push` 가 `Permission denied` 또는 `403`	`origin` 이 아직 강사 저장소다. `git remote -v` 로 확인하고 5단계 (2)를 한다. 또는 다른 GitHub 계정으로 로그인되어 있다.
`remote origin already exists`	`git remote rename origin upstream` 을 먼저 하지 않았다.
`Updates were rejected`	GitHub 저장소를 만들 때 README 등을 함께 만들었다. 저장소를 지우고 빈 저장소로 다시 만든다.
`src refspec main does not match any`	브랜치 이름이 `main` 이 아니다. `git branch` 로 이름을 확인하고 그 이름으로 푸시한다.
dev 컨테이너에서 `git push` 를 했더니 로그인을 묻는다	푸시는 윈도우 PowerShell 에서 한다.
윈도우에서 `git status` 를 하니 모든 파일이 바뀐 것으로 나온다	줄바꿈 설정이 어긋났다. `.gitattributes` 가 있는지 확인한다. 강사에게 알린다.
6단계에서 `port is already allocated`	원래 폴더의 컨테이너가 켜져 있다. `C:\harness-crud-lab` 에서 `docker compose down` 을 먼저 한다.
6단계에서 컨테이너 이름이 겹친다는 오류	`docker-compose.yml` 의 `name:` 이 같아서다. 위와 같이 원래 것을 먼저 내린다.
GitHub 에 `.env` 가 올라갔다	즉시 OpenRouter 에서 그 키를 지운다. 그다음 강사에게 알린다. 파일을 지우는 커밋만으로는 기록에서 사라지지 않는다.

확인 문제

1. 커밋과 푸시의 차이를 "어디에 저장되는가"로 설명하시오.
2. 푸시하기 전에 확인해야 할 것 세 가지와, 각각을 확인하는 명령을 쓰시오.

3. API 키를 실수로 올렸다. 가장 먼저 해야 할 일은 무엇인가? 파일을 지우는 커밋을 올리는 것만으로 충분하지 않은 이유는?
4. `origin` 을 `upstream` 으로 이름을 바꾸고 새 `origin` 을 추가한 이유는 무엇인가?
5. 저장소를 다른 폴더에 새로 내려받아 실행해 본 것은 무엇을 확인하기 위해서인가?
6. 새로 내려받은 저장소에서 메모 목록이 비어 있는 이유를 `.gitignore` 와 바인드 마운트로 설명하시오.
7. 이 과정에서 AI 가 한 일과 사람이 한 일을 각각 세 가지씩 쓰시오.
8. 바이브 코딩과 하네스 엔지니어링의 차이를, 3일 동안 직접 겪은 일 하나를 예로 들어 설명하시오.

부록 A. 하네스 문서

저장소의 `harness/` 폴더에 있는 문서를 그대로 실었다. 5교시부터 이 문서들을 읽고, 채우고, AI 에이전트에게 건넨다.

A-1. 하네스 폴더 안내 (harness/README.md)

하네스(harness) 는 원래 말에게 씌우는 마구(고삐·안장)를 뜻한다. 말은 힘이 세지만, 마구가 없으면 그 힘을 원하는 방향으로 쓸 수 없다. AI 에이전트도 같다. 코드를 빠르게 많이 쓰지만, **무엇을·어디까지·어떤 기준으로** 할지 정해 주는 장치가 없으면 결과를 믿을 수 없다.

이 폴더에는 그 장치들이 들어 있다.

하네스의 다섯 요소와 파일

요소	질문	이 프로젝트에서 담당하는 파일
규칙	어떻게 일해야 하는가?	`AGENTS.md` — 작업 절차, 코드 규칙, 보고 형식
도구	무엇으로 일하는가?	`Dockerfile`·`docker-compose.yml`(도구 상자), `scripts/verify.sh`, `AGENTS.md` 2절
권한	어디까지 해도 되는가?	`opencode.jsonc` — allow / ask / deny 규칙
실행	어디에서, 무엇을 가지고 실행되는가?	dev 컨테이너(`/workspace` 만 보인다), `tasks/` 의 작업 지시서
검증	다 됐다는 것을 어떻게 아는가?	`scripts/verify.sh`, `tests/`, `.githooks/pre-commit`, `VERIFY_CHECKLIST.md`

규칙(`AGENTS.md`)은 AI 에게 하는 **부탁**이고, 권한(`opencode.jsonc`)과 검증(`pre-commit`)은 부탁이 지켜지지 않아도 작동하는 **잠금장치**다. 둘 다 있어야 한다.

파일 목록

파일	누가 쓰는가	언제 쓰는가
`REQUIREMENTS.md`	사람이 쓴다	요구사항 명세. 무엇을 만들지의 기준. 모든 교시에서 참고
`AGENTS.md`	사람이 쓴다, AI 가 읽는다	규칙 완성본. 5교시에 프로젝트 맨 위 폴더로 가져간다
`AGENTS.template.md`	학습자가 채운다	5교시에 규칙을 직접 써 보는 틀
`opencode.jsonc`	사람이 쓴다	권한 규칙 완성본. 5교시에 프로젝트 맨 위 폴더로 가져간다
`TASK_TEMPLATE.md`	학습자가 채운다	AI 에게 일을 시킬 때마다 쓰는 작업 지시서 틀
`tasks/`	사람이 쓴다, AI 가 읽는다	6·8·9·10·11교시의 작업 지시서
`VERIFY_CHECKLIST.md`	학습자가 체크한다	AI 가 "다 했다"고 한 뒤, 커밋하기 전
`WORKLOG.md`	학습자가 쓴다	교시마다 한 일과 이해한 내용을 기록

왜 `AGENTS.md` 가 여기(harness 폴더)에 있는가

OpenCode 는 프로젝트 **맨 위 폴더**의 `AGENTS.md` 를 자동으로 읽는다. 처음에는 맨 위 폴더에 `AGENTS.md` 가 없다. 그래서 2~4교시에는 AI 가 아무 규칙 없이 일한다. 5교시에 규칙을 직접 써서 맨 위 폴더에 놓고, **규칙이 있을 때와 없을 때 AI 의 행동이 어떻게 달라지는지** 비교한다.

한 번의 작업이 흘러가는 길

```
① 요구사항        REQUIREMENTS.md 에서 이번에 만들 번호(R1 ...)를 고른다
    |
② 작업 지시       tasks/NN_….md 를 AI 에게 건넨다  (틀: TASK_TEMPLATE.md)
    |
③ 하네스가 통제   AGENTS.md(규칙) + opencode.jsonc(권한) 안에서만 AI 가 움직인다
    |               · 계획을 먼저 말하고 사람의 "진행"을 기다린다
    |               · 금지된 명령·파일은 실행 자체가 막힌다
    |
④ 실행·테스트     sh scripts/verify.sh  (ruff + pytest)
    |
⑤ 확인·수정·재검증
    ├ 실패 → 오류를 AI 에게 보여 주고 수정 → ④ 로 돌아간다
    └ 통과 → 사람이 VERIFY_CHECKLIST.md 로 검토 → git commit
              (pre-commit 이 한 번 더 검증한다. 실패하면 커밋이 취소된다)
```

A-2. 요구사항 명세 (harness/REQUIREMENTS.md)

> 수업 흐름의 ① **사람이 요구사항 작성**에 해당하는 문서다. "무엇을 만들 것인가"를 사람이 정해서 적는다. "어떻게 만들 것인가"는 적지 않는다. AI 에이전트는 이 문서를 기준으로 구현하고, 테스트는 이 문서의 번호(R1 …)를 하나씩 확인한다.

1. 한 문장 요약

브라우저에서 메모를 쓰고, 목록으로 보고, 고치고, 지울 수 있는 메모장. 메모는 PostgreSQL 에 저장되어 서버를 껐다 켜도 남아 있다.

2. 메모 한 건의 모양

항목	타입	설명
`id`	정수	서버가 자동으로 붙이는 번호 (1, 2, 3 …). 사용자가 정하지 않는다.
`title`	문자열	제목. 필수. 1~100자.
`content`	문자열	내용. 선택. 최대 1000자. 안 보내면 빈 문자열("").
`created_at`	날짜·시각	만든 시각. 서버가 기록한다.
`updated_at`	날짜·시각	마지막으로 고친 시각. 서버가 기록한다.

3. 기능 요구사항

각 요구사항은 **부르는 방법 → 기대하는 결과**로 적는다. 이 형식 그대로 테스트가 된다.

R1. 메모 만들기 (Create)

- `POST /memos` 에 `{"title": "장보기", "content": "우유, 달걀"}` 을 보내면
- 상태 코드 **201** 과 함께, 만들어진 메모(id, title, content, created_at, updated_at)를 돌려준다.
- `content` 를 빼고 보내면 `content` 는 `""` 로 저장된다.

R2. 메모 목록 보기 (Read)

- `GET /memos` 를 부르면 상태 코드 **200** 과 메모 목록(배열)을 돌려준다.
- **나중에 만든 메모가 앞에** 온다 (id 내림차순).
- 메모가 하나도 없으면 빈 배열 `[]` 을 돌려준다.

R3. 메모 한 건 보기 (Read)

- `GET /memos/1` 을 부르면 상태 코드 **200** 과 1번 메모를 돌려준다.
- 그 번호의 메모가 없으면 상태 코드 **404** 와 `{"detail": "메모를 찾을 수 없습니다."}` 를 돌려준다.

R4. 메모 고치기 (Update)

- `PUT /memos/1` 에 `{"title": "새 제목", "content": "새 내용"}` 을 보내면
- 상태 코드 **200** 과 함께 고쳐진 메모를 돌려준다. `updated_at` 은 고친 시각으로 바뀐다.
- 다시 조회해도 고친 내용이 남아 있다.
- 그 번호의 메모가 없으면 **404** (문구는 R3 과 같다).

R5. 메모 지우기 (Delete)

- `DELETE /memos/1` 을 부르면 상태 코드 **204** 를 돌려준다. 응답 내용은 없다.
- 지운 뒤 `GET /memos/1` 을 부르면 **404** 가 온다.
- 그 번호의 메모가 없으면 **404** (문구는 R3 과 같다).

R6. 입력 규칙

만들기(R1)와 고치기(R4)에 똑같이 적용한다. 규칙을 어기면 상태 코드 **422** 를 돌려주고 저장하지 않는다.

- `title` 이 없거나 빈 문자열(`""`)이면 422
- `title` 이 100자를 넘으면 422
- `content` 가 1000자를 넘으면 422
- `title` 이 공백뿐(`" "`)이면 422 — 11교시에 추가
- `title` 의 앞뒤 공백은 잘라 내고 저장한다 (`" 장보기 "` → `"장보기"`) — 11교시에 추가

R7. 웹 화면

- 브라우저로 `GET /` 에 들어가면 "메모장" 화면(HTML)이 나온다.
- 화면에서 메모를 **쓰고, 목록으로 보고, 고치고, 지울 수** 있다. 화면은 R1~R5 의 API 를 불러서 동작한다.
- 저장·수정·삭제 뒤에는 새로 고침 없이 목록이 바뀐다.
- 입력이 규칙에 맞지 않으면 화면에 안내 문구가 나온다.

그 밖의 주소

- `GET /health` → **200**, `{"status": "ok"}` (서버가 살아 있는지 확인)
- `GET /hello` → **200**, `{"message": "안녕하세요, 하네스!"}`
- `GET /hello?name=철수` → **200**, `{"message": "안녕하세요, 철수!"}`

4. 품질 요구사항

기능이 아니라 "어떤 식으로 만들어져야 하는가"에 대한 약속이다.

번호	내용
N1	메모는 PostgreSQL 에 저장한다. 서버를 다시 켜도 메모가 남아 있다. (8교시부터)
N2	SQL 에 값을 넣을 때 자리표시자(`%s`)를 쓴다. 문자열을 이어 붙이지 않는다.
N3	모든 소스 파일에 한국어 주석(파일 머리 설명 + 핵심 줄 설명)이 있다.
N4	API 키와 비밀번호는 코드에 적지 않는다. 환경 변수에서 읽는다.
N5	모든 기능 요구사항에 자동 테스트가 있고, `sh scripts/verify.sh` 가 통과한다.
N6	화면은 프레임워크 없이 HTML·CSS·자바스크립트만으로 만든다. 사용자가 쓴 글은 `textContent` 로 넣는다.

5. 이번에는 만들지 않는 것

범위를 정해 두지 않으면 AI 는 시키지 않은 기능까지 만든다. 아래는 **만들지 않는다.**

- 로그인, 회원 가입, 사용자별 메모
- 검색, 태그, 정렬 바꾸기, 페이지 나누기
- 파일 첨부, 이미지
- 화면 꾸미기 프레임워크(React, Bootstrap 등)

6. 교시별 구현 범위

교시	구현하는 요구사항	저장소
3	`/health`, `/hello`	—
6	R1, R2, R3, R6(공백 규칙 제외)	메모리(딕셔너리)
8	R1, R2, R3, R6 그대로 + N1	PostgreSQL
9	R4, R5	PostgreSQL
10	R7	PostgreSQL
11	R6 의 공백 규칙 두 가지	PostgreSQL

A-3. 에이전트 규칙 (harness/AGENTS.md)

> 이 문서는 **AI 에이전트가 이 프로젝트에서 일할 때 지켜야 하는 규칙**이다. 에이전트는 작업을 시작할 때마다 이 문서를 먼저 읽는다. 사람(학습자)은 이 문서를 고칠 수 있지만, 에이전트는 고칠 수 없다.

1. 프로젝트 소개

- **목표**: 메모를 만들고(Create), 보고(Read), 고치고(Update), 지우는(Delete) 웹 서비스를 만든다.
- **사용자**: 프로그래밍을 처음 배우는 학습자. 학습자가 코드를 **읽고 이해할 수 있어야** 한다.
- **기술**: Python 3.12, FastAPI, PostgreSQL 17, psycopg 3, pytest, 화면은 HTML + 자바스크립트(프레임워크 없음)
- **요구사항**: `harness/REQUIREMENTS.md` 에 번호(R1, R2 ...)로 적혀 있다. 구현의 기준은 항상 이 문서다.

폴더 구조

경로	내용	에이전트가 고쳐도 되는가
`app/`	서버 코드 (`main.py`, `db.py`, `schemas.py`, `static/`)	예
`tests/`	자동 테스트	예 (단, 5절의 테스트 규칙을 지킨다)
`harness/`	요구사항, 작업 지시서, 체크리스트	**아니오** (읽기만)
`scripts/verify.sh`	검증 스크립트	**아니오**
`AGENTS.md`, `opencode.jsonc`, `.githooks/`	하네스 설정	**아니오**
`checkpoints/`	교시별 완성 예시	**아니오** (읽지도 않는다)
`.env`	API 키, 비밀번호	**아니오** (읽지도 않는다)

2. 도구 — 무엇으로 일하는가

에이전트는 **dev 컨테이너 안의 `/workspace`** 에서 실행된다.

하려는 일	명령
전체 검증(코드 검사 + 테스트)	`sh scripts/verify.sh`
테스트만 실행	`pytest -q`
테스트 하나만 실행	`pytest -q tests/test_memos.py::test_create_memo`
코드 검사만 실행	`ruff check app tests scripts`
무엇이 바뀌었는지 보기	`git status`, `git diff`
실행 중인 서버 불러 보기	`curl -s http://app:8000/health`

알아 둘 것:

- 서버는 **app 컨테이너**에서 이미 켜져 있다. `app/` 의 파일을 고치면 저절로 다시 켜진다. 에이전트가 `uvicorn` 을 따로 실행하지 않는다.
- 데이터베이스는 **db 컨테이너**에 있다. 접속 주소는 환경 변수 `DATABASE_URL` 에 들어 있다. 코드에 주소나 비밀번호를 직접 적지 않는다.
- 테스트는 별도의 DB(`memo_test`)에서 돈다. 테스트가 학습자의 진짜 메모(`memo_db`)를 지우는 일이 없어야 한다.
- 컨테이너 안에는 `docker` 명령이 없다. 컨테이너를 켜고 끄는 일은 사람이 한다.

3. 권한 — 해도 되는 일과 안 되는 일

이 규칙은 `opencode.jsonc` 의 `permissions` 에도 같은 내용으로 걸려 있다.

묻지 않고 해도 되는 일

- `app/`, `tests/` 안의 파일을 읽고, 만들고, 고치기
- 2절 표의 검증·조회 명령 실행

사람에게 먼저 물어봐야 하는 일

- 2절 표에 없는 명령 실행
- 새 패키지 추가(`requirements.txt` 수정), `Dockerfile`·`docker-compose.yml` 수정
- 인터넷 검색, 웹 페이지 가져오기
- 작업 지시서에 적히지 않은 파일 고치기

하면 안 되는 일

- `git commit`, `git push`, `git reset` — 저장과 올리기는 사람이 검토한 뒤에 직접 한다.
- `.env` 읽기·고치기, API 키나 비밀번호를 코드·로그·답변에 적기
- `harness/`, `AGENTS.md`, `opencode.jsonc`, `scripts/verify.sh`, `.githooks/` 고치기
- `checkpoints/` 읽기 (완성 예시를 베끼지 않는다)
- 파일이나 폴더를 통째로 지우는 명령(`rm -rf` 등)

4. 작업 절차 — 어떤 순서로 일하는가

모든 작업은 아래 다섯 단계를 순서대로 밟는다. 단계를 건너뛰지 않는다.

1. **읽기** — 작업 지시서와, 거기 적힌 요구사항 번호(R1 …)를 `harness/REQUIREMENTS.md` 에서 찾아 읽는다. 고칠 파일의 현재 내용도 읽는다.
2. **계획** — 무엇을, 어느 파일에, 어떤 순서로 할지 5줄 안팎으로 말한다. **사람이 "진행"이라고 답할 때까지 파일을 고치지 않는다.**
3. **구현** — 계획한 범위 안에서만 고친다. 범위 밖에 고칠 것이 보이면 고치지 말고 보고에 적는다.

4. **검증** — `sh scripts/verify.sh` 를 실행한다. 실패하면 원인을 찾아 고치고 다시 실행한다. 세 번 고쳐도 통과하지 못하면 멈추고 상황을 보고한다.
5. **보고** — 7절의 형식으로 보고한다. 커밋은 하지 않는다.

요청이 모호하면 짐작으로 진행하지 않는다. 질문을 한 번에 모아서 묻는다.

5. 코드 규칙

공통

- 한 번에 한 가지 일만 한다. 지시받지 않은 기능을 덧붙이지 않는다.
- 지금 있는 코드의 방식(이름 짓기, 파일 나누기)을 따른다. 새 파일은 꼭 필요할 때만 만든다.
- 새 패키지를 쓰지 않는다. 필요하다고 판단되면 이유를 말하고 사람에게 묻는다.

학습자가 읽을 수 있는 코드

- 모든 파일의 맨 위에 "이 파일이 보여 주는 개념"을 한국어 주석으로 적는다.
- 핵심 줄에는 "왜 이렇게 쓰는지"를 한국어 주석으로 짧게 적는다.
- 짧게 줄인 기교보다 풀어 쓴 코드를 고른다. 한 줄에 여러 일을 넣지 않는다.
- 변수와 함수 이름은 뜻이 드러나는 영어 단어로 쓴다 (`m` 대신 `memo`).

API

- 주소, 상태 코드, 오류 문구는 `harness/REQUIREMENTS.md` 에 적힌 그대로 구현한다.
- 요청과 응답의 모양은 `app/schemas.py` 의 Pydantic 클래스로 정한다.

데이터베이스

- SQL 에 값을 넣을 때는 반드시 `%s` 자리표시자를 쓴다. 문자열을 이어 붙여 SQL 을 만들지 않는다.
- `UPDATE` 와 `DELETE` 에는 반드시 `WHERE` 를 쓴다.
- DB 를 다루는 코드는 `app/db.py` 와 `app/main.py` 의 해당 함수 안에만 둔다.

테스트

- 기능을 만들면 그 기능의 테스트도 함께 만든다. 정상 경우와 실패 경우(없는 번호, 잘못된 입력)를 모두 넣는다.
- 테스트 이름과 주석에 요구사항 번호(R1 …)가 드러나게 한다.
- **테스트를 통과시키려고 테스트를 지우거나, 느슨하게 고치거나, 건너뛰게 만들지 않는다.** 테스트가 틀렸다고 판단되면 고치지 말고 이유를 보고한다.

화면(HTML·자바스크립트)

- 프레임워크와 외부 라이브러리를 쓰지 않는다.
- 사용자가 입력한 글은 `textContent` 로 넣는다. `innerHTML` 로 넣지 않는다.

6. 검증 — "다 됐다"의 기준

아래를 **모두** 만족해야 작업이 끝난 것이다.

- ■ `sh scripts/verify.sh` 가 "검증 통과"로 끝난다. (에이전트가 직접 실행해 확인한다)
- ■ 작업 지시서의 완료 조건을 하나씩 확인했다.
- ■ 새로 만든 기능마다 테스트가 있다.
- ■ 지시받지 않은 파일은 바뀌지 않았다. (`git status` 로 확인한다)
- ■ 비밀 값이 코드에 들어가지 않았다.

검증을 실행하지 않았거나 실패했는데 "완료했습니다"라고 말하지 않는다. 실패했으면 실패했다고 말한다.

7. 보고 형식

작업을 마치면 아래 형식으로 한국어로 보고한다.

```
[한 일]      무엇을 만들었거나 고쳤는지 2~3줄
[바뀐 파일]  파일 이름과, 파일마다 한 줄 설명
[검증 결과]  sh scripts/verify.sh 의 마지막 줄 (통과한 테스트 수)
[요구사항]   이번 작업이 다룬 요구사항 번호와 그것을 확인하는 테스트 이름
[확인 방법]  사람이 직접 눈으로 확인할 수 있는 방법 (주소, 명령)
[남은 일]    하지 못한 일, 범위 밖이라 손대지 않은 문제, 사람이 결정해야 할 것
```

8. 설명 요청을 받았을 때

학습자가 "이 코드를 설명해 줘"라고 하면:

- 파일을 고치지 않는다. 설명만 한다.
- 전문 용어를 쓸 때는 한 줄로 뜻을 덧붙인다.
- "요청이 들어와서 응답이 나갈 때까지" 순서대로 설명한다.
- 설명 끝에 학습자가 스스로 확인해 볼 수 있는 질문을 1~2개 붙인다.

A-4. 작업 지시서 틀 (harness/TASK_TEMPLATE.md)

AI 에이전트에게 일을 시킬 때 쓰는 틀이다. 수업 흐름의 ② **AI 에이전트에게 개발 지시**에 해당한다. "메모 앱 만들어 줘" 한 줄 대신, 아래 여섯 칸을 채워서 건넨다. 채운 예시는 `tasks/` 폴더에 있다.

```
# 작업 지시서: (작업 이름)

## 1. 목표
(이 작업이 끝나면 무엇이 되어 있어야 하는지 한두 문장)

## 2. 요구사항
(harness/REQUIREMENTS.md 의 번호. 예: R1, R2, R3)

## 3. 범위
- 고쳐도 되는 파일: (예: app/main.py, app/schemas.py, tests/test_memos.py)
- 고치면 안 되는 것: (예: tests/test_health.py, 이미 통과하는 테스트의 내용)

## 4. 하지 말 것
- (이번 작업에서 만들지 않을 기능. 예: 수정·삭제는 만들지 않는다)
- (쓰지 않을 방법. 예: 새 패키지를 추가하지 않는다)

## 5. 완료 조건
- [ ] sh scripts/verify.sh 가 "검증 통과"로 끝난다
- [ ] (눈으로 확인할 수 있는 조건. 예: POST /memos 를 부르면 201 이 온다)
- [ ] (요구사항마다 테스트가 있다)

## 6. 진행 방법
AGENTS.md 의 작업 절차를 따른다. 먼저 계획을 말하고, 내가 "진행"이라고 하면 구현한다.
끝나면 AGENTS.md 의 보고 형식으로 보고한다.
```

칸마다 왜 필요한가

칸	비워 두면 생기는 일
1. 목표	AI 가 "무엇이 끝인지" 몰라서 계속 덧붙이거나, 반만 하고 끝낸다.
2. 요구사항	주소·상태 코드·오류 문구를 AI 가 제멋대로 정한다. 매번 결과가 달라진다.
3. 범위	관계없는 파일까지 고친다. 무엇이 왜 바뀌었는지 사람이 따라갈 수 없다.
4. 하지 말 것	시키지 않은 기능(로그인, 검색 …)이 생긴다. 읽고 검토할 코드가 불어난다.
5. 완료 조건	"다 했습니다"라는 말을 확인할 방법이 없다.
6. 진행 방법	계획 없이 바로 고치기 시작한다. 방향이 틀려도 끝난 뒤에야 알게 된다.

좋은 지시와 나쁜 지시

나쁜 지시	왜 나쁜가	고친 지시
"메모 기능 만들어 줘"	범위도 기준도 없다	"R1~R3 을 구현해 줘. 저장소는 메모리. 수정·삭제는 만들지 않는다."
"에러 나는데 고쳐 줘"	어떤 오류인지 AI 가 추측해야 한다	"`sh scripts/verify.sh` 결과를 붙인다. `test_get_memo_not_found` 가 실패한다. 원인을 먼저 설명하고 고쳐 줘."
"알아서 잘 해 줘"	판단을 전부 넘긴다	"두 가지 방법이 있으면 장단점을 말하고 내 선택을 기다려 줘."
"테스트 통과하게 해 줘"	테스트를 고쳐서 통과시킬 수 있다	"`app/` 의 코드를 고쳐서 통과시켜 줘. `tests/` 는 고치지 않는다."

한 번에 얼마만큼 시킬 것인가

작업은 **작게** 나눈다. 이 과정에서 CRUD 를 한 번에 시키지 않고 6·8·9·10·11교시로 나눈 이유다.

- 작은 작업은 AI 가 틀릴 여지가 적다.
- 바뀐 코드가 적어서 사람이 **읽고 이해할 수 있다.**
- 문제가 생기면 그 작업만 되돌리면 된다.

기준: **AI 가 바꾼 내용(`git diff`)을 내가 10분 안에 읽고 설명할 수 있는 크기.**

A-5. 6교시 작업 지시서 (harness/tasks/06_memo_draft.md)

1. 목표

메모를 만들고, 목록으로 보고, 한 건을 볼 수 있는 API 를 만든다. 저장소는 아직 DB 가 아니라 파이썬 메모리(딕셔너리)를 쓴다.

2. 요구사항

`harness/REQUIREMENTS.md` 의 **R1, R2, R3, R6** 을 구현한다. 단, R6 중 "11교시에 추가"라고 적힌 공백 규칙 두 가지는 이번에 만들지 않는다.

3. 범위

- 고쳐도 되는 파일: `app/main.py`, `tests/conftest.py`
- 새로 만드는 파일: `app/schemas.py`, `tests/test_memos.py`
- 고치면 안 되는 것: `tests/test_health.py`, 지금 있는 `/health`·`/hello` 의 동작

4. 하지 말 것

- 수정(PUT)과 삭제(DELETE)는 만들지 않는다. (9교시에 한다)
- PostgreSQL 에 연결하지 않는다. (8교시에 한다)
- 화면(HTML)을 만들지 않는다. (10교시에 한다)
- 새 패키지를 추가하지 않는다.
- 4교시에 손으로 만든 `/memos` 코드(리스트 사용, `id` 를 `len(memos) + 1` 로 정함)는 요구사항에 맞게 다시 써도 된다.

5. 완료 조건

- [ ] `sh scripts/verify.sh` 가 "검증 통과"로 끝난다.
- [ ] 요청·응답의 모양이 `app/schemas.py` 에 Pydantic 클래스로 정의되어 있다.
- [ ] 응답에 `id`, `title`, `content`, `created_at`, `updated_at` 이 모두 들어 있다.
- [ ] 없는 번호를 조회하면 404 와 `{"detail": "메모를 찾을 수 없습니다."}` 가 온다.
- [ ] `tests/test_memos.py` 에 R1, R2, R3, R6 각각의 테스트가 있다. (정상 경우 + 실패 경우)
- [ ] 테스트끼리 서로 영향을 주지 않는다. (테스트마다 저장소가 비워진 상태에서 시작한다)
- [ ] 모든 파일에 한국어 주석(파일 머리 설명 + 핵심 줄)이 있다.

6. 진행 방법

`AGENTS.md` 의 작업 절차를 따른다. 먼저 계획을 말하고, 내가 "진행"이라고 하면 구현한다. 끝나면 `AGENTS.md` 의 보고 형식으로 보고한다.

A-6. 8교시 작업 지시서 (harness/tasks/08_db_connect.md)

1. 목표

메모를 메모리 대신 PostgreSQL 에 저장한다. API 의 겉모습(주소, 요청, 응답, 상태 코드)은 **하나도 바꾸지 않는다.** 바뀌는 것은 저장하는 곳뿐이다.

2. 요구사항

`harness/REQUIREMENTS.md` 의 **R1, R2, R3, R6** 을 그대로 유지하면서 **N1, N2, N4** 를 만족한다.

3. 범위

- 새로 만드는 파일: `app/db.py`
- 고쳐도 되는 파일: `app/main.py`, `tests/conftest.py`
- 고치면 안 되는 것: **`tests/test_memos.py`, `tests/test_health.py`** (테스트는 한 글자도 바꾸지 않는다)

4. 하지 말 것

- 수정(PUT)과 삭제(DELETE)는 만들지 않는다. (9교시에 한다)
- ORM(SQLAlchemy 등)을 쓰지 않는다. `psycopg` 로 SQL 을 직접 실행한다. (7교시에 배운 SQL 이 코드에 그대로 보이게 한다)
- 접속 주소나 비밀번호를 코드에 적지 않는다.
- 새 패키지를 추가하지 않는다. (`psycopg` 는 이미 설치되어 있다)

5. 구현 조건

- 접속 주소는 환경 변수 `DATABASE_URL` 에서 읽는다.
- 테이블 이름은 `memos`. 열은 `id`(SERIAL PRIMARY KEY), `title`(VARCHAR(100) NOT NULL), `content`(TEXT NOT NULL DEFAULT ''), `created_at`·`updated_at`(TIMESTAMPTZ NOT NULL DEFAULT now()).
- 서버가 켜질 때 테이블이 없으면 만든다. (`CREATE TABLE IF NOT EXISTS`)
- SQL 에 값을 넣을 때는 `%s` 자리표시자를 쓴다.
- 테스트는 환경 변수 `TEST_DATABASE_URL` 이 가리키는 **테스트 전용 DB**(`memo_test`)에서만 돈다.
  - 테스트 전용 DB 가 없으면 `tests/conftest.py` 가 만든다.
  - 테스트 하나가 시작될 때마다 `memos` 테이블을 비우고 번호를 1부터 다시 시작한다.

6. 완료 조건

- ■ `sh scripts/verify.sh` 가 "검증 통과"로 끝난다. **테스트 개수는 작업 전과 같다.**
- ■ `git diff --stat` 에 `tests/test_memos.py` 가 나타나지 않는다.
- ■ 메모를 만든 뒤 서버를 다시 켜도 메모가 남아 있다. (사람이 확인한다)
- ■ 테스트를 실행해도 `memo_db` 의 메모는 지워지지 않는다. (사람이 확인한다)
- ■ `app/main.py` 에 메모리 저장소(딕셔너리, `next_id`, `reset_store`)가 남아 있지 않다.

7. 진행 방법

`AGENTS.md` 의 작업 절차를 따른다. 먼저 계획을 말하고, 내가 "진행"이라고 하면 구현한다. 끝나면 `AGENTS.md` 의 보고 형식으로 보고한다.

A-7. 9교시 작업 지시서 (harness/tasks/09_update_delete.md)

1. 목표

메모를 고치는 API 와 지우는 API 를 더해서 CRUD 네 가지를 완성한다.

2. 요구사항

`harness/REQUIREMENTS.md` 의 **R4, R5** 를 구현한다. 고치기에는 **R6**(입력 규칙)이 똑같이 적용된다.

3. 범위

- 고쳐도 되는 파일: `app/main.py`, `tests/test_memos.py`(테스트 **추가**만)
- 고치면 안 되는 것: 이미 있는 테스트의 내용, `app/db.py` 의 테이블 구조

4. 하지 말 것

- "일부만 고치기"(PATCH)는 만들지 않는다. 고치기는 PUT 하나로, 제목과 내용을 모두 받는다.
- 지운 메모를 되살리는 기능(휴지통)을 만들지 않는다.
- 화면(HTML)을 만들지 않는다. (10교시에 한다)

5. 구현 조건

- 고칠 때 `updated_at` 을 현재 시각으로 바꾼다. `created_at` 은 바꾸지 않는다.
- `UPDATE` 와 `DELETE` 에는 반드시 `WHERE id = %s` 를 쓴다.
- 없는 번호를 고치거나 지우려 하면 404 와 `{"detail": "메모를 찾을 수 없습니다."}` 를 돌려준다. (R3 과 같은 문구)
- 404 를 만드는 코드가 여러 곳에 반복되면 한 곳으로 모은다.

6. 완료 조건

- ■ `sh scripts/verify.sh` 가 "검증 통과"로 끝난다. 테스트 개수가 작업 전보다 **늘었다.**
- ■ R4 테스트: 고치기 성공, 고친 뒤 다시 조회해도 바뀐 내용이 남아 있음, 없는 번호는 404, 빈 제목은 422
- ■ R5 테스트: 지우기 성공(204), 지운 뒤 조회하면 404, 없는 번호는 404
- ■ `git diff tests/` 에서 **지워진 줄(-)이 없다.** (기존 테스트를 건드리지 않았다)

7. 진행 방법

`AGENTS.md` 의 작업 절차를 따른다. 먼저 계획을 말하고, 내가 "진행"이라고 하면 구현한다. 끝나면 `AGENTS.md` 의 보고 형식으로 보고한다.

A-8. 10교시 작업 지시서 (harness/tasks/10_web_page.md)

1. 목표

브라우저에서 메모를 쓰고, 보고, 고치고, 지울 수 있는 화면을 만든다. 화면은 이미 만들어 둔 API(`/memos`)를 불러서 동작한다.

2. 요구사항

`harness/REQUIREMENTS.md` 의 **R7** 과 **N6** 을 구현한다.

3. 범위

- 새로 만드는 파일: `app/static/index.html`, `app/static/app.js`, `app/static/style.css`
- 고쳐도 되는 파일: `app/main.py`(화면을 내려 주는 부분만 추가), `tests/test_memos.py`(테스트 **추가**만)
- 고치면 안 되는 것: `/memos` API 의 동작, 이미 있는 테스트의 내용

4. 하지 말 것

- 프레임워크나 외부 라이브러리(React, Vue, jQuery, Bootstrap, CDN 링크)를 쓰지 않는다.
- API 를 새로 만들거나 바꾸지 않는다. 화면에 필요한 것은 R1~R5 로 충분하다.
- 검색, 정렬, 페이지 나누기를 만들지 않는다.

5. 구현 조건

- `GET /` 는 `app/static/index.html` 을 돌려준다. `/static/…` 주소로 `app.js`, `style.css` 를 내려 준다.
- 화면 구성: 제목 입력 칸, 내용 입력 칸, 저장 버튼, 안내 문구 자리, 메모 목록. 목록의 메모마다 "수정", "삭제" 버튼.
- "수정"을 누르면 위의 입력 칸에 그 메모의 내용이 채워지고, 저장 버튼이 "수정 저장"으로 바뀐다. "수정 취소" 버튼으로 되돌릴 수 있다.
- "삭제"는 지우기 전에 한 번 묻는다.
- 저장·수정·삭제 뒤에는 목록을 다시 불러온다. (페이지 새로 고침 없이)
- 서버가 422 나 404 를 돌려주면 안내 문구 자리에 한국어 문장을 보여 준다.
- **사용자가 쓴 글(제목·내용)은 `textContent` 로 화면에 넣는다. `innerHTML` 을 쓰지 않는다.**
- HTML, 자바스크립트, CSS 파일마다 한국어 주석(파일 머리 설명 + 핵심 줄)을 단다.

6. 완료 조건

- ■ `sh scripts/verify.sh` 가 "검증 통과"로 끝난다.
- ■ `GET /` 가 200 이고 응답에 "메모장"이라는 글자가 있는지 확인하는 테스트가 있다.
- ■ `app/static/app.js` 에 `innerHTML` 이 없다. (`grep -n innerHTML app/static/app.js` 결과가 비어 있다)

- ■ 브라우저에서 쓰기 → 목록에 보임 → 수정 → 삭제가 모두 된다. (사람이 확인한다)
- ■ 제목에 `<b>굵게</b>` 를 넣어 저장하면 글자 그대로 보인다. (사람이 확인한다)

7. 진행 방법

`AGENTS.md` 의 작업 절차를 따른다. 먼저 계획을 말하고, 내가 "진행"이라고 하면 구현한다. 끝나면 `AGENTS.md` 의 보고 형식으로 보고한다.

A-9. 11교시 작업 지시서 (harness/tasks/11_blank_title_bug.md)

1. 목표

제목에 공백만 넣어도 메모가 저장되는 문제를 고친다.

2. 현재 상황 (사람이 직접 확인한 사실)

- `POST /memos` 에 `{"title": " ", "content": "공백 제목"}` 을 보내면 **201** 이 오고 메모가 저장된다.
- 요구사항 R6 에 따르면 **422** 가 와야 한다.
- 이 문제를 잡아내는 테스트 두 개를 사람이 `tests/test_memos.py` 에 먼저 추가했다.
  - `test_create_memo_blank_title`
  - `test_title_is_trimmed`
- 지금 `sh scripts/verify.sh` 를 실행하면 이 두 테스트가 **실패한다.** (의도한 상태다)

3. 요구사항

`harness/REQUIREMENTS.md` 의 **R6** 중 아래 두 줄.

- `title` 이 공백뿐이면 422
- `title` 의 앞뒤 공백은 잘라 내고 저장한다

4. 범위

- 고쳐도 되는 파일: `app/schemas.py`
- 고치면 안 되는 것: **`tests/` 폴더 전체** (테스트는 사람이 이미 썼다. 한 글자도 바꾸지 않는다)

5. 하지 말 것

- 테스트를 고치거나 지우거나 건너뛰게 만들어서 통과시키지 않는다.
- `app/main.py` 의 함수마다 검사 코드를 넣지 않는다. 입력 규칙은 한 곳(`app/schemas.py`)에 둔다. 그래야 만들기와 고치기에 똑같이 적용된다.
- 이 버그와 관계없는 코드를 고치지 않는다.

6. 완료 조건

- ■ `sh scripts/verify.sh` 가 "검증 통과"로 끝난다. 실패하던 두 테스트가 통과한다.
- ■ 원래 통과하던 테스트가 여전히 모두 통과한다.
- ■ `git diff --stat` 에 `app/schemas.py` 만 나타난다.

7. 진행 방법

1. 먼저 **원인을 설명한다.** (왜 `" "` 가 `min_length=1` 검사를 통과했는가)
2. 고칠 방법을 말하고, 내가 "진행"이라고 하면 구현한다.
3. 끝나면 `AGENTS.md` 의 보고 형식으로 보고한다.

A-10. 검증 체크리스트 (harness/VERIFY_CHECKLIST.md)

> AI 가 "다 했습니다"라고 한 뒤, **커밋하기 전에** 사람이 확인하는 목록이다. 수업 흐름의 ④ **코드 실행 및 테스트**와 ⑤ **오류 확인 → AI 수정 → 재검증**에 해당한다. 하나라도 "아니오"면 커밋하지 않는다. AI 에게 고치게 하거나, 직접 고친 뒤 처음부터 다시 확인한다.

1단계. 기계가 확인하는 것 (1분)

dev 컨테이너에서 실행한다.

```
sh scripts/verify.sh
```

- 마지막 줄이 `검증 통과: 이제 저장(commit)해도 됩니다.` 이다.
- 통과한 테스트 개수가 지난번보다 **줄지 않았다.** (줄었다면 테스트가 지워진 것이다)

2단계. 무엇이 바뀌었는지 확인 (3분)

```
git status
git diff --stat
```

- 바뀐 파일이 **작업 지시서의 "범위"에 적은 파일뿐**이다.
- `AGENTS.md`, `opencode.jsonc`, `harness/`, `scripts/verify.sh`, `.githooks/` 는 바뀌지 않았다.
- `.env` 가 목록에 없다. (`.gitignore` 가 막고 있어야 한다)
- 뜻 모를 새 파일(임시 파일, 백업 파일)이 생기지 않았다.

3단계. 테스트를 사람이 읽는다 (5분)

테스트가 통과했다는 것은 "테스트에 적힌 것"이 맞다는 뜻일 뿐이다. 테스트 자체가 엉터리면 통과는 아무 의미가 없다.

```
git diff tests/
```

- 이번 작업의 요구사항 번호마다 테스트가 있다. (아래 대응표로 확인)
- 정상 경우뿐 아니라 **실패 경우**(없는 번호 → 404, 잘못된 입력 → 422)도 테스트한다.
- 기존 테스트에서 **지워진 줄(-)이 없다.** 있다면 왜 지웠는지 AI 에게 묻는다.
- `assert True` 처럼 항상 통과하는 테스트, `skip` 으로 건너뛰는 테스트가 없다.
- 테스트가 확인하는 값이 요구사항 명세의 값(상태 코드, 오류 문구)과 같다.

요구사항 ↔ 테스트 대응표

요구사항	확인하는 테스트 (완성본 기준)	확인
R1 만들기	`test_create_memo`, `test_create_memo_without_content`	[ ]
R2 목록	`test_list_memos_empty`, `test_list_memos_newest_first`	[ ]
R3 한 건 보기	`test_get_memo`, `test_get_memo_not_found`	[ ]
R4 고치기	`test_update_memo`, `test_update_memo_not_found`, `test_update_memo_empty_title`	[ ]
R5 지우기	`test_delete_memo`, `test_delete_memo_not_found`	[ ]
R6 입력 규칙	`test_create_memo_empty_title`, `test_create_memo_title_too_long`, `test_create_memo_content_too_long`, `test_create_memo_blank_title`, `test_title_is_trimmed`	[ ]
R7 웹 화면	`test_index_page` + 브라우저에서 직접 확인	[ ]

AI 가 지은 테스트 이름은 이 표와 다를 수 있다. 이름이 아니라 **무엇을 확인하는지**를 본다.

4단계. 코드를 사람이 읽는다 (10분)

```
git diff app/
```

- 바뀐 코드를 **위에서 아래로 한 번 다 읽었다.**
- 모르는 줄이 있으면 AI 에게 물어서 이해했다. (예: "app/main.py 의 RETURNING * 이 무슨 뜻이야?")
- SQL 에 값을 넣는 곳이 모두 %s 다. f-문자열이나 + 로 SQL 을 만들지 않았다.
- UPDATE, DELETE 에 WHERE 가 있다.
- 비밀번호, API 키, 접속 주소가 코드에 적혀 있지 않다.
- 시키지 않은 기능이 들어 있지 않다.
- 한국어 주석이 있고, 주석의 내용이 코드와 맞다.

5단계. 직접 실행해 본다 (5분)

테스트가 못 잡는 것(화면 모양, 실제 DB 에 남는지)을 눈으로 확인한다.

- 브라우저에서 http://localhost:8000/docs 를 열어 이번에 만든 주소를 직접 불러 봤다.
- (8교시 이후) 서버를 다시 켠 뒤에도 메모가 남아 있다.
- (10교시 이후) http://localhost:8000 화면에서 쓰기·수정·삭제를 해 봤다.

6단계. 내 말로 설명한다 (5분)

WORKLOG.md 에 적는다. 적지 못하면 아직 이해하지 못한 것이다. 4단계로 돌아간다.

- 이번에 바뀐 내용을 **세 문장**으로 적었다.
- 새로 알게 된 용어나 문법을 **하나 이상** 적었다.

7단계. 저장한다

```
git add -A
git commit -m "9교시: 메모 수정·삭제 구현 (R4, R5)"
```

- 커밋 메시지에 **무엇을 했는지**와 **요구사항 번호**가 들어 있다.
- pre-commit 이 검증을 한 번 더 실행했고, 통과해서 커밋이 만들어졌다.

검증이 실패했을 때

1. 오류 메시지를 **끝까지** 읽는다. 대부분 마지막 몇 줄에 원인이 있다.
2. 어떤 테스트가, 무엇을 기대했는데, 실제로 무엇이 왔는지 찾는다. (assert 201 == 422 → 422 를 기대했는데 201 이 왔다)
3. AI 에게 **오류 전체를 붙여서** 전달한다. "안 돼요"라고만 하지 않는다.

```
sh scripts/verify.sh 가 실패했어. 아래가 출력이야.
원인을 먼저 설명하고, tests/ 는 건드리지 말고 app/ 을 고쳐 줘.

(오류 출력 붙여넣기)
```

4. AI 가 고치면 **1단계부터 다시** 확인한다.
5. 세 번 고쳐도 안 되면 멈춘다. 작업을 되돌리고(git restore .) 지시서를 더 작게 나누어 다시 시킨다.

A-11. 작업 기록 (harness/WORKLOG.md)

> 교시마다 **내가 시킨 일**과 **내가 이해한 것**을 적는다. AI 가 만든 코드를 "내 코드"로 만드는 과정이다. 설명하지 못하는 코드는 커밋하지 않는다. 12교시 발표는 이 기록을 바탕으로 한다.

적는 방법

교시가 끝날 때마다 아래 틀을 복사해서 채운다.

```
## N교시 — (작업 이름)

- 날짜:
- 작업 지시서: harness/tasks/…
- 커밋: (git log --oneline -1 의 결과)

### 1. AI 에게 시킨 일 (한 문장)

### 2. AI 가 세운 계획 중 내가 고치게 한 것
(없으면 "없음". 있으면 무엇을 왜 고치게 했는지)

### 3. 하네스가 막거나 물어본 것
(권한 때문에 막힌 명령, 물어봐서 내가 허락/거절한 것, 검증이 실패한 횟수)

### 4. 바뀐 코드를 내 말로 설명하기 (세 문장)
1.
2.
3.

### 5. 새로 알게 된 것
(용어, 문법, 명령)

### 6. 아직 이해하지 못한 것
(AI 에게 물어볼 질문으로 적는다)
```

작성 예시

6교시 — 메모 만들기·보기 초안

- 날짜: (작성)
- 작업 지시서: harness/tasks/06_memo_draft.md
- 커밋: (작성)

1. AI 에게 시킨 일 (한 문장)

메모를 만들고(POST), 목록과 한 건을 보는(GET) API 를 메모리 저장소로 만들게 했다.

2. AI 가 세운 계획 중 내가 고치게 한 것

계획에 "삭제 API 도 함께 만든다"가 있어서, 지시서 4번(하지 말 것)을 가리키며 빼게 했다.

3. 하네스가 막거나 물어본 것

- `pip install` 을 실행하려고 해서 물어봤다 → 거절했다. (새 패키지는 쓰지 않기로 했다)
- 검증 1회 실패(안 쓰는 import) → AI 가 스스로 고치고 다시 실행해서 통과.

4. 바뀐 코드를 내 말로 설명하기 (세 문장)

1. `schemas.py` 의 `MemoIn` 이 요청의 모양을 정하고, 제목이 비었거나 100자를 넘으면 FastAPI 가 422 를 돌려준다.
2. 메모는 `{번호: 메모}` 딕셔너리에 넣고, 번호는 `next_id` 로 따로 세어서 겹치지 않게 한다.
3. 없는 번호를 조회하면 `HTTPException(404)` 를 일으켜서 정해진 오류 문구를 돌려준다.

5. 새로 알게 된 것

- `response_model`: 응답의 모양을 정해 두면 그 모양대로만 나간다.
- `raise`: 함수를 그 자리에서 끝내고 오류를 알린다.

6. 아직 이해하지 못한 것

- `global next_id` 는 왜 필요한가? 없으면 어떤 오류가 나는가?

(여기부터 내 기록을 적는다)

A-12. 권한 규칙 (harness/opencode.jsonc)

```
// ============================================================
// opencode.jsonc - AI 에이전트(OpenCode) 설정 파일  [완성본: 권한 규칙 포함]
//
// 하네스의 "권한" 담당. AGENTS.md 가 "부탁"이라면 이 파일은 "잠금장치"다.
// 에이전트가 규칙을 잊거나 어겨도, 여기서 deny 한 일은 실행되지 않는다.
//
// 규칙 읽는 법
//   action   : 무엇을 하려는가   (shell = 명령 실행, edit = 파일 고치기, read = 파일 읽기 ...)
//   resource : 무엇에 대해서인가 (* 는 "아무 글자나"라는 뜻)
//   effect   : allow(그냥 허용) / ask(사람에게 물어봄) / deny(금지)
//
// 위에서 아래로 읽으며, 여러 규칙에 걸리면 "가장 아래 규칙"이 이긴다.
// 그래서 넓은 규칙을 위에, 예외를 아래에 적는다.
// ============================================================
{
  "$schema": "https://opencode.ai/config.json",
  "model": "openrouter/anthropic/claude-haiku-5.5",
  "share": "disabled",

  "permissions": [
    // ---------- 1. 명령 실행(shell) ----------
    // 기본값: 모든 명령은 실행 전에 사람에게 물어본다.
    { "action": "shell", "resource": "*", "effect": "ask" },

    // 검증에 필요한 명령과, 읽기만 하는 명령은 묻지 않고 허용한다.
    { "action": "shell", "resource": "sh scripts/verify.sh *", "effect": "allow" },
    { "action": "shell", "resource": "pytest *", "effect": "allow" },
    { "action": "shell", "resource": "ruff check *", "effect": "allow" },
    { "action": "shell", "resource": "git status *", "effect": "allow" },
    { "action": "shell", "resource": "git diff *", "effect": "allow" },
    { "action": "shell", "resource": "git log *", "effect": "allow" },
    { "action": "shell", "resource": "ls *", "effect": "allow" },

    // 저장(commit)과 올리기(push)는 사람만 한다. 에이전트는 금지.
    { "action": "shell", "resource": "git commit *", "effect": "deny" },
    { "action": "shell", "resource": "git push *", "effect": "deny" },
    // 되돌리기 어려운 명령도 금지.
    { "action": "shell", "resource": "git reset *", "effect": "deny" },
    { "action": "shell", "resource": "rm -rf *", "effect": "deny" },

    // ---------- 2. 파일 고치기(edit) ----------
    // app, tests 폴더는 자유롭게 고칠 수 있다(기본값이 allow).
    // 아래 파일들은 "하네스 자체"이므로 에이전트가 고치지 못한다.
    // (시험 보는 사람이 채점 기준을 고칠 수 없는 것과 같다)
    { "action": "edit", "resource": "*AGENTS.md", "effect": "deny" },
    { "action": "edit", "resource": "*opencode.jsonc", "effect": "deny" },
    { "action": "edit", "resource": "*/harness/*", "effect": "deny" },
    { "action": "edit", "resource": "*/scripts/verify.sh", "effect": "deny" },
    { "action": "edit", "resource": "*/.githooks/*", "effect": "deny" },
    { "action": "edit", "resource": "*/checkpoints/*", "effect": "deny" },
    // checkpoints 폴더(교시별 완성 예시)는 읽지도 못한다. 답을 베끼지 않고 요구사항을 보고 만들게 한다.
    { "action": "read", "resource": "*/checkpoints/*", "effect": "deny" },
    // 실행 환경 파일은 고치기 전에 사람에게 물어본다.
    { "action": "edit", "resource": "*Dockerfile", "effect": "ask" },
    { "action": "edit", "resource": "*docker-compose*.yml", "effect": "ask" },
    { "action": "edit", "resource": "*requirements.txt", "effect": "ask" },

    // ---------- 3. 비밀 값(.env) ----------
    // API 키가 들어 있는 .env 는 읽지도 고치지도 못한다. 견본(.env.example)은 읽어도 된다.
    { "action": "read", "resource": "*.env", "effect": "deny" },
    { "action": "edit", "resource": "*.env", "effect": "deny" },
    { "action": "read", "resource": "*.env.example", "effect": "allow" },
```

```
    // ---------- 4. 인터넷 ----------
    // 웹 페이지를 가져오거나 검색하기 전에는 사람에게 물어본다.
    { "action": "webfetch", "resource": "*", "effect": "ask" },
    { "action": "websearch", "resource": "*", "effect": "ask" }
  ]
}
```

부록 B. 실행 환경 파일

Docker 로 도구 상자를 만드는 파일과 검증 장치를 그대로 실었다. 주석을 따라 읽는다.

docker-compose.yml

```
# ============================================================
# docker-compose.yml - 이 과정에서 쓰는 컨테이너 3개의 설계도
#
#   db   : PostgreSQL 데이터베이스
#   app  : FastAPI 서버 (http://localhost:8000)
#   dev  : 작업실 - pytest, git, AI 에이전트(OpenCode)를 실행하는 곳
#
# 켜기  : docker compose up -d --build
# 끄기  : docker compose down
# 상태  : docker compose ps
#
# ${이름:-기본값} 은 ".env 파일에 그 이름이 있으면 그 값을, 없으면 기본값을 쓴다"는 뜻이다.
# ============================================================

name: harness-crud-lab

services:
  # ---------- 데이터베이스 ----------
  db:
    image: postgres:17
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-memo}            # DB 계정
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-memo1234} # DB 비밀번호(실습용)
      POSTGRES_DB: ${POSTGRES_DB:-memo_db}             # 처음 켤 때 자동으로 만들어지는 DB
      TZ: Asia/Seoul
    volumes:
      # [바인드 마운트] C 드라이브의 data\postgres 폴더 ↔ 컨테이너의 데이터 폴더
      # → 컨테이너를 지워도 메모 데이터는 C 드라이브에 남는다.
      - ./data/postgres:/var/lib/postgresql/data
    healthcheck:
      # DB 가 "접속 받을 준비가 됐는지" 5초마다 확인한다.
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 10

  # ---------- FastAPI 서버 ----------
  app:
    build: .
    image: harness-crud-lab-dev
    # --reload: app 폴더의 파일을 고치면 서버가 저절로 다시 켜진다.
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload --reload-dir app
    environment:
      # 접속 주소의 호스트 이름이 localhost 가 아니라 "db"(위 서비스 이름)인 점에 주목!
      DATABASE_URL: postgresql://${POSTGRES_USER:-memo}:${POSTGRES_PASSWORD:-memo1234}@db:5432/${POSTGRES_DB:-memo_db
}
      # 윈도우 폴더의 파일 변경은 컨테이너에 "알림"으로 전달되지 않아, 주기적으로 살펴보게 한다.
      WATCHFILES_FORCE_POLLING: "true"
      TZ: Asia/Seoul
    ports:
      - "8000:8000" # 내 PC 의 8000 번 ↔ 컨테이너의 8000 번
    volumes:
      # [바인드 마운트] C 드라이브의 프로젝트 폴더(이 파일이 있는 폴더) ↔ 컨테이너의 /workspace
      # → 윈도우에서 파일을 고치면 컨테이너 안에서도 바로 바뀐다.
      - .:/workspace
    depends_on:
      db:
        condition: service_healthy # DB 가 준비된 뒤에 서버를 켠다

  # ---------- 작업실 ----------
  dev:
    build: .
```

```
    image: harness-crud-lab-dev
    command: sleep infinity # 아무 일도 하지 않고 켜져 있다가, 우리가 들어가서 명령을 내린다
    environment:
      DATABASE_URL: postgresql://${POSTGRES_USER:-memo}:${POSTGRES_PASSWORD:-memo1234}@db:5432/${POSTGRES_DB:-memo_db
}
      # 테스트는 진짜 메모를 건드리지 않게 별도의 DB(memo_test)에서 돈다.
      TEST_DATABASE_URL: postgresql://${POSTGRES_USER:-memo}:${POSTGRES_PASSWORD:-memo1234}@db:5432/memo_test
      # OpenRouter 키와 모델은 .env 파일에서 읽어 온다. (키를 이 파일에 직접 적지 않는다!)
      OPENROUTER_API_KEY: ${OPENROUTER_API_KEY:-}
      OPENROUTER_MODEL: ${OPENROUTER_MODEL:-anthropic/claude-haiku-5.5}
      TZ: Asia/Seoul
    volumes:
      - .:/workspace # app 과 같은 폴더를 본다
    depends_on:
      db:
        condition: service_healthy
```

Dockerfile

```
# ============================================================
# Dockerfile - 이 과정에서 쓰는 "도구 상자" 이미지의 설계도
#
# 내 PC 에는 Docker Desktop 만 설치한다.
# Python, FastAPI, pytest, AI 에이전트(OpenCode)는 모두 이 이미지 안에 들어 있다.
# → 누구의 PC 에서나 똑같은 환경이 만들어진다.
# ============================================================

# 1) 파이썬 3.12 가 설치된 작은 리눅스에서 시작한다.
FROM python:3.12-slim

# 2) 파이썬 동작 방식 몇 가지를 정한다.
#    - PYTHONDONTWRITEBYTECODE: __pycache__ 폴더를 만들지 않는다(윈도우 폴더가 지저분해지지 않게)
#    - PYTHONUNBUFFERED: print 한 내용이 바로바로 화면에 보이게 한다
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1

# 3) 리눅스 기본 도구 설치: git(저장), curl(내려받기), ripgrep(에이전트가 파일을 찾을 때 사용)
RUN apt-get update \
    && apt-get install -y --no-install-recommends bash ca-certificates curl git ripgrep tar \
    && rm -rf /var/lib/apt/lists/*

# 4) AI 에이전트 OpenCode 설치 (공식 설치 스크립트, 버전 고정)
#    버전을 고정하는 이유: 수업 도중에 도구가 저절로 바뀌어 화면이 달라지는 일을 막는다.
ARG OPENCODE_VERSION=2.2.6
RUN curl -fsSL https://opencode.ai/v2/install | bash -s -- --version ${OPENCODE_VERSION} --no-modify-path
ENV PATH="/root/.opencode/bin:${PATH}"

# 5) 컨테이너 안의 git 설정
#    - safe.directory: 윈도우에서 연결(바인드 마운트)한 폴더를 git 이 믿고 쓰게 한다
#    - core.fileMode false: 윈도우 폴더의 "실행 권한" 차이를 변경으로 보지 않게 한다
RUN git config --system --add safe.directory /workspace \
    && git config --system core.fileMode false \
    && git config --system init.defaultBranch main

# 6) 작업 폴더. docker-compose.yml 이 여기에 C 드라이브의 프로젝트 폴더를 연결한다.
WORKDIR /workspace

# 7) 파이썬 패키지 설치 (목록은 requirements.txt)
COPY requirements.txt /tmp/requirements.txt
RUN pip install -r /tmp/requirements.txt

# 8) 따로 명령을 주지 않으면 셸을 연다.
CMD ["bash"]
```

.env.example

```
# ============================================================
# .env.example - 환경 변수 "견본" 파일
#
# 사용법: 이 파일을 복사해 .env 라는 이름으로 만들고, 값을 채운다.
#   PowerShell>  Copy-Item .env.example .env
#
# .env 는 .gitignore 에 들어 있어 GitHub 에 올라가지 않는다.
# 이 견본 파일(.env.example)에는 진짜 키를 절대 적지 않는다.
# ============================================================

# OpenRouter API 키 (https://openrouter.ai/keys 에서 발급, sk-or-v1- 로 시작)
OPENROUTER_API_KEY=

# scripts/ask_ai.py 가 사용할 모델. (에이전트 OpenCode 의 모델은 opencode.jsonc 에서 정한다)
OPENROUTER_MODEL=anthropic/claude-haiku-5.5

# PostgreSQL 계정 (실습용 값. 그대로 두면 된다)
POSTGRES_USER=memo
POSTGRES_PASSWORD=memo1234
POSTGRES_DB=memo_db
```

scripts/verify.sh

```
#!/bin/sh
# ============================================================
# verify.sh - 검증 스크립트 (하네스의 "검증" 담당)
#
# 사람이든 AI 든, 코드를 고친 뒤에는 반드시 이 스크립트를 통과해야 한다.
# 통과하지 못하면 Git 에 저장(commit)할 수 없다. (.githooks/pre-commit 이 막는다)
#
# 실행 (dev 컨테이너 안에서):  sh scripts/verify.sh
# ============================================================

# 명령 하나라도 실패하면 그 자리에서 멈춘다.
set -e

echo "[1/2] 코드 검사 (ruff) ----------------------------"
# 오타, 안 쓰는 import, 정의되지 않은 이름 같은 실수를 찾는다.
ruff check app tests scripts

echo "[2/2] 자동 테스트 (pytest) ------------------------"
# tests 폴더의 테스트를 모두 실행한다. 하나라도 실패하면 여기서 멈춘다.
pytest -q

echo "------------------------------------------------------"
echo "검증 통과: 이제 저장(commit)해도 됩니다."
```

.githooks/pre-commit

```
#!/bin/sh
# ============================================================
# pre-commit - "커밋 직전에" git 이 자동으로 실행하는 스크립트 (하네스의 마지막 문)
#
# 검증(scripts/verify.sh)이 실패하면 커밋 자체가 취소된다.
# → "테스트를 통과한 코드만 Git 에 저장된다"는 규칙을 사람의 기억이 아니라 도구가 지킨다.
#
# 켜는 법(5교시에 한 번만):  git config core.hooksPath .githooks
# ============================================================

echo "[pre-commit] 저장하기 전에 검증을 실행합니다..."

if [ -f /.dockerenv ] || [ -n "$HARNESS_LOCAL_VERIFY" ]; then
    # dev 컨테이너 안에서 커밋하는 경우: 바로 검증한다.
    # (HARNESS_LOCAL_VERIFY=1 은 PC 에 파이썬을 직접 설치해 쓰는 사람을 위한 스위치다)
    sh scripts/verify.sh
else
    # 윈도우(컨테이너 밖)에서 커밋하는 경우: dev 컨테이너에게 검증을 시킨다.
    docker compose exec -T dev sh scripts/verify.sh
fi
result=$?

if [ $result -ne 0 ]; then
    echo ""
    echo "[pre-commit] 검증 실패: 커밋을 취소했습니다."
    echo "[pre-commit] 위의 오류를 고친 뒤 다시 커밋하세요."
    exit 1
fi
```

scripts/ask_ai.py

```
"""
ask_ai.py - OpenRouter 를 통해 AI 모델에게 질문 하나를 보내고 답을 출력한다. (2교시)

이 파일이 보여 주는 개념
  1) AI 모델을 "부른다"는 것은 결국 인터넷 주소로 HTTP 요청(POST)을 보내는 일이다.
  2) API 키는 코드에 적지 않고 환경 변수(OPENROUTER_API_KEY)에서 읽는다.
  3) 이 스크립트는 "질문 → 답" 한 번으로 끝난다. 파일을 고치거나 명령을 실행하지 못한다.
     (그 일까지 하는 프로그램이 "에이전트"다. 이 과정에서는 OpenCode 를 쓴다.)

사용법 (dev 컨테이너 안에서)
  python scripts/ask_ai.py "FastAPI 가 뭐야? 두 문장으로 알려 줘"
  python scripts/ask_ai.py --file app/main.py "이 코드를 한 줄씩 설명해 줘"
  python scripts/ask_ai.py --model google/gemma-4-31b-it:free "안녕"
"""

import argparse
import os
import sys
from pathlib import Path

import httpx

# OpenRouter 의 채팅 주소. 어떤 회사의 모델이든 이 주소 하나로 부른다.
API_URL = "https://openrouter.ai/api/v1/chat/completions"
DEFAULT_MODEL = "anthropic/claude-haiku-5.5"

# 상태 코드별 안내 문구 (자주 만나는 것만)
ERROR_HINTS = {
    401: "API 키가 틀렸습니다. .env 의 OPENROUTER_API_KEY 값을 확인하세요.",
    402: "크레딧이 부족합니다. https://openrouter.ai/credits 에서 잔액을 확인하세요.",
    404: "모델 이름이 틀렸습니다. https://openrouter.ai/models 에서 이름을 확인하세요.",
    429: "요청이 너무 많습니다(사용 한도). 잠시 뒤에 다시 시도하세요.",
}


def build_prompt(question: str, file_path: str | None) -> str:
    """질문 글을 만든다. --file 을 줬으면 파일 내용을 질문 아래에 붙인다."""
    if file_path is None:
        return question
    text = Path(file_path).read_text(encoding="utf-8")
    return f"{question}\n\n--- {file_path} ---\n{text}"


def ask(prompt: str, model: str, api_key: str) -> dict:
    """OpenRouter 에 요청을 보내고, 응답(JSON)을 딕셔너리로 돌려준다."""
    response = httpx.post(
        API_URL,
        # "나는 이 키의 주인입니다"라고 알리는 부분
        headers={"Authorization": f"Bearer {api_key}"},
        json={
            "model": model,  # 어떤 모델에게 물을지
            "messages": [
                # system: AI 에게 주는 기본 지시 / user: 내가 하는 질문
                {"role": "system", "content": "초보자에게 설명하듯 쉬운 한국어로 답한다."},
                {"role": "user", "content": prompt},
            ],
        },
        timeout=120,  # 답이 올 때까지 최대 120초 기다린다
    )
    if response.status_code != 200:
        hint = ERROR_HINTS.get(response.status_code, "응답 내용을 읽고 원인을 찾아보세요.")
        print(f"[실패] 상태 코드 {response.status_code}: {hint}")
```

```
        print(response.text[:500])
        sys.exit(1)
    return response.json()


def main() -> None:
    # 명령줄에서 받을 값들을 정한다.
    parser = argparse.ArgumentParser(description="OpenRouter 로 AI 모델에게 질문하기")
    parser.add_argument("question", help="AI 에게 할 질문")
    parser.add_argument("--file", help="질문과 함께 보낼 파일 경로")
    parser.add_argument("--model", help="사용할 모델 이름 (기본: .env 의 OPENROUTER_MODEL)")
    args = parser.parse_args()

    # 키와 모델을 환경 변수에서 읽는다. (.env → docker-compose.yml → 컨테이너의 환경 변수)
    api_key = os.environ.get("OPENROUTER_API_KEY", "")
    model = args.model or os.environ.get("OPENROUTER_MODEL") or DEFAULT_MODEL
    if not api_key:
        print("[실패] OPENROUTER_API_KEY 가 비어 있습니다.")
        print("  1) .env 파일에 키를 적었는지 확인하세요.")
        print("  2) 컨테이너를 다시 만드세요: docker compose up -d  (윈도우 PowerShell 에서)")
        sys.exit(1)

    data = ask(build_prompt(args.question, args.file), model, api_key)

    # 응답에서 필요한 부분만 꺼내 보여 준다.
    print(f"[모델] {data.get('model', model)}")
    print("[답변]")
    print(data["choices"][0]["message"]["content"])
    usage = data.get("usage", {})
    # 토큰: AI 가 글을 세는 단위. 보낸 양(prompt)과 받은 양(completion)에 따라 요금이 정해진다.
    print(f"[사용량] 보낸 토큰 {usage.get('prompt_tokens')} / 받은 토큰 {usage.get('completion_tokens')}")


if __name__ == "__main__":
    main()
```

부록 C. 완성본 코드

checkpoints/lesson11/ 의 코드다. AI 가 만든 내 코드와 똑같을 필요는 없다. 같은 요구사항을 만족하는 한 가지 예로 읽는다.

app/schemas.py

```
"""
schemas.py - 주고받는 데이터의 "모양"을 정해 둔 파일

이 파일이 보여 주는 개념
  1) 요청(클라이언트 → 서버)과 응답(서버 → 클라이언트)의 모양을 클래스로 적는다.
  2) 모양에 맞지 않는 요청은 FastAPI 가 알아서 422 오류로 돌려보낸다.
     → 요구사항 명세의 "입력 규칙"이 이 파일에 코드로 옮겨져 있다.
"""

from datetime import datetime

from pydantic import BaseModel, Field, field_validator


class MemoIn(BaseModel):
    """메모를 만들거나 고칠 때 클라이언트가 보내는 데이터"""

    # 제목: 반드시 있어야 하고 1~100자
    title: str = Field(min_length=1, max_length=100)
    # 내용: 안 보내면 빈 문자열, 최대 1000자
    content: str = Field(default="", max_length=1000)

    @field_validator("title")
    @classmethod
    def title_must_not_be_blank(cls, value: str) -> str:
        """11교시에 추가: 공백만 있는 제목("   ")은 받지 않는다."""
        stripped = value.strip()  # 앞뒤 공백을 잘라 낸다
        if not stripped:
            raise ValueError("제목은 공백만으로 채울 수 없습니다.")
        return stripped  # 잘라 낸 값으로 저장한다


class MemoOut(BaseModel):
    """서버가 돌려주는 메모 한 건"""

    id: int
    title: str
    content: str
    created_at: datetime
    updated_at: datetime
```

app/db.py

```
"""
db.py - PostgreSQL 과 대화하는 부분만 모아 둔 파일

이 파일이 보여 주는 개념
  1) 접속 주소(DATABASE_URL)는 코드에 적지 않고 "환경 변수"에서 읽는다.
  2) 연결(connection)은 필요할 때 열고, 일이 끝나면 바로 닫는다.
  3) 테이블은 서버가 켜질 때 "없으면 만든다"(CREATE TABLE IF NOT EXISTS).
"""

import os

import psycopg
from psycopg.rows import dict_row

# 7교시에 psql 로 직접 쳐 본 것과 같은 SQL 이다. 코드 안에 그대로 들어 있다.
CREATE_TABLE_SQL = """
CREATE TABLE IF NOT EXISTS memos (
    id          SERIAL PRIMARY KEY,                  -- 1, 2, 3 ... 자동으로 붙는 번호
    title       VARCHAR(100) NOT NULL,               -- 제목(최대 100자, 비어 있을 수 없음)
    content     TEXT NOT NULL DEFAULT '',            -- 내용(없으면 빈 문자열)
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  -- 만든 시각
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()   -- 마지막으로 고친 시각
);
"""


def get_database_url() -> str:
    """환경 변수에서 접속 주소를 읽는다. (docker-compose.yml 이 넣어 준다)"""
    # 함수 안에서 읽는 이유: 테스트가 주소를 테스트용 DB 로 바꿔 끼울 수 있게 하려고.
    return os.environ["DATABASE_URL"]


def get_conn() -> psycopg.Connection:
    """DB 연결을 하나 연다. 결과 행은 {"열이름": 값} 딕셔너리로 받는다."""
    return psycopg.connect(get_database_url(), row_factory=dict_row)


def init_db() -> None:
    """memos 테이블이 없으면 만든다. 이미 있으면 아무 일도 하지 않는다."""
    # with 블록을 빠져나올 때 자동으로 저장(commit)하고 연결을 닫는다.
    with get_conn() as conn:
        conn.execute(CREATE_TABLE_SQL)
```

app/main.py

```
"""
main.py - 메모 CRUD API (완성본)

이 파일이 보여 주는 개념
  1) 주소(URL) + 방식(GET/POST/PUT/DELETE) 하나가 함수 하나에 연결된다.
  2) CRUD 네 가지가 SQL 네 가지(INSERT/SELECT/UPDATE/DELETE)와 짝을 이룬다.
  3) 없는 메모를 찾으면 404, 입력이 잘못되면 422 를 돌려준다.

주소 목록
  GET    /health        서버가 살아 있는지 확인
  GET    /hello         인사말 (3교시)
  POST   /memos         메모 만들기      (Create)
  GET    /memos         메모 목록 보기   (Read)
  GET    /memos/{id}    메모 한 건 보기  (Read)
  PUT    /memos/{id}    메모 고치기      (Update)
  DELETE /memos/{id}    메모 지우기      (Delete)
  GET    /              웹 화면 (10교시)
"""

from contextlib import asynccontextmanager
from pathlib import Path

from fastapi import FastAPI, HTTPException
from fastapi.responses import FileResponse
from fastapi.staticfiles import StaticFiles

from app.db import get_conn, init_db
from app.schemas import MemoIn, MemoOut

# 화면 파일(index.html, app.js, style.css)이 들어 있는 폴더
STATIC_DIR = Path(__file__).parent / "static"


@asynccontextmanager
async def lifespan(app: FastAPI):
    """서버가 켜질 때 한 번 실행된다: 테이블이 없으면 만든다."""
    init_db()
    yield  # 여기서부터 서버가 요청을 받는다


app = FastAPI(title="메모 CRUD", lifespan=lifespan)


def not_found() -> HTTPException:
    """'그 번호의 메모가 없다'는 404 오류를 한 곳에서 만든다."""
    return HTTPException(status_code=404, detail="메모를 찾을 수 없습니다.")


@app.get("/health")
def health() -> dict:
    """서버가 살아 있는지 확인하는 주소"""
    return {"status": "ok"}


@app.get("/hello")
def hello(name: str = "하네스") -> dict:
    """/hello?name=철수 처럼 부르면 이름을 넣어 인사한다."""
    return {"message": f"안녕하세요, {name}!"}


# ---------- Create: 메모 만들기 ----------
@app.post("/memos", response_model=MemoOut, status_code=201)
```

```
def create_memo(memo: MemoIn) -> dict:
    with get_conn() as conn:
        # %s 자리에 값이 안전하게 들어간다(SQL 을 문자열로 이어 붙이지 않는다).
        row = conn.execute(
            "INSERT INTO memos (title, content) VALUES (%s, %s) RETURNING *",
            (memo.title, memo.content),
        ).fetchone()
    return row


# ---------- Read: 목록 보기 ----------
@app.get("/memos", response_model=list[MemoOut])
def list_memos() -> list[dict]:
    with get_conn() as conn:
        # 최신 메모가 위에 오도록 id 를 거꾸로 정렬한다.
        return conn.execute("SELECT * FROM memos ORDER BY id DESC").fetchall()


# ---------- Read: 한 건 보기 ----------
@app.get("/memos/{memo_id}", response_model=MemoOut)
def get_memo(memo_id: int) -> dict:
    with get_conn() as conn:
        row = conn.execute("SELECT * FROM memos WHERE id = %s", (memo_id,)).fetchone()
    if row is None:  # 그런 번호가 없으면
        raise not_found()
    return row


# ---------- Update: 고치기 ----------
@app.put("/memos/{memo_id}", response_model=MemoOut)
def update_memo(memo_id: int, memo: MemoIn) -> dict:
    with get_conn() as conn:
        # WHERE 가 없으면 모든 메모가 바뀐다. 반드시 id 로 한 건만 고른다.
        row = conn.execute(
            "UPDATE memos SET title = %s, content = %s, updated_at = now() "
            "WHERE id = %s RETURNING *",
            (memo.title, memo.content, memo_id),
        ).fetchone()
    if row is None:
        raise not_found()
    return row


# ---------- Delete: 지우기 ----------
@app.delete("/memos/{memo_id}", status_code=204)
def delete_memo(memo_id: int) -> None:
    with get_conn() as conn:
        row = conn.execute(
            "DELETE FROM memos WHERE id = %s RETURNING id", (memo_id,)
        ).fetchone()
    if row is None:
        raise not_found()
    # 204 는 "성공했고, 돌려줄 내용은 없음"이라는 뜻이라 아무것도 돌려주지 않는다.


# ---------- 웹 화면 ----------
@app.get("/", include_in_schema=False)
def index() -> FileResponse:
    """브라우저로 http://localhost:8000 에 들어오면 화면 파일을 보낸다."""
    return FileResponse(STATIC_DIR / "index.html")


# /static/app.js, /static/style.css 같은 파일을 그대로 내려 준다.
app.mount("/static", StaticFiles(directory=STATIC_DIR), name="static")
```

app/static/index.html

```
<!DOCTYPE html>
<!--
  index.html - 메모장 화면 (10교시)

  이 파일이 보여 주는 개념
    1) HTML 은 화면의 "뼈대"만 만든다. (입력 칸, 버튼, 목록 자리)
    2) 실제 데이터는 app.js 가 API(/memos)를 불러서 채운다.
-->
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>메모장</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
  <main>
    <h1>메모장</h1>

    <!-- 메모를 쓰는 폼. 수정할 때도 이 폼을 다시 쓴다. -->
    <form id="memo-form">
      <input id="title" type="text" placeholder="제목 (필수, 100자까지)" maxlength="100" required>
      <textarea id="content" placeholder="내용 (1000자까지)" maxlength="1000" rows="3"></textarea>
      <div class="buttons">
        <button id="save-button" type="submit">저장</button>
        <button id="cancel-button" type="button" hidden>수정 취소</button>
      </div>
    </form>

    <!-- 오류나 안내 문구가 나오는 자리 -->
    <p id="message" role="status"></p>

    <!-- 메모 목록이 들어가는 자리. app.js 가 채운다. -->
    <ul id="memo-list"></ul>
  </main>

  <script src="/static/app.js"></script>
</body>
</html>
```

app/static/app.js

```
/*
  app.js - 화면과 API 를 이어 주는 자바스크립트 (10교시)

  이 파일이 보여 주는 개념
    1) fetch() 로 서버 API 를 부른다. /docs 에서 손으로 하던 일을 코드가 대신한다.
    2) 화면의 버튼 하나가 CRUD 하나에 연결된다.
         저장 버튼  → POST /memos   (수정 중이면 PUT /memos/{id})
         화면 열기  → GET /memos
         삭제 버튼  → DELETE /memos/{id}
    3) 사용자가 쓴 글은 textContent 로 넣는다. (innerHTML 로 넣으면 글 속의 태그가 실행될 수 있다)
*/

const form = document.getElementById("memo-form");
const titleInput = document.getElementById("title");
const contentInput = document.getElementById("content");
const saveButton = document.getElementById("save-button");
const cancelButton = document.getElementById("cancel-button");
const message = document.getElementById("message");
const memoList = document.getElementById("memo-list");

// 지금 수정 중인 메모 번호. null 이면 "새 메모 쓰는 중"이라는 뜻이다.
let editingId = null;

// 안내 문구를 화면에 보여 준다.
function showMessage(text) {
  message.textContent = text;
}

// 서버가 오류를 돌려줬을 때, 사람이 읽을 문장으로 바꾼다.
async function readError(response) {
  const body = await response.json().catch(() => null);
  if (body && typeof body.detail === "string") return body.detail; // 404 등
  if (body && Array.isArray(body.detail)) return "입력 값을 확인해 주세요. (제목은 1~100자)"; // 422
  return `요청에 실패했습니다. (상태 코드 ${response.status})`;
}

// ---------- Read: 목록을 불러와 화면에 그린다 ----------
async function loadMemos() {
  const response = await fetch("/memos"); // GET 은 기본값이라 따로 적지 않는다
  const memos = await response.json();

  memoList.replaceChildren(); // 이전 목록을 비운다
  for (const memo of memos) {
    memoList.appendChild(makeMemoItem(memo));
  }
  if (memos.length === 0) showMessage("아직 메모가 없습니다. 첫 메모를 써 보세요.");
}

// 메모 한 건을 <li> 로 만든다.
function makeMemoItem(memo) {
  const item = document.createElement("li");

  const title = document.createElement("strong");
  title.textContent = memo.title; // textContent: 글자를 글자 그대로 넣는다

  const content = document.createElement("p");
  content.textContent = memo.content;

  const time = document.createElement("small");
  time.textContent = `#${memo.id} · ${new Date(memo.updated_at).toLocaleString("ko-KR")}`;

  const editButton = document.createElement("button");
```

```
  editButton.textContent = "수정";
  editButton.addEventListener("click", () => startEdit(memo));

  const deleteButton = document.createElement("button");
  deleteButton.textContent = "삭제";
  deleteButton.addEventListener("click", () => deleteMemo(memo.id));

  item.append(title, content, time, editButton, deleteButton);
  return item;
}

// ---------- Create / Update: 저장 버튼 ----------
form.addEventListener("submit", async (event) => {
  event.preventDefault(); // 폼이 페이지를 새로 고치지 않게 막는다

  const body = { title: titleInput.value, content: contentInput.value };
  // 수정 중이면 PUT, 아니면 POST
  const url = editingId === null ? "/memos" : `/memos/${editingId}`;
  const method = editingId === null ? "POST" : "PUT";

  const response = await fetch(url, {
    method,
    headers: { "Content-Type": "application/json" }, // "JSON 을 보냅니다"라고 알린다
    body: JSON.stringify(body), // 자바스크립트 객체 → JSON 글자
  });

  if (!response.ok) {
    showMessage(await readError(response));
    return;
  }
  showMessage(editingId === null ? "저장했습니다." : "수정했습니다.");
  stopEdit();
  await loadMemos(); // 목록을 다시 불러와 방금 바뀐 내용을 보여 준다
});

// 수정 시작: 폼에 기존 내용을 채운다.
function startEdit(memo) {
  editingId = memo.id;
  titleInput.value = memo.title;
  contentInput.value = memo.content;
  saveButton.textContent = "수정 저장";
  cancelButton.hidden = false;
  titleInput.focus();
}

// 수정 끝: 폼을 비우고 "새 메모" 상태로 되돌린다.
function stopEdit() {
  editingId = null;
  form.reset();
  saveButton.textContent = "저장";
  cancelButton.hidden = true;
}

cancelButton.addEventListener("click", stopEdit);

// ---------- Delete: 삭제 버튼 ----------
async function deleteMemo(id) {
  if (!confirm(`#${id} 메모를 지울까요?`)) return; // 되돌릴 수 없으니 한 번 묻는다

  const response = await fetch(`/memos/${id}`, { method: "DELETE" });
  if (!response.ok) {
    showMessage(await readError(response));
    return;
  }
```

```
  showMessage("삭제했습니다.");
  if (editingId === id) stopEdit(); // 수정하던 메모를 지웠다면 폼도 비운다
  await loadMemos();
}

// 화면이 열리면 목록부터 불러온다.
loadMemos();
```

tests/conftest.py

```
"""
conftest.py - 모든 테스트가 함께 쓰는 준비물

이 파일이 보여 주는 개념
  1) 테스트는 "테스트 전용 DB"(memo_test)에서만 돈다. 진짜 메모(memo_db)는 건드리지 않는다.
  2) 테스트 하나가 시작될 때마다 테이블을 비운다. 그래야 테스트끼리 서로 영향을 주지 않는다.
  3) TestClient 는 서버를 실제로 띄우지 않고도 API 를 불러 볼 수 있게 해 준다.
"""

import os

import psycopg
import pytest
from fastapi.testclient import TestClient
from psycopg.conninfo import conninfo_to_dict

# docker-compose.yml 이 dev 컨테이너에 넣어 준 두 주소
MAIN_URL = os.environ["DATABASE_URL"]  # 평소에 쓰는 DB (memo_db)
TEST_URL = os.environ["TEST_DATABASE_URL"]  # 테스트 전용 DB (memo_test)

# 앱이 테스트 전용 DB 를 보게 주소를 바꿔 끼운다. (app 을 불러오기 전에 해야 한다)
os.environ["DATABASE_URL"] = TEST_URL

from app.db import get_conn
from app.main import app


@pytest.fixture(scope="session", autouse=True)
def create_test_database():
    """테스트 전용 DB 가 없으면 만든다. 테스트 전체에서 딱 한 번 실행된다."""
    test_db_name = conninfo_to_dict(TEST_URL)["dbname"]
    # CREATE DATABASE 는 autocommit 상태에서만 실행할 수 있다.
    with psycopg.connect(MAIN_URL, autocommit=True) as conn:
        exists = conn.execute(
            "SELECT 1 FROM pg_database WHERE datname = %s", (test_db_name,)
        ).fetchone()
        if not exists:
            conn.execute(f'CREATE DATABASE "{test_db_name}"')


@pytest.fixture
def client():
    """테스트 함수가 인자로 client 를 받으면, 깨끗한 DB 와 연결된 클라이언트를 준다."""
    # with 를 쓰면 서버가 켜질 때 하는 일(lifespan → 테이블 만들기)이 함께 실행된다.
    with TestClient(app) as test_client:
        with get_conn() as conn:
            # 메모를 모두 지우고 번호도 1번부터 다시 시작하게 한다.
            conn.execute("TRUNCATE TABLE memos RESTART IDENTITY")
        yield test_client
```

tests/test_memos.py

```
"""
test_memos.py - 메모 CRUD 테스트

이 파일이 보여 주는 개념
  1) 요구사항 명세(harness/REQUIREMENTS.md)의 번호(R1, R2 ...)마다 테스트가 있다.
     → "요구사항을 지켰는가?"를 사람이 눈으로 보지 않고 명령 한 줄로 확인한다.
  2) 정상 경우뿐 아니라 "잘못된 경우"(없는 번호, 빈 제목)도 테스트한다.
"""


def make_memo(client, title="장보기", content="우유, 달걀"):
    """테스트에서 자주 쓰는 '메모 한 건 만들기'를 함수로 빼 두었다."""
    response = client.post("/memos", json={"title": title, "content": content})
    assert response.status_code == 201
    return response.json()


# ---------- R1. 메모 만들기 ----------
def test_create_memo(client):
    memo = make_memo(client)
    assert memo["id"] == 1  # 첫 메모의 번호는 1
    assert memo["title"] == "장보기"
    assert memo["content"] == "우유, 달걀"
    assert "created_at" in memo and "updated_at" in memo  # 시각이 함께 온다


def test_create_memo_without_content(client):
    # 내용을 안 보내면 빈 문자열로 저장된다.
    response = client.post("/memos", json={"title": "제목만"})
    assert response.status_code == 201
    assert response.json()["content"] == ""


# ---------- R2. 목록 보기 ----------
def test_list_memos_empty(client):
    response = client.get("/memos")
    assert response.status_code == 200
    assert response.json() == []  # 메모가 없으면 빈 목록


def test_list_memos_newest_first(client):
    make_memo(client, title="첫째")
    make_memo(client, title="둘째")
    titles = [memo["title"] for memo in client.get("/memos").json()]
    assert titles == ["둘째", "첫째"]  # 나중에 쓴 메모가 위에 온다


# ---------- R3. 한 건 보기 ----------
def test_get_memo(client):
    created = make_memo(client)
    response = client.get(f"/memos/{created['id']}")
    assert response.status_code == 200
    assert response.json()["title"] == "장보기"


def test_get_memo_not_found(client):
    response = client.get("/memos/999")  # 없는 번호
    assert response.status_code == 404
    assert response.json() == {"detail": "메모를 찾을 수 없습니다."}


# ---------- R4. 고치기 ----------
```

```
def test_update_memo(client):
    created = make_memo(client)
    response = client.put(
        f"/memos/{created['id']}", json={"title": "장보기(수정)", "content": "우유"}
    )
    assert response.status_code == 200
    assert response.json()["title"] == "장보기(수정)"
    # 다시 조회해도 바뀐 내용이 남아 있어야 한다(진짜로 저장되었는지 확인).
    assert client.get(f"/memos/{created['id']}").json()["content"] == "우유"


def test_update_memo_not_found(client):
    response = client.put("/memos/999", json={"title": "없음", "content": ""})
    assert response.status_code == 404


def test_update_memo_empty_title(client):
    # 고칠 때도 입력 규칙(R6)이 똑같이 적용된다.
    created = make_memo(client)
    response = client.put(f"/memos/{created['id']}", json={"title": "", "content": ""})
    assert response.status_code == 422
    # 실패한 수정은 저장되지 않아야 한다.
    assert client.get(f"/memos/{created['id']}").json()["title"] == "장보기"


# ---------- R5. 지우기 ----------
def test_delete_memo(client):
    created = make_memo(client)
    response = client.delete(f"/memos/{created['id']}")
    assert response.status_code == 204  # 성공했고 돌려줄 내용은 없음
    # 지운 뒤에는 조회되지 않아야 한다.
    assert client.get(f"/memos/{created['id']}").status_code == 404


def test_delete_memo_not_found(client):
    assert client.delete("/memos/999").status_code == 404


# ---------- R6. 입력 규칙 ----------
def test_create_memo_empty_title(client):
    response = client.post("/memos", json={"title": "", "content": "제목 없음"})
    assert response.status_code == 422  # 입력이 규칙에 맞지 않음


def test_create_memo_title_too_long(client):
    response = client.post("/memos", json={"title": "가" * 101})
    assert response.status_code == 422  # 제목은 100자까지


def test_create_memo_content_too_long(client):
    response = client.post("/memos", json={"title": "긴 내용", "content": "가" * 1001})
    assert response.status_code == 422  # 내용은 1000자까지


def test_create_memo_blank_title(client):
    # 11교시에 추가한 테스트: 공백만 있는 제목은 받지 않는다.
    response = client.post("/memos", json={"title": "   ", "content": "공백 제목"})
    assert response.status_code == 422


def test_title_is_trimmed(client):
    # 11교시에 추가한 테스트: 제목 앞뒤 공백은 잘라서 저장한다.
    response = client.post("/memos", json={"title": "  장보기  ", "content": ""})
    assert response.status_code == 201
```

```
    assert response.json()["title"] == "장보기"


# ---------- R7. 웹 화면 ----------
def test_index_page(client):
    response = client.get("/")
    assert response.status_code == 200
    assert "메모장" in response.text  # 화면에 '메모장'이라는 글자가 있다
```

부록 D. 자주 쓰는 명령 모음

명령을 **어디에서 치는지**가 가장 중요하다. 표시를 먼저 확인한다.

표시	어디	프롬프트 모양	들어가는 법
[PS]	윈도우 PowerShell	`PS C:\harness-crud-lab>`	시작 메뉴 → PowerShell → `cd C:\harness-crud-lab`
[dev]	dev 컨테이너의 셸	`root@…:/workspace#`	**[PS]** `docker compose exec dev bash`
[psql]	PostgreSQL 접속 상태	`memo_db=#`	**[PS]** `docker compose exec db psql -U memo -d memo_db`
[AI]	OpenCode 입력 칸	(OpenCode 화면)	**[dev]** `opencode`

1. 하루의 시작과 끝 — [PS]

하려는 일	명령
수업 시작 (컨테이너 켜기)	`docker compose up -d`
상태 보기	`docker compose ps`
작업실 들어가기	`docker compose exec dev bash`
수업 끝 (멈추기. 지우지 않는다)	`docker compose stop`
멈춘 것 다시 켜기	`docker compose start`

PC 를 다시 켠 뒤에는 Docker Desktop 을 먼저 실행하고 "Engine running" 을 기다린다.

2. 컨테이너 다루기 — [PS]

하려는 일	명령
처음 만들기, `Dockerfile`·`requirements.txt` 를 고친 뒤	`docker compose up -d --build`
`.env` 를 고친 뒤	`docker compose up -d` (바뀐 컨테이너를 다시 만든다)
서버만 다시 켜기	`docker compose restart app`
서버 로그 마지막 30줄	`docker compose logs app --tail 30`
서버 로그 계속 보기 (Ctrl+C 로 멈춤)	`docker compose logs app -f`
DB 로그	`docker compose logs db --tail 30`
컨테이너 지우기 (소스와 DB 데이터는 C 드라이브에 남는다)	`docker compose down`
바인드 마운트 확인	`docker inspect harness-crud-lab-dev-1 --format "{{ json .Mounts }}"`

3. 검증 — [dev]

하려는 일	명령
전체 검증 (코드 검사 + 테스트)	`sh scripts/verify.sh`
테스트만	`pytest -q`
테스트 이름까지 보기	`pytest -v`
테스트 하나만	`pytest -q tests/test_memos.py::test_create_memo`
이름에 낱말이 든 테스트만	`pytest -q -k update`
코드 검사만	`ruff check app tests scripts`

4. API 불러 보기 — [dev]

컨테이너 안에서는 주소가 `http://app:8000` 이다. 브라우저에서는 `http://localhost:8000` 이다.

하려는 일	명령
목록 보기	`curl -s http://app:8000/memos`
한 건 보기	`curl -s http://app:8000/memos/1`
만들기	`curl -s -X POST http://app:8000/memos -H "Content-Type: application/json" -d '{"title": "장보기", "content": "우유"}'`
고치기	`curl -s -X PUT http://app:8000/memos/1 -H "Content-Type: application/json" -d '{"title": "새 제목", "content": "새 내용"}'`
지우기	`curl -s -X DELETE http://app:8000/memos/1`
상태 코드만 보기	위 명령에 `-o /dev/null -w "%{http_code}\n"` 를 붙인다

브라우저에서 눌러 보기: http://localhost:8000/docs

5. Git — [dev]

하려는 일	명령
무엇이 바뀌었나 (파일 목록)	`git status`
무엇이 바뀌었나 (파일별 줄 수)	`git diff --stat`
무엇이 바뀌었나 (내용)	`git diff` (`q` 로 나온다)
특정 폴더만	`git diff tests/`
테스트에서 지워진 줄만	`git diff tests/ | grep "^-" | grep -v "^---"`
저장할 파일 고르기	`git add -A` (전부) 또는 `git add 파일이름`
저장	`git commit -m "9교시: 무엇을 했는지 (요구사항 번호)"`
기록 보기	`git log --oneline`
커밋 하나에서 바뀐 파일	`git show --stat 커밋번호`
고친 것 버리기 (마지막 커밋 상태로)	`git restore .`
`git add` 한 것 취소	`git restore --staged 파일이름`
훅 켜기 (저장소마다 한 번)	`git config core.hooksPath .githooks`

`git restore .` 는 **커밋하지 않은 변경을 모두 버린다.** 되돌릴 수 없다. 새로 만들어진 파일은 지워지지 않으므로 `git status` 로 확인해 직접 지운다.

올리기(push)는 **[PS]** 에서 한다: `git push`

6. AI 부르기 — [dev]

하려는 일	명령
질문 하나 (파일을 고치지 못한다)	`python scripts/ask_ai.py "질문"`
파일을 붙여서 질문	`python scripts/ask_ai.py --file app/main.py "질문"`
다른 모델로 질문	`python scripts/ask_ai.py --model 모델이름 "질문"`
에이전트에게 한 번 시키기	`opencode run "시킬 일"`
에이전트 대화 화면	`opencode`
키가 전달되었나 (글자 수만)	`echo ${#OPENROUTER_API_KEY}`

7. OpenCode 안에서 — [AI]

입력	하는 일
`/exit`	끝낸다
`/new`	새 대화 (앞의 대화를 잊는다)
`/undo`	마지막 요청과 그 요청으로 바뀐 파일을 되돌린다
`/redo`	`/undo` 를 취소한다
`/models`	모델 목록
`@파일이름`	파일을 대화에 첨부한다

자주 쓰는 지시 문장

상황	문장
작업 시작	`harness/tasks/NN_….md` 를 읽고 그 지시대로 작업해 줘. 먼저 계획만 말하고 내 승인을 기다려.
계획 승인	진행
계획 수정	계획 N번은 지시서 4번(하지 말 것)에 어긋나. 빼고 다시 말해 줘.
설명 요청	파일은 고치지 말고 설명만 해 줘. (파일)의 (함수/줄)이 무슨 일을 해?
검증 실패	`sh scripts/verify.sh` 가 실패했어. 아래가 출력이야. 원인을 먼저 설명하고, `tests/` 는 건드리지 말고 고쳐 줘. + 출력 붙여넣기
범위 위반	`git diff --stat` 에 (파일)이 나와. 지시서 3번에서 고치면 안 된다고 했어. 되돌려 줘.

8. 데이터베이스 — [PS] 와 [psql]

하려는 일	명령
접속	**[PS]** `docker compose exec db psql -U memo -d memo_db`
접속하지 않고 SQL 한 줄	**[PS]** `docker compose exec db psql -U memo -d memo_db -c "SELECT id, title FROM memos ORDER BY id;"`
데이터베이스 목록	**[psql]** `\l`
다른 데이터베이스로	**[psql]** `\c practice`
테이블 목록	**[psql]** `\dt`
테이블 구조	**[psql]** `\d memos`
나가기	**[psql]** `\q`

SQL	하는 일
`SELECT * FROM memos ORDER BY id DESC;`	목록 (최신이 위)
`SELECT * FROM memos WHERE id = 1;`	한 건
`INSERT INTO memos (title, content) VALUES ('제목', '내용');`	만들기
`UPDATE memos SET title = '새 제목' WHERE id = 1;`	고치기 — **WHERE 필수**
`DELETE FROM memos WHERE id = 1;`	지우기 — **WHERE 필수**

9. 막혔을 때 — [dev]

하려는 일	명령
N교시 완성 예시 가져오기	`sh scripts/checkpoint.sh 08`
저장 안 한 변경이 있어도 가져오기	`sh scripts/checkpoint.sh 08 --force`
가져올 수 있는 목록 보기	`sh scripts/checkpoint.sh`
하네스 파일 다시 가져오기	`cp harness/AGENTS.md AGENTS.md` , `cp harness/opencode.jsonc opencode.jsonc`

체크포인트는 `app/` 과 `tests/` 를 **통째로 바꿔치기**한다. 내 작업이 사라지므로, 남기고 싶으면 먼저 커밋한다.

교시를 시작할 때 필요한 상태	가져올 체크포인트
4교시	`03`
5·6교시	`04`
7·8교시	`06`
9교시	`08`
10교시	`09`
11교시	`10`
12교시	`11`

부록 E. 용어 풀이

처음 나오는 교시를 함께 적었다. 낱말이 기억나지 않으면 그 교시로 돌아가 본다.

AI 와 하네스

용어	뜻	교시
AI 모델	글을 받아 글을 돌려주는 프로그램. 파일을 보거나 명령을 실행하지 못한다.	1
AI 에이전트	모델의 지시에 따라 파일을 읽고 고치고 명령을 실행하는 프로그램. 이 과정에서는 OpenCode.	1
하네스(harness)	에이전트가 어떤 규칙으로, 어떤 도구를, 어디까지 써서 일하고, 무엇을 통과해야 끝나는지 정해 둔 장치 전체. 원래 뜻은 말에게 씌우는 마구.	1
하네스 엔지니어링	AI 가 일하는 환경(규칙·도구·권한·실행·검증)을 설계하는 일.	1
바이브 코딩	AI 에게 말로 시키고, 돌아가는 것 같으면 코드를 읽지 않고 받아들이는 방식.	1
규칙	AI 에게 하는 부탁. `AGENTS.md` 에 적는다.	1, 5
권한	AI 가 할 수 있는 일의 범위를 정한 잠금장치. `opencode.jsonc` 의 `permissions`.	1, 5
allow / ask / deny	권한의 세 가지 처리. 그냥 허용 / 사람에게 물어봄 / 금지.	5
검증	결과가 기준을 만족하는지 확인하는 일. 이 과정에서는 `scripts/verify.sh`.	3, 5
요구사항 명세	무엇을 만들지를 "이렇게 부르면 → 이런 결과"로 적은 문서. `harness/REQUIREMENTS.md`.	1, 4
작업 지시서	AI 에게 일을 시킬 때 건네는 문서. 목표·요구사항·범위·하지 말 것·완료 조건·진행 방법.	6
범위	이번 작업에서 고쳐도 되는 파일과 고치면 안 되는 파일.	6
완료 조건	"다 됐다"고 말할 수 있는 조건. 확인할 수 있는 문장으로 쓴다.	6
OpenRouter	여러 회사의 AI 모델을 주소 하나, 키 하나로 부르게 해 주는 중개 서비스.	2
OpenCode	이 과정에서 쓰는 AI 에이전트. 터미널에서 실행한다.	2
API 키	"이 요청은 내 계정의 것"임을 증명하는 긴 문자열. 비밀번호처럼 다룬다.	2
토큰	AI 가 글을 세는 단위. 요금은 토큰 수로 계산한다.	2
프롬프트	AI 에게 보내는 글(질문이나 지시).	2

Docker

용어	뜻	교시
Docker	프로그램과 실행 환경을 상자(컨테이너)에 담아 실행하는 도구.	2
이미지	프로그램과 실행 환경을 담아 둔 틀. 바뀌지 않는다.	2
컨테이너	이미지로 만들어 실제로 돌아가는 것. 지우고 다시 만들 수 있다.	2
Dockerfile	이미지를 만드는 순서를 적은 파일.	2
docker compose	컨테이너 여러 개를 한꺼번에 켜고 끄는 도구. 설정은 `docker-compose.yml`.	2
서비스	`docker-compose.yml` 에 적은 컨테이너 하나의 이름(`db`, `app`, `dev`). 컨테이너끼리는 이 이름으로 서로를 부른다.	2, 4
바인드 마운트	내 PC 의 폴더를 컨테이너 안의 폴더에 그대로 연결하는 것. 복사가 아니라 같은 폴더를 함께 본다.	2
포트	한 컴퓨터 안에서 프로그램을 구분하는 번호. `8000:8000` 은 내 PC 의 8000 번을 컨테이너의 8000 번에 연결한다.	2
환경 변수	프로그램 밖에서 넣어 주는 설정 값. 접속 주소나 키를 코드에 적지 않으려고 쓴다.	2, 8
`.env`	환경 변수를 적어 두는 파일. Git 에 올리지 않는다.	2

파이썬과 FastAPI

용어	뜻	교시
변수	값에 붙인 이름.	3
문자열(str)	따옴표로 감싼 글자.	3
딕셔너리(dict)	`{"이름": 값}` 모양의 묶음.	3
리스트(list)	`[값, 값]` 모양의, 순서가 있는 묶음.	3
함수	이름을 붙인 일 묶음. `def` 로 만든다.	3
인자	함수에 넘겨주는 값.	3
import	다른 파일이나 패키지의 기능을 가져오는 것.	3
데코레이터	함수 위에 `@` 로 붙이는 이름표. `@app.get("/hello")`.	3
FastAPI	파이썬으로 웹 API 를 만드는 프레임워크.	3
uvicorn	FastAPI 앱을 실제로 실행해 주는 서버 프로그램.	3
쿼리 파라미터	주소 뒤에 `?이름=값` 으로 붙이는 값.	3
경로 파라미터	주소 안의 `{memo_id}` 자리에 들어가는 값. `/memos/7` 의 `7`.	6
Pydantic	데이터의 모양을 클래스로 정하고 검사해 주는 패키지.	4, 6
`Field`	항목의 규칙(최소·최대 길이, 기본값)을 적는 방법.	6
`response_model`	응답의 모양을 정하는 설정.	6
`HTTPException`	오류 응답(404 등)을 보내려고 일으키는 것.	6
`raise`	함수를 그 자리에서 끝내고 오류를 알린다.	6
`/docs`	FastAPI 가 코드를 읽고 자동으로 만들어 주는 설명서 겸 시험 화면.	3

HTTP

용어	뜻	교시
CRUD	Create(만들기)·Read(읽기)·Update(고치기)·Delete(지우기).	1, 4
API	프로그램끼리 정해진 방식으로 주고받는 창구.	3
HTTP	브라우저와 서버가 주고받는 말의 규칙.	4
요청 / 응답	클라이언트가 보내는 것 / 서버가 돌려주는 것.	4
방식(메서드)	무엇을 하려는지. GET(보기)·POST(만들기)·PUT(고치기)·DELETE(지우기).	4
본문(body)	요청이나 응답에 실어 보내는 데이터.	4
상태 코드	결과를 알리는 세 자리 숫자. 200·201·204·404·422·500.	4
JSON	데이터를 글자로 적는 형식. `{"title": "장보기"}`.	4
curl	명령 한 줄로 HTTP 요청을 보내는 도구.	4
클라이언트	요청을 보내는 쪽(브라우저, curl, 테스트).	4

데이터베이스

용어	뜻	교시
데이터베이스(DB)	데이터를 디스크에 저장하고 안전하게 읽고 쓰게 해 주는 프로그램. 또는 그 안의 테이블 묶음.	7
PostgreSQL	널리 쓰이는 무료 데이터베이스.	7
테이블 / 행 / 열	표 / 데이터 한 건 / 항목.	7
SQL	데이터베이스에게 일을 시키는 말.	7
psql	PostgreSQL 에 접속해 SQL 을 치는 프로그램.	7
`SERIAL`	1, 2, 3 … 자동으로 번호를 붙이는 타입.	7
`PRIMARY KEY`	행을 구분하는 열. 겹치지 않는다.	7
`NOT NULL`	비워 둘 수 없다는 규칙.	7
`DEFAULT`	값을 안 넣었을 때 쓰는 기본값.	7
`WHERE`	조건에 맞는 행만 고른다. UPDATE·DELETE 에 빠뜨리면 모든 행이 대상이 된다.	7
`RETURNING`	INSERT·UPDATE·DELETE 한 행을 바로 돌려받는다.	7
psycopg	파이썬에서 PostgreSQL 에 접속하는 드라이버.	8
연결(connection)	프로그램과 DB 사이의 통로. 열고, 쓰고, 닫는다.	8
자리표시자(`%s`)	SQL 문장에서 값이 들어갈 자리. 값은 따로 전달한다.	8
SQL 인젝션	사용자가 보낸 글이 SQL 명령의 일부로 실행되는 공격. 자리표시자로 막는다.	7, 8
ORM	SQL 을 파이썬 코드 뒤에 감춰 주는 도구. 이 과정에서는 쓰지 않는다.	8
커밋(DB)	DB 에서 변경을 확정하는 것. Git 의 커밋과 이름만 같다.	8

테스트

용어	뜻	교시
테스트	"이렇게 부르면 이런 답이 와야 한다"를 코드로 적어 둔 것.	3
pytest	파이썬 테스트를 실행하는 도구.	3
`assert`	"이 식이 참이어야 한다". 거짓이면 테스트가 실패한다.	3
fixture	테스트가 쓸 준비물을 만드는 함수. `client` 가 그 예다.	6
`conftest.py`	여러 테스트가 함께 쓰는 준비물을 모아 둔 파일.	3, 6
TestClient	서버를 띄우지 않고 API 를 불러 보는 가짜 브라우저.	3
ruff	코드를 실행하지 않고 읽어서 실수를 찾는 검사 도구.	3, 5
테스트 전용 DB	테스트만 쓰는 데이터베이스(`memo_test`). 진짜 데이터를 지우지 않으려고 따로 둔다.	8
빨강 / 초록	테스트 실패 / 통과. 버그를 고칠 때 빨강을 먼저 확인하고 초록으로 만든다.	11

Git

용어	뜻	교시
Git	파일의 변경 기록을 관리하는 도구.	2, 3
저장소(repository)	Git 이 기록을 관리하는 폴더.	2
clone	인터넷의 저장소를 내 PC 로 내려받는 것.	2
커밋(commit)	지금 상태를 기록으로 남기는 것. 게임의 저장 지점.	3
`git diff`	무엇이 바뀌었는지 보여 준다. `+` 는 추가, `-` 는 삭제.	3
`git add`	이번 커밋에 넣을 파일을 고른다.	3
`.gitignore`	Git 이 무시할 파일 목록.	2
훅(hook)	커밋 같은 일이 일어날 때 Git 이 자동으로 실행하는 스크립트.	5
pre-commit	커밋 직전에 실행되는 훅. 실패하면 커밋이 취소된다.	5
GitHub	Git 저장소를 인터넷에 올려 두는 서비스.	12
push	내 PC 의 커밋을 GitHub 로 보내는 것.	12
원격 저장소(remote)	인터넷에 있는 저장소. `origin` 은 그 별명이다.	12

화면

용어	뜻	교시
HTML	화면의 뼈대를 적는 언어.	10
CSS	화면을 꾸미는 언어.	10
자바스크립트	화면의 동작을 적는 언어.	10
`fetch`	자바스크립트에서 HTTP 요청을 보내는 함수.	10
`await` / `async`	"답이 올 때까지 기다린다" / 그런 기다림이 든 함수라는 표시.	10
이벤트	클릭, 제출 같은 사용자의 행동.	10
`textContent`	글자를 글자 그대로 화면에 넣는다.	10
`innerHTML`	글자를 HTML 로 해석해 넣는다. 사용자 글에 쓰면 위험하다.	10
XSS	사용자가 넣은 글이 다른 사람의 브라우저에서 코드로 실행되는 공격.	10
개발자 도구	브라우저에서 F12 로 여는 창. Network 탭에서 요청을, Console 탭에서 오류를 본다.	10

부록 F. 오류 해결 모음

교시별 안내서의 "자주 만나는 오류"를 한곳에 모으고, 길게 설명해야 하는 것을 더했다.

먼저 할 일 — 오류를 읽는 순서

1. **어디에서 난 오류인가?** 명령을 친 곳이 [PS] 인가 [dev] 인가. 틀린 곳에서 치면 `command not found` 가 난다.
2. **오류 메시지의 마지막 줄을 읽는다.** 원인은 대부분 맨 아래에 있다.
3. **로그를 본다.** 서버 문제는 `docker compose logs app --tail 30`, DB 문제는 `docker compose logs db --tail 30`.
4. **바꾼 것을 되돌려 본다.** `git status` 로 무엇을 바꿨는지 보고, 방금 바꾼 것이 원인인지 확인한다.
5. 그래도 모르면 **오류 전체를 붙여서** AI 나 강사에게 묻는다. "안 돼요"가 아니라 "이 명령을 쳤더니 이 출력이 나왔다".

1. DB 가 켜지지 않는다 (가장 중요)

증상. `docker compose ps` 에서 `db` 가 `healthy` 가 되지 않거나 목록에서 사라진다. `app`, `dev` 도 켜지지 않는다(DB 를 기다리기 때문이다).

확인.

■ [PS] 윈도우 PowerShell

```
docker compose logs db --tail 30
```

경우 A — 윈도우 폴더의 권한 문제

로그에 아래와 비슷한 문구가 보인다.

```
chmod: changing permissions of '/var/lib/postgresql/data': Operation not permitted
FATAL:  data directory "/var/lib/postgresql/data" has wrong ownership
initdb: error: could not change permissions of directory
```

원인. 이 과정은 DB 데이터를 C 드라이브 폴더(`data\postgres`)에 바인드 마운트한다. PostgreSQL 은 데이터 폴더의 소유자와 권한을 엄격하게 검사하는데, PC 의 Docker 설정에 따라 윈도우 폴더에서는 그 검사를 통과하지 못할 수 있다.

해결. 예비 설정으로 바꾼다. DB 데이터를 Docker 가 관리하는 저장 공간(이름 있는 볼륨)에 둔다.

■ [PS] 윈도우 PowerShell

```
docker compose down
Remove-Item -Recurse -Force .\data\postgres
docker compose -f docker-compose.yml -f docker-compose.volume.yml up -d --build
docker compose -f docker-compose.yml -f docker-compose.volume.yml ps
```

`-f` 를 매번 치지 않으려면 `.env` 맨 아래에 한 줄을 추가한다. 그러면 `docker compose up -d` 만 쳐도 두 파일이 함께 쓰인다.

```
COMPOSE_FILE=docker-compose.yml;docker-compose.volume.yml
```

(윈도우에서는 두 파일 이름을 세미콜론 `;` 으로 나눈다.)

달라지는 점.

	기본 설정 (바인드 마운트)	예비 설정 (이름 있는 볼륨)
데이터가 놓이는 곳	`C:\harness-crud-lab\data\postgres`	Docker 내부의 볼륨 `harness-crud-lab_pgdata`
탐색기에서 보이는가	보인다	보이지 않는다
`docker compose down` 뒤에 남는가	남는다	남는다
완전히 지우려면	폴더를 지운다	`docker compose down -v`

소스 폴더(`C:\harness-crud-lab` ↔ `/workspace`)의 바인드 마운트는 그대로다. 7교시 8단계의 "컨테이너를 지워도 데이터가 남는다" 실험도 그대로 할 수 있다. `dir data\postgres` 로 폴더를 들여다보는 부분만 건너뛴다.

볼륨이 있는지 보는 명령:

■ [PS] 윈도우 PowerShell

```
docker volume ls
```

경우 B — 데이터 폴더가 반쯤 만들어진 채 남았다

로그에 `directory "/var/lib/postgresql/data" exists but is not empty` 가 보인다. 처음 켜다가 중간에 끊긴 경우다.

■ [PS] 윈도우 PowerShell

```
docker compose down
Remove-Item -Recurse -Force .\data\postgres
docker compose up -d
```

경우 C — .env 의 계정을 중간에 바꿨다

로그에 `password authentication failed` 또는 `role "…" does not exist` 가 보인다. DB 계정은 **데이터 폴더를 처음 만들 때 한 번만** 정해진다. 그 뒤에 `.env` 의 `POSTGRES_USER`·`POSTGRES_PASSWORD` 를 바꿔도 반영되지 않는다.

`.env` 의 값을 처음 값(`memo` / `memo1234`)으로 되돌린다. 계정을 꼭 바꿔야 하면 경우 B 처럼 데이터 폴더를 지우고 새로 만든다(메모가 모두 사라진다).

2. Docker

증상	원인과 해결
`docker : 용어가 … 인식되지 않습니다`	Docker Desktop 이 설치되지 않았거나, 설치 뒤 PowerShell 을 새로 열지 않았다.
`error during connect … docker_engine … cannot find the file`	Docker Desktop 이 실행 중이 아니다. 시작 메뉴에서 실행하고 "Engine running" 을 기다린다.
Docker Desktop 이 `WSL update` 를 요구한다	관리자 PowerShell 에서 `wsl --update` 를 실행하고 PC 를 다시 시작한다.
`port is already allocated` (8000)	다른 프로그램이 8000 번을 쓴다. 그 프로그램을 끄거나 `docker-compose.yml` 의 `"8000:8000"` 을 `"8001:8000"` 으로 바꾼다(브라우저 주소도 8001).
`no configuration file provided: not found`	`docker-compose.yml` 이 없는 폴더에서 쳤다. `cd C:\harness-crud-lab` 을 먼저 한다.
빌드 중 OpenCode 설치 단계에서 실패	인터넷이나 보안 프로그램이 내려받기를 막았다. `docker compose build --no-cache` 로 다시 해 본다.
빌드가 매우 느리다	처음 한 번은 이미지를 내려받느라 몇 분 걸린다. 두 번째부터는 빠르다.
`service "dev" is not running`	컨테이너가 꺼져 있다. `docker compose up -d` 뒤 `docker compose ps`.
디스크가 부족하다	쓰지 않는 이미지를 지운다: `docker image prune`.

3. 파일이 반영되지 않는다 (바인드 마운트)

증상	원인과 해결
윈도우에서 고친 파일이 컨테이너에서 그대로다	저장하지 않았다(VS Code 탭의 흰 점). 또는 `C:\harness-crud-lab` 이 아닌 다른 사본을 고치고 있다. VS Code 창의 제목에서 폴더 경로를 확인한다.
파일을 고쳤는데 서버가 다시 켜지지 않는다	`app` 폴더 밖의 파일은 서버가 살펴보지 않는다. `app` 안의 파일인데도 그렇다면 `docker compose restart app`.
`/workspace` 가 비어 있다	`docker compose` 를 다른 폴더에서 실행했다. `docker inspect harness-crud-lab-dev-1 --format "{{ json .Mounts }}"` 의 `Source` 를 확인한다.
`.sh` 스크립트가 `not found` 또는 `\r` 오류를 낸다	줄바꿈이 윈도우 방식(CRLF)으로 바뀌었다. `.gitattributes` 가 있는지 확인하고, VS Code 오른쪽 아래의 `CRLF` 를 눌러 `LF` 로 바꾼 뒤 저장한다.
저장소를 OneDrive·바탕 화면 아래에 두었다	동기화 프로그램과 겹쳐 파일이 잠기거나 느려진다. `C:\harness-crud-lab` 으로 옮긴다.

4. OpenRouter 와 OpenCode

증상	원인과 해결
`echo ${#OPENROUTER_API_KEY}` 가 `0`	`.env` 를 고친 뒤 컨테이너를 다시 만들지 않았다. **[PS]** `docker compose up -d` 뒤 dev 에 다시 들어간다. 파일 이름이 `.env.txt` 가 아닌지, 등호 앞뒤에 공백이 없는지 본다.
`401`	키가 틀렸다. 앞뒤 공백, 따옴표, 빠진 글자를 본다. 키를 지웠다면 새로 만든다.
`402`	크레딧이 없거나 키의 한도를 다 썼다. OpenRouter 의 Credits·Keys 화면을 본다.
`404` (모델)	모델 이름이 틀렸거나 없어졌다. https://openrouter.ai/models 에서 이름을 확인한다.
`429`	요청이 너무 잦다. 잠시 뒤 다시 한다. `:free` 모델은 하루 한도가 작다.
OpenCode 가 모델을 찾지 못한다	`opencode.jsonc` 의 `"model"` 이 `openrouter/` 로 시작하는지 본다. OpenCode 안에서 `/models` 로 목록을 본다.
OpenCode 가 설정 오류를 낸다	`opencode.jsonc` 의 쉼표·중괄호가 깨졌다. `cp harness/opencode.jsonc opencode.jsonc` 로 다시 가져온다.
OpenCode 가 `AGENTS.md` 를 따르지 않는 것 같다	파일이 맨 위 폴더에 있는지 본다. OpenCode 는 시작할 때 읽으므로 `/exit` 뒤 다시 연다. 대화가 길어졌으면 `/new` 로 새로 시작한다.
권한 규칙이 기대대로 막지 않는다	설정을 바꾼 뒤 OpenCode 를 다시 열었는지 본다. 그래도 그대로면 강사에게 알린다. 규칙의 `resource` 무늬가 맞지 않는 경우다.
AI 의 답이 중간에 끊긴다	네트워크나 사용 한도 문제다. 같은 요청을 다시 보낸다.
OpenCode 화면의 글자가 깨진다	Windows Terminal 을 쓴다(시작 메뉴에서 "터미널"). 창이 너무 작으면 넓힌다.

모델을 바꾸려면. 두 군데를 바꾼다.

- `ask_ai.py` 가 쓰는 모델: `.env` 의 `OPENROUTER_MODEL` → **[PS]** `docker compose up -d`
- OpenCode 가 쓰는 모델: `opencode.jsonc` 의 `"model"` (`openrouter/` 뒤에 모델 이름) → OpenCode 를 다시 연다

5. 서버 (FastAPI)

증상	원인과 해결
브라우저가 "연결할 수 없음"	코드 오류로 서버가 켜지지 못했다. `docker compose logs app --tail 30` 에서 `Error` 가 든 줄과 줄 번호를 찾는다.
`SyntaxError`, `IndentationError`	오타나 들여쓰기 문제. 로그가 알려 주는 파일과 줄을 본다.
`ModuleNotFoundError: No module named 'app.…'`	파일 이름이 다르거나 `app/__init__.py` 가 없다.
`{"detail":"Not Found"}` (404)	그 주소에 연결된 함수가 없다. 주소 오타, 저장 안 함.
`{"detail":"Method Not Allowed"}` (405)	주소는 있는데 그 방식의 함수가 없다.
422	요청이 `MemoIn` 의 모양·규칙에 맞지 않는다. 응답의 `loc` 과 `msg` 를 읽는다.
500	서버 코드가 오류를 냈다. `docker compose logs app --tail 30` 의 `Traceback` 마지막 줄을 읽는다.
메모가 자꾸 사라진다 (6교시까지)	메모리 저장소라서 서버가 다시 켜질 때마다 비워진다. 정상이다.

6. 데이터베이스 연결 (8교시부터)

증상	원인과 해결
`Name or service not known`, `could not translate host name "db"`	컨테이너 밖에서 실행했거나, 주소의 호스트를 `localhost` 로 적었다. 컨테이너에서는 `db` 다.
`connection refused`	DB 가 꺼져 있다. `docker compose ps` 로 `db` 가 `healthy` 인지 본다.
`KeyError: 'DATABASE_URL'`	환경 변수가 없다. dev 컨테이너 안에서 실행한다.
`relation "memos" does not exist`	테이블을 만드는 코드가 실행되지 않았다. `lifespan` 과 `init_db` 를 본다.
`database "memo_test" does not exist`	`conftest.py` 가 테스트 DB 를 만들지 않는다. 급하면 **[PS]** `docker compose exec db psql -U memo -d memo_db -c "CREATE DATABASE memo_test;"`
`the query has N placeholders but M parameters were passed`	`%s` 개수와 값 개수가 다르다. 값이 하나일 때 `(memo_id,)` 의 쉼표를 본다.
테스트를 돌리면 진짜 메모가 사라진다	테스트가 `memo_db` 에서 돈다. `conftest.py` 가 `TEST_DATABASE_URL` 로 바꿔 끼우는지 본다.

DB 를 처음 상태로 되돌리려면. 메모가 모두 사라진다.

■ [PS] 윈도우 PowerShell

```
docker compose down
Remove-Item -Recurse -Force .\data\postgres
docker compose up -d
```

메모만 비우려면: **[PS]** `docker compose exec db psql -U memo -d memo_db -c "TRUNCATE TABLE memos RESTART IDENTITY;"`

7. 테스트와 검증

증상	원인과 해결
`pytest: command not found`	윈도우에서 쳤다. dev 컨테이너에서 친다.
`fixture 'client' not found`	`tests/conftest.py` 가 없거나 `client` 함수가 없다.
따로 돌리면 통과, 함께 돌리면 실패	테스트끼리 데이터를 공유한다. `conftest.py` 가 테스트마다 저장소를 비우는지 본다.
`ruff` 가 `F401 imported but unused`	가져오고 쓰지 않은 import 가 있다. 그 줄을 지운다.
`ruff` 가 `F821 undefined name`	정의하지 않은 이름을 썼다. 오타이거나 import 를 빠뜨렸다.
`verify.sh` 는 통과하는데 동작이 요구사항과 다르다	그 요구사항을 확인하는 테스트가 없다. 테스트를 추가한다(11교시).

8. Git

증상	원인과 해결
`Please tell me who you are`	이름·메일을 정하지 않았다. `git config user.name "이름"`, `git config user.email "메일"`.
`detected dubious ownership`	컨테이너의 Git 이 폴더를 믿지 못한다. 이미지에 설정이 들어 있으니 `docker compose up -d --build` 로 다시 만든다.
커밋이 취소된다	`pre-commit` 이 검증 실패를 잡았다. `[pre-commit]` 위의 출력을 읽고 고친다.
커밋할 때 `[pre-commit]` 이 나오지 않는다	훅이 꺼져 있다. `git config core.hooksPath .githooks`.
윈도우에서 커밋하면 훅이 실패한다	윈도우에서는 훅이 dev 컨테이너에게 검증을 시킨다. 컨테이너가 켜져 있어야 한다. 커밋은 dev 컨테이너에서 하는 것을 권한다.
아무것도 안 고쳤는데 모든 파일이 바뀐 것으로 나온다	줄바꿈 설정이 어긋났다. `.gitattributes` 가 있는지 본다.
`git diff` 화면에서 빠져나오지 못한다	`q` 를 누른다.
실수로 `git restore .` 를 했다	커밋하지 않은 변경은 되살릴 수 없다. VS Code 에 파일이 열려 있었다면 Ctrl+Z 를 해 본다.
`git push` 가 `403`	`origin` 이 강사 저장소다. 12교시 5단계를 한다.

9. 화면 (10교시부터)

증상	원인과 해결
고쳤는데 화면이 그대로다	브라우저가 옛 파일을 기억한다. **Ctrl+F5**.
꾸밈이 없고 버튼이 동작하지 않는다	`app.js·style.css` 가 404 다. F12 → Network 에서 확인하고 경로를 본다.
버튼을 눌러도 아무 일이 없다	F12 → Console 의 빨간 글씨를 읽는다.
저장하면 화면이 깜빡이고 주소에 `?` 가 붙는다	`event.preventDefault()` 가 없다.
시각이 9시간 어긋난다	서버는 세계 표준시로 준다. 화면에서 `toLocaleString("ko-KR")` 로 바꾼다.

그래도 안 될 때 — 처음부터 다시

소스는 Git 에, 하네스는 `harness/` 에 있으므로 실행 환경은 언제든 새로 만들 수 있다.

■ [PS] 윈도우 PowerShell

```
cd C:\harness-crud-lab
docker compose down
docker compose up -d --build
docker compose ps
```

코드가 엉켰으면 그 교시의 시작 상태로 맞춘다. (어느 체크포인트인지는 `commands.md` 9절의 표를 본다.)

■ [dev] dev 컨테이너

```
sh scripts/checkpoint.sh 08 --force
sh scripts/verify.sh
```